# [16] Neural Networks

# Tensorflow

# Deep Learning

# Welcome to the Build Deep Learning Models with TensorFlow Skill Path

**Diving into deep learning!**

Welcome to the Build Deep Learning Models with TensorFlow skill path! Let's go over what you will find along this path. There are four main units:

In each of these sections, you will dive into the concepts of deep learning and be prepared to put your skills to the test with fun quizzes and projects.
After this Path, you will be able to:

You will demonstrate your knowledge in a Portfolio Project at the end of the Path. You can complete the Portfolio Project either in parallel with or after taking the prerequisite content—it's up to you!
Throughout this path you may see some items that aren't made by the Codecademy team. We've included these to ensure that you learn all the topics you need to get your career started. Though we didn't write them, we've vetted them ourselves for technical accuracy and good teaching practices. When possible, we've written relevant assessments so that you can check your understanding back on our platform.
We're excited for you to begin this journey!

# Additional Resources for Build Deep Learning Models with TensorFlow Skill Path

**Resources that can help along your journey**

Congratulations on enrolling in the Build Deep Learning Models with TensorFlow Skill Path. We're excited that you're here. In order to successfully take part in your learning journey, we've provided you with a set of resources.

## Learning to Code with Others

Learning a programming language can be like learning a spoken language. It's an incredible feeling to understand when someone speaks to you! But you are empowered to do more when you are able to begin speaking the new language with others.
Coding languages work similarly. To gain mastery, you must also practice using the language. You can begin practicing what you've learned by visiting the Codecademy Community forums. In the forums, you will be able to see new posts from learners like you and test your skills by posting a response to help them.
The Community forums are a great place to read about others' experiences, ask questions, and get help from your peers when you are stuck. They are also a safe place to test out your new skills by helping someone else with their question.

## Join a Chapter, Learn Together

Looking for other learners to study with? Codecademy Chapters are a great way to team up and reach your goals. Chapters meet virtually at a set time, and are peer-led by a Codecademy learner, just like you. Each Chapter will have a focus, ranging from web development projects, to mobile development, to technical interview preparedness. Find a Chapter near you or start your own.

## Additional Resources

Here are some additional resources to check out that will help you throughout your Path:

- Book: Deep Learning with Python, François Chollet
- Documentation: Keras Library
- Documentation: Tensorflow Library
- Book: Algorithms of Oppression: How Search Engines Reinforce Racism, Safiya Umoja Noble
- Book: Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy, Cathy O'Neil

Best of luck!

# Introduction: Foundations of Deep Learning and Perceptrons

**Getting started with the fundamentals**

## Foundations of Deep Learning and Perceptrons

### Goals of this Unit

The goal of this unit is to grasp the foundational concepts of deep learning (DL). Through articles and interactive applets, you will gain an insight into the fundamentals of DL and move forward with knowledge of a powerful and widely used machine learning method. You will then investigate perceptrons as a prelude to the development of deep learning models to get yourself acquainted with the basic structure of a neural network in Python.
After this unit, you will be able to:

- Explain what deep learning is to a friend

Know different use cases for deep learning models
Trace the beginning-to-end path of data that journeys a neural network and understand the inner workings behind the journey
Decipher potential dangers for the use cases that involve deep learning models
Develop your own perceptron algorithms

Happy coding!

# What Is Deep Learning?

**A quick overview of deep learning and its applications**

## What is Deep Learning?

Have you ever wondered what powers ChatGPT? What technology developers are using to create self-driving cars? How your phone can recognize faces? Why it seems like your photos app is better at recognizing your friends' faces than even you can? What is behind all of this? Is it magic? Well, not exactly; it is a powerful technology called *deep learning (DL)*. Let's dive in and see what this is all about!

## Deep Learning vs. Machine Learning

First, let's focus on *learning*. If you have come across *machine learning* before, you might be familiar with the concept of a *learning model*. Learning describes the process by which models analyze data and finds patterns. A machine learning algorithm learns from patterns to find the best representation of this data, which it then uses to make predictions about new data that it has never seen before.

Deep learning is a subfield of machine learning, and the concept of *learning* is pretty much the same.

Deep learning models are used with many different types of data, such as text, images, audio, and more, making them applicable to many different domains.

## What does “deep” mean?

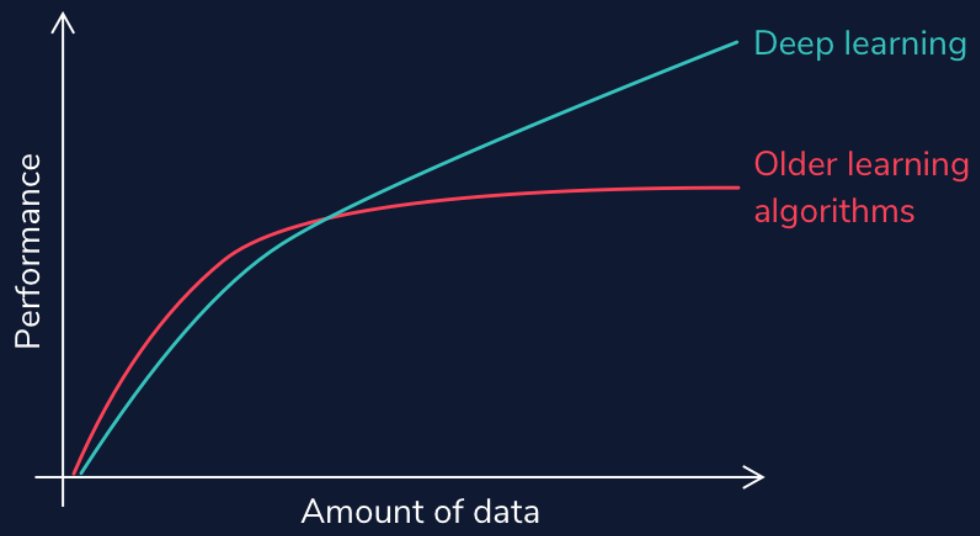
That leaves the question: what is this “deep” aspect of deep learning? It separates deep learning from typical machine learning models and why it is a powerful tool that is becoming more prevalent in today’s society.

The deep part of deep learning refers to the numerous “layers” that transform data. This architecture mimics the structure of the brain, where each successive layer attempts to learn progressively complex patterns from the data fed into the model. This may seem a bit abstract, so let’s look at a concrete example, such as facial recognition. With facial recognition, a deep learning model takes in a photo as an input, and numerous layers perform specific steps to identify whose face is in the picture. The steps taken by each layer might be the following:


This structure of many abstract layers makes deep learning incredibly powerful. Feeding high volumes of data into the model makes the connections between layers more intricate. Deep learning models tend to perform better with more massive amounts of data than other learning algorithms.

## High Volume of Data

Notice that without large amounts of data, deep learning models are no more powerful (and maybe even less accurate) than less complex learning models. However, with large amounts of data, deep learning models can improve performance to the point that they outperform humans in tasks such as classifying objects or faces in images or driving. Deep learning is fundamentally a future learning system (also known as representation learning). It learns from raw data without human intervention. Hence, given massive amounts of data, a deep learning system will perform better than traditional machine learning systems that rely on feature extractions from developers.




Autonomous vehicles, such as Tesla, have to process thousands, even millions of stop signs, to understand that cars are supposed to stop when they see a red hexagon with “Stop” written on them (and this is only for U.S. stop signs!). Think about the enormous number of situations a self-driving vehicle must train on to ensure safety.

## With Deep Learning Comes Deep Responsibility

Even beyond identifying objects, deep learning models can generate audio and visual content that is deceivingly real. They can modify existing images, such as in this cool applet that allows you to add in trees or alter buildings in a set of photos. However, they can have a darker side. DL models can produce artificial media in which the identity of someone in an image, video, or audio is replaced with someone else. These are known as *deepfakes*, and they can have scary implications, such as financial fraud and the distribution of fake news and hoaxes.

## Graphics Processing Units

One final thing to note about deep learning is that with large amounts of data and layers of complexity, you may imagine that this takes a lot of time and processing power. That intuition would be correct. These models often require high-performance *GPUs* (graphics processing

units) to run in a reasonable amount of time. These processors have a large memory bandwidth and can process multiple computations simultaneously. CPUs can run deep learning models as well; however, they will be much slower.

The development of GPUs has been critical to the success of deep learning. It is interesting to note that one of the driving factors for this development was not the need for better deep learning tools, but the demand for better video game graphics. It just so happened that GPUs are perfect for processing large datasets. This makes them a perfect tool for learning models and has put deep learning specifically at the forefront of machine learning conversations.

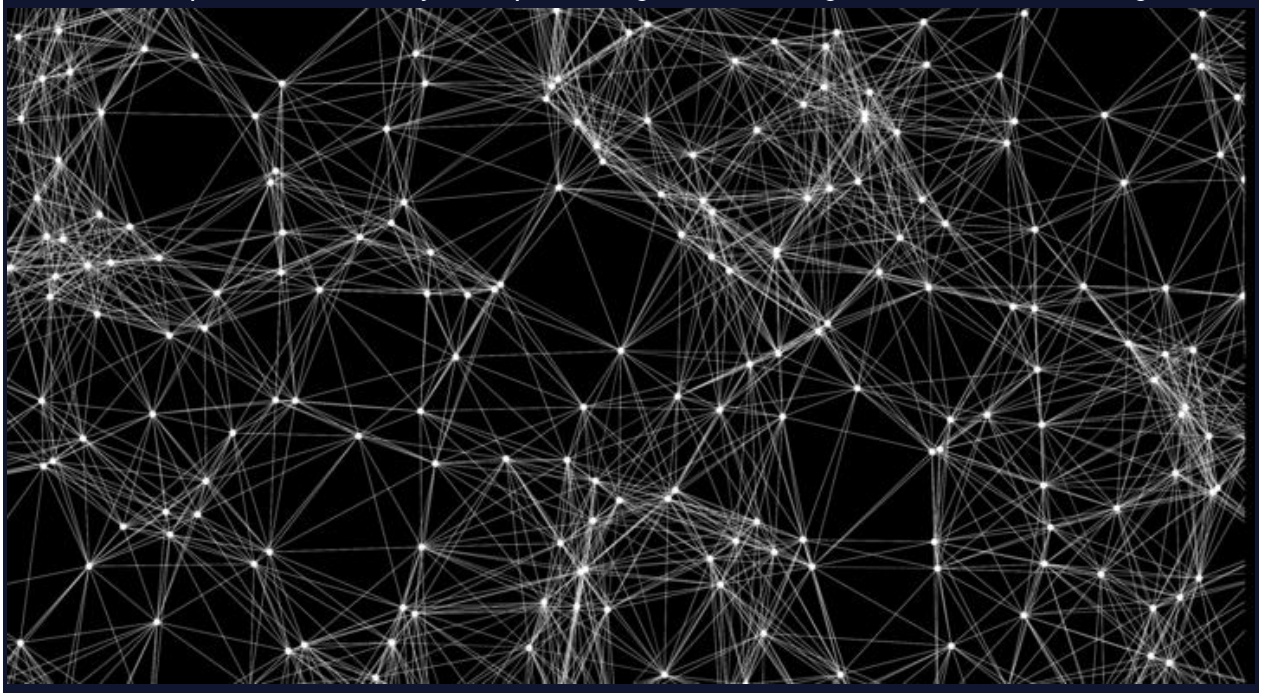
We have only just begun our dive into deep learning. Let's keep digging in and see where our journey takes us!

# What are Neural Networks?

**An artificial neural network is an interconnected group of nodes, an attempt to mimic to the vast network of neurons in a brain.**

**Understanding Neural Networks**

As you are reading this article, the very same brain that sometimes forgets why you walked into a room is magically translating these pixels into letters, words, and sentences — a feat that puts the world's fastest supercomputers to shame. Within the brain, thousands of neurons are firing at incredible speed and accuracy to help us recognize text, images, and the world at large.



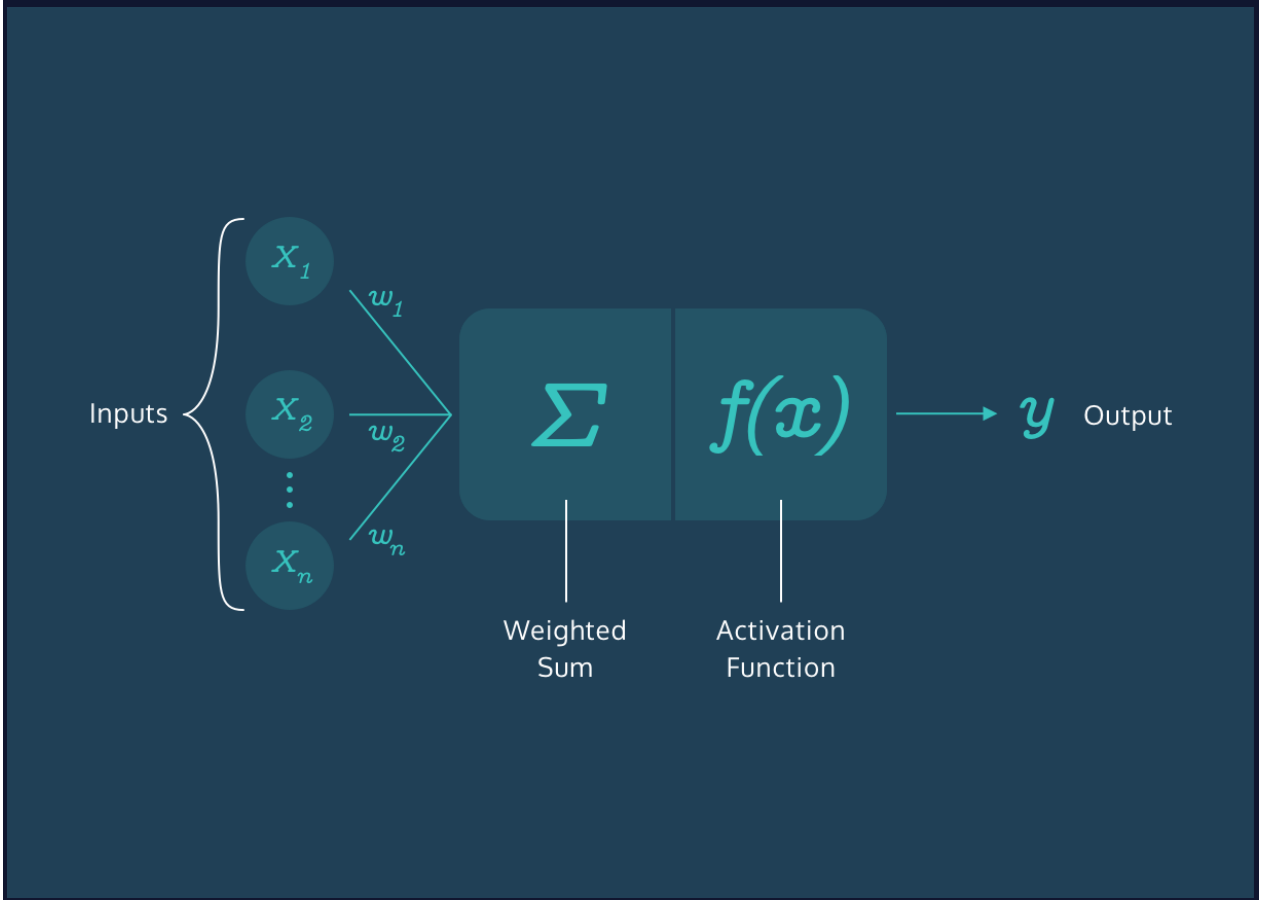
A **neural network** is a programming model inspired by the human brain. Let's explore how it came into existence.

**The Birth of an Artificial Neuron**

Computers have been designed to excel at number-crunching tasks, something that most humans find terrifying. On the other hand, humans are naturally wired to effortlessly recognize objects and patterns, something that computers find difficult.

This juxtaposition brought up two important questions in the 1950s:

In 1957, Frank Rosenblatt explored the second question and invented the **Perceptron** algorithm that allowed an artificial neuron to simulate a biological neuron! The artificial neuron could take in an input, process it based on some rules, and fire a result. But computers had been doing this for years — what was so remarkable?

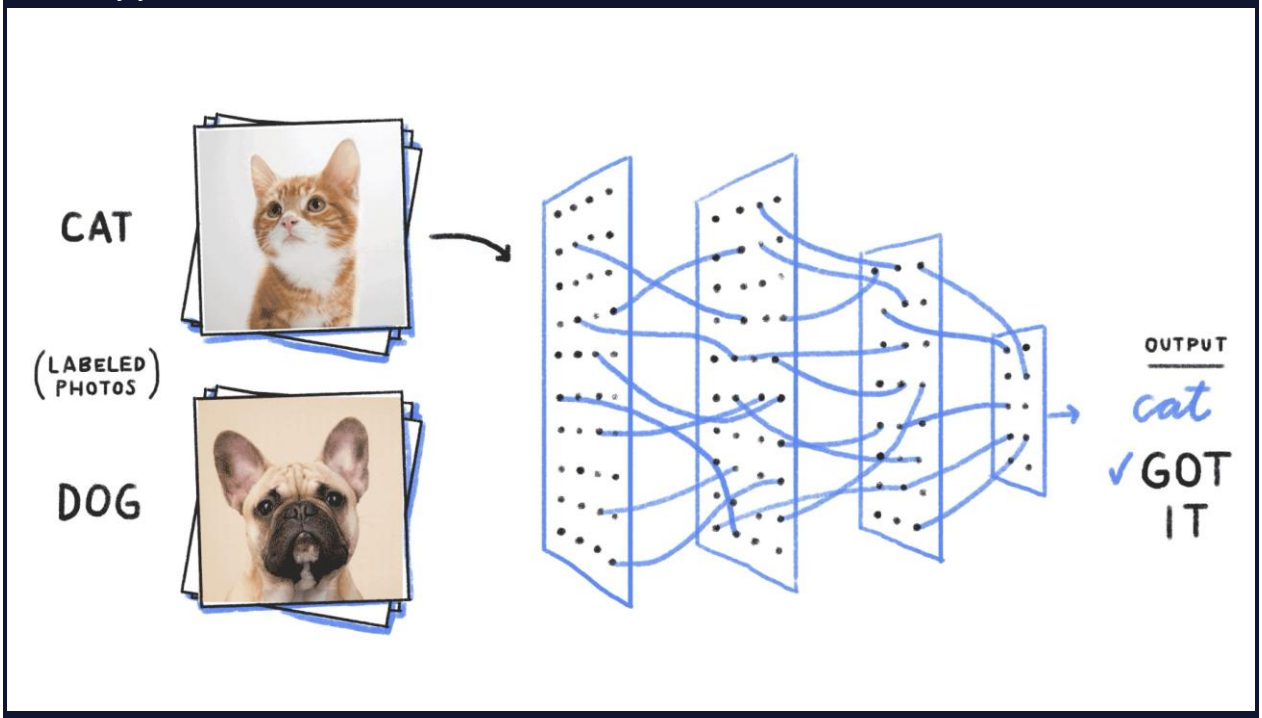



There was a final step in the Perceptron algorithm that would give rise to the incredibly mysterious world of Neural Networks — the artificial neuron could *train itself based on its own results, and fire better results in the future*. In other words, it could learn by trial and error, just like a biological neuron.

**More Neurons**

The Perceptron Algorithm used multiple artificial neurons, or perceptrons, for image recognition tasks and opened up a whole new way to solve computational problems. However, as it turns out, this wasn't enough to solve a wide range of problems, and interest in the Perceptron Algorithm along with Neural Networks waned for many years.

But many years later, the neurons fired back.




It was found out that creating multiple layers of neurons — with one layer feeding its output to the next layer as input — could process a wide range of inputs, make complex decisions, and still produce meaningful results. With some tweaks, the algorithm became known as the **Multilayer Perceptron**, which led to the rise of Feedforward Neural Networks.

**60 Years Later…**

With Feedforward Networks, the results improved. But it was only recently, with the development of high-speed processors, that neural networks finally got the necessary computing power to seamlessly integrate into daily human life.

Today, the applications of neural networks have become widespread — from simple tasks like speech recognition to more complicated tasks like self-driving vehicles.

In 2012, Alex Krizhevsky and his team at University of Toronto entered the ImageNet competition (the annual Olympics of computer vision) and trained a **deep convolutional neural network** [pdf]. No one truly understood how it made the decisions it did, but it worked better than any other traditional classifier, by a huge 10.8% margin.

**Summary**

The neurons have come a long way. They have braved the AI winter and remained patient amidst the lack of computing power in the 20th century. They have now taken the world by storm and deservedly so.

Neural networks are ridiculously good at generating results but also mysteriously complex; the apparent complexity of the decision-making process makes it difficult to say exactly how neural networks arrive at their superhuman level of accuracy.
Let's dive into the world of Neural Networks and relish in all its mystery!

**Deep Learning Math**

# Review

This overview completes the necessary mathematical intuition you need to move forward and begin coding your own learning models! To recap all the things we have learned (so many things!!!):

- Scalars, vectors, matrices, and tensors
  - A *scalar* is a singular quantity like a number.
  - A *vector* is an
  - array
  - Preview: Docs Loading link description
  - of numbers (scalar values).
  - A *matrix* is a grid of information with rows and columns.
  - A *tensor* is a multidimensional array and is a generalized version of a vector and matrix.
- Matrix Algebra
  - In *scalar multiplication*, every entry of the matrix is multiplied by a scalar value.
  - In *matrix addition*, corresponding matrix entries are added together.
  - In *matrix multiplication*, the dot product between the corresponding rows of the first matrix and columns of the second matrix is calculated.
  - A *matrix transpose* turns the rows of a matrix into columns.
- In *forward propagation*, data is sent through a neural network to get initial outputs and error values.
  - *Weights* are the learning parameters of a deep learning model that determine the strength of the connection between two nodes.
  - A *bias node* shifts the activation function either left or right to create the best fit for the given data in a deep learning model.
  - *Activation Functions* are used in each layer of a neural network and determine whether neurons should be "fired" or not based on output from a weighted sum.
  - *Loss functions* are used to calculate the error between the predicted values and actual values of our training set in a learning model.
- In *backpropagation*, the gradient of the loss function is calculated with respect to the weight parameters within a neural network.

*Gradient descent* updates our weight parameters by iteratively minimizing our loss function to increase our model's accuracy.
*Stochastic gradient descent* is a variant of gradient descent, where instead of using all data points to update parameters, a random data point is selected.
*Adam optimization* is a variant of SGD that allows for adaptive learning rates.
*Mini-batch gradient descent* is a variant of GD that uses random batches of data to update parameters instead of a random datapoint.

This has been a ton of material, so do not fret if you are confused or still wrapping your head around some of the concepts. Many of this will be reviewed as you begin coding models. Get excited to see how these concepts are powered with TensorFlow!

# Dangers of the Black Box

**Deep learning models have deep implications.**

## What Makes Deep Learning Models Dangerous?

When talking about machine learning, deep learning, and artificial intelligence, people tend to focus on the progress and amazing feats we could potentially achieve. While it is true that these disciplines have the potential to change the world we live in and allow us to perform otherwise impossible feats, there are often unintended consequences.

We live in an imperfect world, and the learning algorithms we design are not immune to these imperfections. Before we dive into creating our models, we must review some of the issues and implications that they can have on people's lives. Deep Learning models can be especially scary as they are some of the most powerful, and they are becoming more commonplace in society. They are also most often *black boxes* (hard for users to understand as to why and how specific results occur).

## When to Use Them

Due to this nature of deep learning models, we should only ever use them if there is an apparent, significant reason for using one. If there is not a clear reason, we can use basic machine learning or statistical analysis approaches depending on what suffices.

When choosing solutions to a problem, we have to juggle many numerous factors, including

- speed
- accuracy
- training time
- interpretability
- maintenance
- enhancement

size of the trained model

along with even more before beginning to prototype. Asking questions along these lines is vital because we design learning algorithms that can harm individuals or even entire communities in their daily lives if we do not craft them with care and awareness.

## Misconceptions

There are often misconceptions about AI and deep learning models and what implications they have on our world. Science fiction has illustrated the dangers as some sort of robot apocalypse where humans are outsmarted and overpowered. This depiction does not reflect the real risks that are already present from the growing dependence on learning models in our everyday lives. Healthcare, banking, and the criminal justice system have all seen a massive rise in the reliance on learning algorithms to make decisions in recent years. While these have led to increased efficiency and developments, trusting these systems to make high-risk decisions has also led to various risks.

## Key Issues

There are some key things to address about machine learning models that can lead to potentially problematic implications. Let's dive into this through the lens of healthcare.

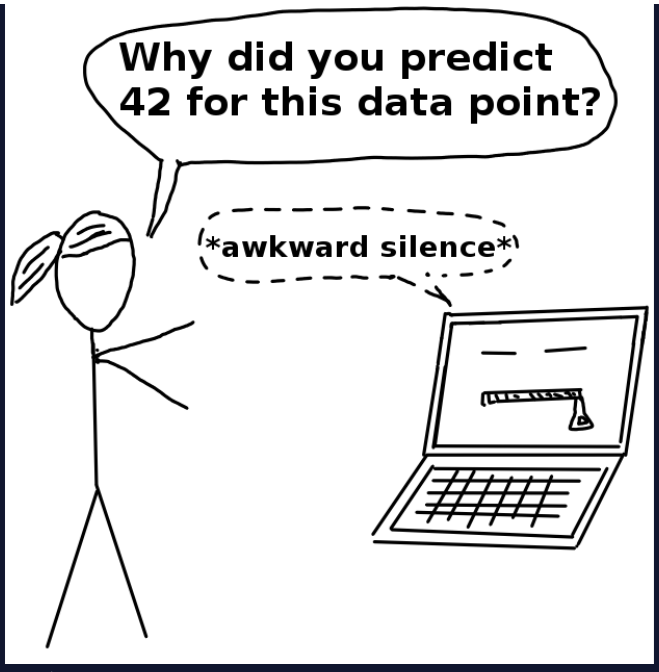
**Machine learning algorithms can only be as good as the data it is trained on**. If we train the data on a model that does not match environmental data well, accuracy will not translate into the real world. A good example is this study on personalized risk assessments for acute kidney injury. While patient data evolved and disease patterns changed, the predictive model became less accurate and, therefore, much less useful. It is crucial for developers to monitor outputs periodically and account for data-shift problems over time. The work is not complete after we deploy a model; it must be managed and continuously refined to account for continuous environmental developments.

There is also a history of the health industry not including enough women and people of color in medical studies. Different demographics can have unique manifestations and risk factors with diseases. If training data is not diverse and representative of all potential sample individuals, inaccuracies and misdiagnoses could occur. It is vital to have orthogonal data in that it is both high in volume and diversity. Without attention to this, social health disparities that are already present could become widened.

**Machine learning models do not understand the impact of a false negative vs. a false positive diagnostic (at least not like humans can)**. When diagnosing patients, doctor's often "err on the side of caution." For example, being falsely diagnosed with a malignant tumor (false positive) is much less dangerous than being incorrectly diagnosed with a benign tumor (false negative). Further testing would occur in the former situation, whereas the latter could cause much scarier consequences with a malignant tumor being unaddressed.

Models may not have this "err on the side of caution" attitude, especially if we do not design them with this implication in mind. If we solely train it to be as accurate as possible, it is at the expense of missing malignant diagnoses. Developers can create custom loss functions that can account for the implications of false negatives vs. false positives. However, to do this, they must understand the domain well.

**For many of the clinicians and the patients, the models are a black box.** Stakeholders only can make decisions based on the outcome, and the predictions are not open to inspection. Therefore, it is hard for anyone to determine that there are patterns of inaccurate predictions until prolonged use has already occurred. For example, imagine an X-ray analysis model that is inaccurate under certain conditions because it was not present in the training data. The doctor would not identify this until continuously observing incorrect diagnoses because the only aspect of the model focuses on is the outcome.

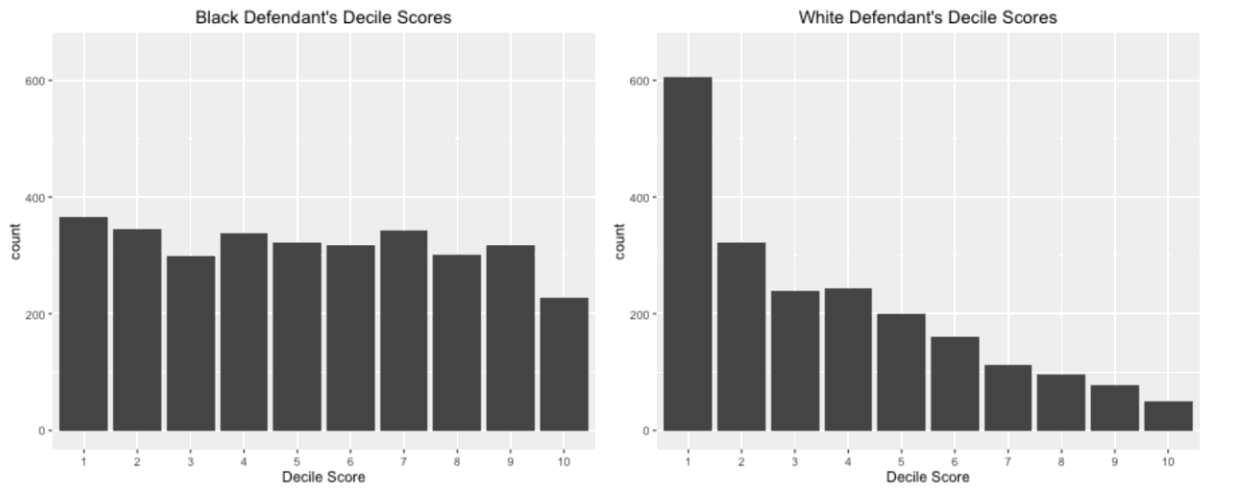



(*Source: Christoph Molnar*)

## More Examples

This is just the tip of the iceberg for concerns about the use of learning models in the healthcare industry, and we have yet to even go into implications within other sectors. Facial recognition has shown implicit bias within Google's algorithm, which incorrectly identified a woman and her friend as gorillas. An error like this leads to emotional harm, and it shows that learning models are not immune to social issues and can reproduce or even exacerbate them.

Employing black box technology becomes more of an issue when used in contexts without transparency. For example, in criminal justice or banking, biased data is used to deny people of color loans at a higher rate or label them as "high risk" repeat offenders. A real-life example of this is a machine learning questionnaire algorithm known as COMPAS (Correctional Offender Management Profiling for Alternative Sanctions). It was used to determine the risk of whether an arrestee would being a repeat offender. A study showed that this algorithm was racially-biased despite never explicitly asking about race. Individuals were asked questions that modeled existing social inequalities, and minorities, particularly blacks, were more likely to be labeled "high-risk" repeat offenders. The graph below shows the distribution of risk scores for black defendants and white defendants. You can read more about the analysis of this report here.




(Source: *ProPublica*)

Models like these can even go beyond mirroring existing inequity. They can go onto perpetuate them and contribute to vicious cycles. For example, if we were to use systems like these to determine patrol routes, this could bias crime data for individuals in those communities. As we add more data to the learning model, the problem is exacerbated. What is even scarier about these models is that they are often beyond dispute or appeal. Given the result of a mathematical formula, we usually take it as fact. Someone who ends up negatively affected by this cannot argue against it or reason with a machine. They cannot explain the full reality to a computer. Instead, a machine will define its own reality to justify its results. While an outside observer can question, “why did this individual get a high-risk score despite only a minor crime?” a machine is merely operating under its historical data and findings.

## Personal Responsibility

This is not to say that we should not use machine learning models in ways that impact everyday lives. It is meant to outline some of the issues that can arise when used to make high-stakes decisions and how they can cause harm. As you move forward to developing your own neural networks, consider the social implications of what you have made. As developers, we must work to ensure that our models are free from bias and will not misrepresent certain populations. The Institute for Ethical AI and Machine Learning has a framework for developing responsible machine learning that we encourage you to read up on.

## Interpretability and Transparency

It is also imperative that the models we build are used in ways that are transparent to potential stakeholders. If someone is impacted due to the output of a learning model, they should understand:

The final thing to consider is making your model’s inner workings understandable for your users, also known as *interpretable machine learning*. Making the inner workings of a model

understandable allows users to identify inaccuracies earlier and explain results to potential stakeholders. A great example of this is a simple classifier model that identifies huskies vs. wolves. In the case below, a husky in incorrectly classified as a wolf. With the implementation of a system called LIME, predictions are explained and can be understood by users. In this case, the explanation is that if a picture contains snow, it will be classified as a wolf. This information gives developers an indicator of why their model contains inaccuracies and clarifies how to improve it.

(a) Husky classified as wolf (b) Explanation

(Source: *arXiv.org*)

## In Conclusion

There is no doubt that machine learning can help us make waves of progress. However, in a world riddled with inequity, developers must attempt design systems so that no one is left behind. By designing learning models that account for limitations of interpretability and likelihood of bias, we can take strides to ensure that individuals and communities are not unjustly harmed.

**recommended reading**

**Read the following:**

## Weapons of Math Destruction: How Big Data Increases Inequality and Threatens Democracy by Cathy O'Neil

### Read Now

In this book, you will learn how mathematical models of data are used in damaging ways within banking, education, insurance, criminal justice, employment, and advertising. This is helpful if you wish to mitigate bad data science and its impact.

Read the following selection: Weapons of Math Destruction, Cathy O'Neil, Chapter 1 excerpt.

**recommended reading**

**Read the following:**

## Algorithms of Oppression: How Search Engines Reinforce Racism by Safiya Umoja Noble

### Read Now

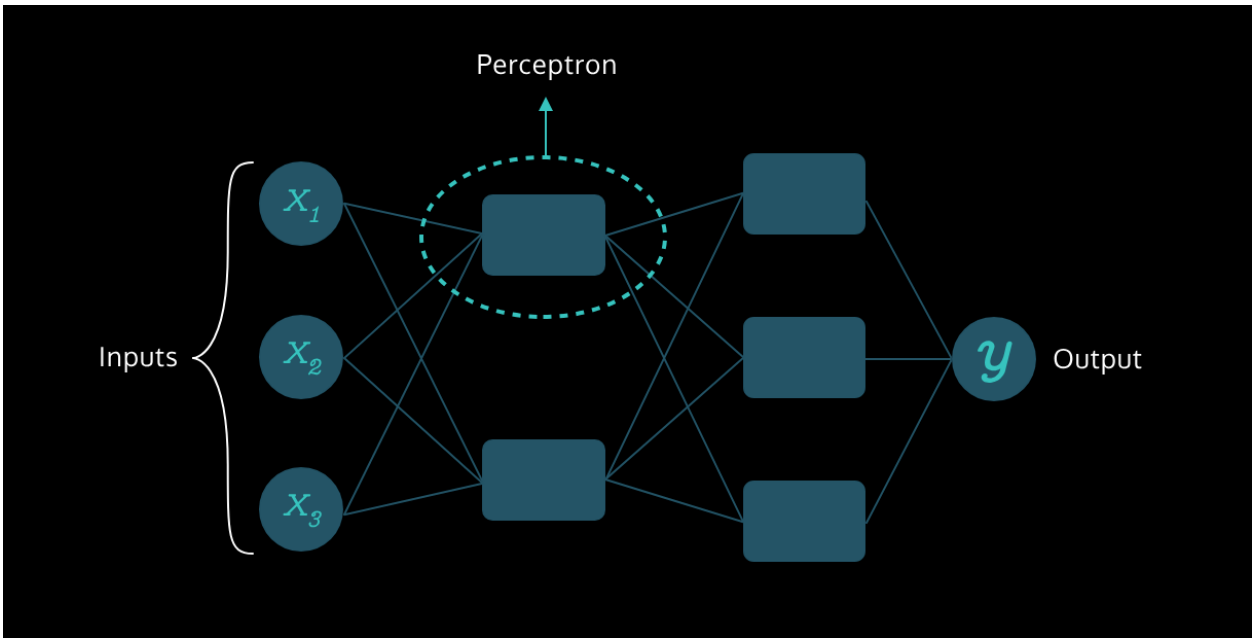
In this book, you will learn how a biased set of search algorithms privilege whiteness and discriminate against people of color, specifically women of color. This is helpful if you want to think about how to mitigate the impact of biased algorithms in your data science work.

**Perceptron**

# What's Next? Neural Networks

Congratulations! You have now built your own perceptron from scratch.

Let's step back and think about what you just accomplished and see if there are any limits to a single perceptron.
Earlier, the data points in the training set were *linearly separable* i.e. a single line could easily separate the two dissimilar sets of points.
What would happen if the data points were scattered in such a way that a line could no longer classify the points? A single perceptron with only two inputs wouldn't work for such a scenario because it cannot represent a *non-linear decision boundary*.
That's when more perceptrons and features come into play!
By increasing the number of features and perceptrons, we can give rise to the Multilayer Perceptrons, also known as Neural Networks, which can solve much more complicated problems.
With a solid understanding of perceptrons, you are now ready to dive into the incredible world of Neural Networks!




# Review: Foundations of Deep Learning and Perceptrons

**You have finished your introductory dive into deep learning concepts!**

## Foundations Deep Learning and Perceptrons

### Goals of this Unit

Congratulations! The goal of this unit was to grasp the foundational concepts of deep learning (DL). You gained an insight into the fundamentals of DL and investigated perceptrons as a

preview to deep learning and to get acquainted with the basic structure of a neural network. You are ready to move forward with investigating deep learning models with Python!
Having completed this unit, you are now able to:

- Explain what deep learning is to a friend
- Know different use cases for deep learning models
- Trace the beginning-to-end path of data that journeys a neural network and understand the inner workings behind the journey
- Decipher potential dangers for the use cases that involve deep learning models
- Develop your own perceptron algorithms

Happy coding!

# Introduction: Getting Started with TensorFlow

**Create your own neural networks using TensorFlow!**

## Implementing Neural Networks

### Goals of this Unit

The goal of this unit is to take the abstract concepts about deep learning you have learned and translate them into Python using TensorFlow. In this section, you will code your own neural network and investigate ways to fine-tune your program with hyperparameters. After exploring these concepts in lessons, you will get to work on your own regression projects where you will have to design and develop your own neural network to solve a given problem.
After this unit, you will be able to:

- Translate abstract deep learning concepts into Python
- Design a neural network from scratch
- Use deep learning models to solve regression problems
- Tune the hyperparameters of your model and improve its performance through iterative testing

Happy coding!

**Implementing Neural Networks**

### Introduction

A

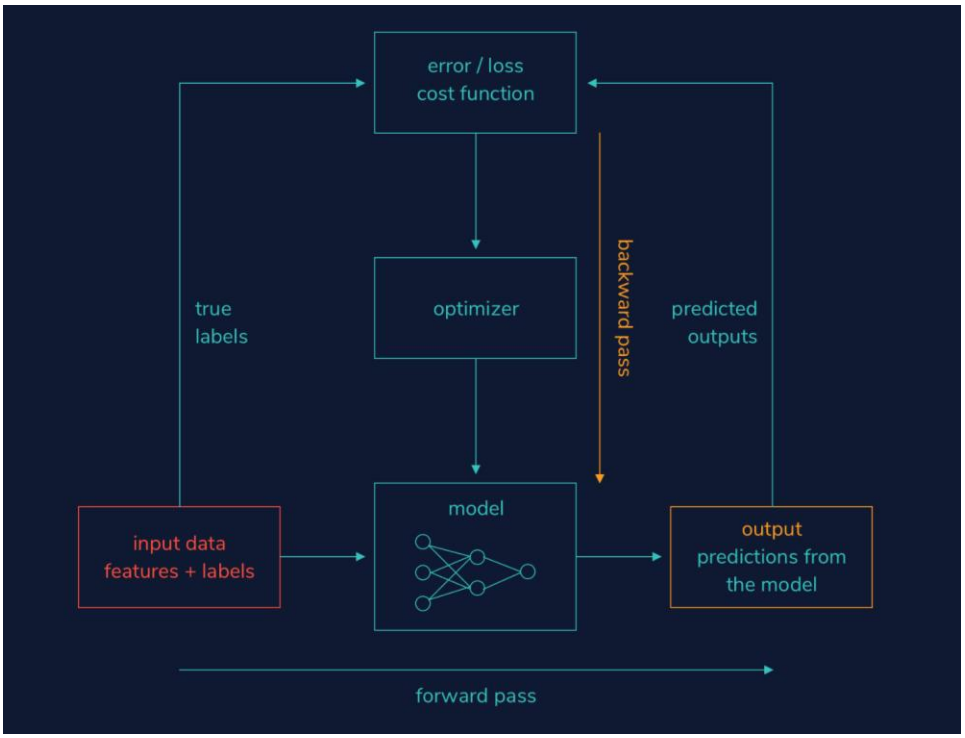
neural network
Preview: Docs A neural network, inspired by the human brain, is a model that learns from data to make decisions and solve complex problems.
, just like any machine learning method, learns how to perform tasks by processing data and adjusting its model to best predict the desired outcome. Most popular machine learning tasks are:

> Classification: given data and true labels or categories for each data point, train a model that predicts for each data example what its label should be. For example, given data of previous fire hazards, our model can learn how to predict whether a fire will occur for a given day in the future, with all the factors taken into account.
>
> Regression: given data and true continuous value for each data point, train a model that can predict values for each data example. For example, given the previous stock market data, we can build a regression model that forecasts what the stock market price will be at a specific point in time when the data is available.

Parametric models such as neural networks are described by *parameters*: configuration variables representing the model's knowledge. We can tweak the parameters using the training data and we can evaluate the performance of the model using hold-out test data the model has not seen during training.

**Instructions**

1. Take a look at the main components of a neural network learning pipeline depicted in the workspace:
   *Input data*: this is used to train a neural network model you need to provide it with some training data.
   An *optimizer*: this is an algorithm that based on the training data adjusts the parameters of the network in order to perform the task at hand.
   A *loss or cost function*: this informs the optimizer whether it is doing a good job on the training data and how to adjust the parameters in the right direction.
   *Evaluation metrics*: these tell us how well the current model performs on validation data. For example, mean absolute error for regression tells us how far the predictions are on average from the true values.




Implementing Neural Networks

# Predicting medical costs: loading the data

Every machine learning pipeline starts with data and a task. Let's take a look at the Medical Cost Personal Datasets dataset, which consists of seven columns with the following descriptions:

| Column names | Description | Data type |
|---|---|---|
| age | age of primary beneficiary | numerical/integer |
| sex | insurance contractor gender | integer (female is 1, male is 0) |
| bmi | body mass index | numerical/real value |
| children | number of children covered by health insurance | numerical/integer |

| | | |
|---|---|---|
| smoker | smoking or not | integer (true is 1, false is 0) |
| region | the beneficiary's residential area in the US | categorical (northeast, northwest, southeast, southwest) |
| charges | individual medical costs billed by health insurance | numerical/real value |

We would like to predict the individual medical costs (charges) given the rest of the columns/features. Since charges represent continuous values (in dollars), we're performing a regression task. Think about the potential implications of using `sex` or `bmi` to predict what the individual insurance charges should be? Should they be even used for prediction?
Our data is in the **.csv** format and we load it with pandas:

```
dataset = pd.read_csv('insurance.csv')
#view the first 5 entries of the dataset
print(dataset.head())
```

Copy to Clipboard

Next, we split the data into *features* (independent variables) and the *target variable* (dependent variable):

```
#dataframe slicing using iloc
features = dataset.iloc[:,0:6]
# we select the last column with -1
labels = dataset.iloc[:,-1]
```

Copy to Clipboard

The pandas `shape` property tells us the shape of our data — a vector of two values: the number of samples and the number of features. To check the shape of our dataset, we can do:

```
print(features.shape)
```

Copy to Clipboard

Or, to make things clearer:

```
print("Number of features: ", features.shape[1])
print("Number of samples: ", features.shape[0])
```

Copy to Clipboard

To see a useful summary statistics of the dataset we do:

```
print(features.describe())
```

Copy to Clipboard

**Instructions**

Checkpoint 1 Passed

**1.**

Use the `.shape` property of pandas DataFrames to print the number of samples of the `labels` Series.

To print the shape of a pandas Series we write the following:

```
print(my_series.shape)
```

Copy to Clipboard

You need to replace `my_series` with `labels`.

`labels` is a vector with dimensions (1338, ) The first dimension is the number of samples (the same as for `features`), and the second dimension is empty because `labels` is 1-dimensional (Series).

Checkpoint 2 Passed

**2.**

Use the `.describe()` method to print the summary statistics of the `labels` Series.

To describe a pandas DataFrame or Series we do the following:

```
print(my_series.describe())
```

Copy to Clipboard

You need to replace `my_series` with `labels`.

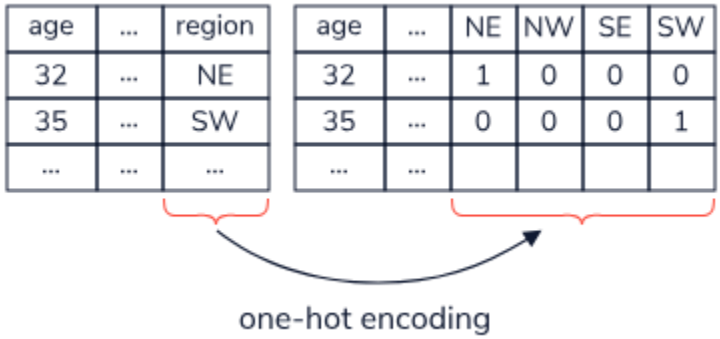
```
import pandas as pd

#load the dataset
dataset = pd.read_csv('insurance.csv')
#choose first 7 columns as features
features = dataset.iloc[:,0:6]
#choose the final column for prediction
labels = dataset.iloc[:,-1]

#print the number of features in the dataset
print("Number of features: ", features.shape[1])
#print the number of samples in the dataset
print("Number of samples: ", features.shape[0])
#see useful summary statistics for numeric features
print(features.describe())

#your code here below
print(labels.shape)
print(labels.describe())
```

### Implementing Neural Networks

# Data preprocessing: one-hot encoding and standardization

**One-hot encoding of categorical features:**

Since neural networks cannot work with string data directly, we need to convert our categorical features ("region") into numerical. *One-hot encoding* creates a binary column for each category. For example, since the "region" variable has four categories, the one-hot encoding will result in four binary columns: "northeast", "northwest", "southeast", "southwest" as shown in the table below.




One-hot encoding can be accomplished by using the pandas `get_dummies()` function:

```
features  = pd.get_dummies(features)
```

Copy to Clipboard

**Split data into train and test sets:**

In machine learning, we train a model on a training data, and we evaluate its performance on a held-out set of data, our test set, not seen during the learning:

```
from sklearn.model_selection import train_test_split
features_train, features_test, labels_train, labels_test =
train_test_split(features, labels, test_size=0.33, random_state=42)
```

Copy to Clipboard

Here we chose the test size to be 33% of the total data, and random state controls the shuffling applied to the data before applying the split.

**Standardize/normalize numerical features:**

The usual preprocessing step for numerical variables, among others, is *standardization* that rescales features to zero mean and unit variance. Why do we want to do that? Well, our features have different scales or units: "age" has an interval of [18, 64] and the "children" column's interval is much smaller, [0, 5]. By having features with differing scales, the optimizer might update some weights faster than the others.

*Normalization* is another way of preprocessing numerical data: it scales the numerical features to a fixed range - usually between 0 and 1.

So which should you use? Well, there isn't always a clear answer, but you can try them all out and choose the one method that gives the best results.

To normalize the numerical features we use an exciting addition to `scikit-learn`, `ColumnTransformer`, in the following way:

```
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import Normalizer
from sklearn.compose import ColumnTransformer

ct = ColumnTransformer([('normalize', Normalizer(), ['age', 'bmi',
'children'])], remainder='passthrough')
```

```
features_train_norm = ct.fit_transform(features_train)
features_test_norm = ct.transform(features_test)
```

Copy to Clipboard

The name of the column transformer is “only numeric”, it applies a `Normalizer()` to the ‘age’, ‘bmi’, and ‘children’ columns, and for the rest of the columns it just passes through. `ColumnTransformer()` returns NumPy arrays and we convert them back to a pandas DataFrame so we can see some useful summaries of the scaled data.

To convert a NumPy array back into a pandas DataFrame, we can do:

```
features_train_norm = pd.DataFrame(features_train_norm, columns = features_train.columns)
```

Copy to Clipboard

Note that we fit the scaler to the training data only, and then we apply the trained scaler onto the test data. This way we avoid “information leakage” from the training set to the test set. These two datasets should be completely unaware of each other!

**Instructions**

Checkpoint 1 Passed

**1.**

Scroll down to the bottom of **neural_networks.py**.

Create a new ColumnTransformer instance called `my_ct` that uses `StandardScaler()` and `'scale'` (instead of `Normalizer()` and `'normalize'`) with the same numerical features (`age`, `bmi`, `children`). Make sure to `passthrough` the remainder of the columns. We already imported the StandardScaler module for you.

We did something similar here with:

```
ct = ColumnTransformer([('normalize', Normalizer(), ['age', 'bmi', 'children'])], remainder='passthrough')
```

Copy to Clipboard

Checkpoint 2 Passed

**2.**

Use the `.fit_transform()` method of `my_ct` to fit the column transformer to the `features_train` DataFrame and at the same time transform it. Assign the result to a variable called `features_train_scale`.

For example, to fit and transform a pandas DataFrame `my_dataframe` using a `ColumnTransformer` instance `ct`:

```
transformed = ct.fit_transform(my_dataframe)
```

Copy to Clipboard

Checkpoint 3 Passed

**3.**

Use the `.transform()` method to transform the trained column transformer `my_ct` to the `features_test` DataFrame. Assign the result to a variable called `features_test_scale`.

To transform any pandas DataFrame using a `ColumnTransformer` instance `ct` we write the following:

```
transformed = ct.fit_transform(my_dataframe)
```

Copy to Clipboard

You need to replace `transformed` with `features_test_scale` and `my_dataframe` with `features_test`.

Checkpoint 4 Passed

**4.**

Transform the `features_train_scale` NumPy array back to a DataFrame using `pd.DataFrame()` and assign the result back to a variable called `features_train_scale`. For the `columns` attribute use the `.columns` property of `features_train`.

For example, to transform a pandas DataFrame `scaled_dataframe` into a NumPy array obtained by scaling `original_dataframe`, we write:

```
my_numpyarray = pd.DataFrame(scaled_dataframe, columns =
original_dataframe.columns)
```

Copy to Clipboard

Checkpoint 5 Passed

**5.**

Transform the `features_test_scale` NumPy array back to DataFrame using `pd.DataFrame()` and assign the result back to a variable called `features_test_scale`. For the `columns` attribute use the `.columns` property of `features_test`.

For example, to transform `scaled_dataframe` (a NumPy array obtained by scaling `original_dataframe`) into a pandas DataFrame, we write:

```
my_numpyarray = pd.DataFrame(scaled_dataframe, columns =
original_dataframe.columns)
```

Copy to Clipboard

Checkpoint 6 Passed

**6.**

Print the statistics summary of the resulting train and test DataFrames, `features_train_scale` and `features_test_scale`.

Observe the statistics of the numeric columns (mean, variance).

For example, to print summary statistics for a pandas DataFrame called `my_dataframe` we use the following command:

```
print(my_dataframe.describe())
```

Copy to Clipboard

Notice that the mean values for `bmi`, `age` and `children` columns are very low (close to 0), and the standard deviation is 1. This is exactly what we wanted to accomplish with scaling the numerical features.

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from sklearn.preprocessing import Normalizer

#load the dataset
dataset = pd.read_csv('insurance.csv')
#choose first 7 columns as features
```

```
features = dataset.iloc[:,0:6]
#choose the final column for prediction
labels = dataset.iloc[:,-1]

#one-hot encoding for categorical variables
features = pd.get_dummies(features)
#split the data into training and test data
features_train, features_test, labels_train, labels_test =
train_test_split(features, labels, test_size=0.33, random_state=42)

#normalize the numeric columns using ColumnTransformer
ct = ColumnTransformer([('normalize', Normalizer(), ['age', 'bmi',
'children'])], remainder='passthrough')
#fit the normalizer to the training data and convert from numpy arrays to
pandas frame
features_train_norm = ct.fit_transform(features_train)
#applied the trained normalizer on the test data and convert from numpy
arrays to pandas frame
features_test_norm = ct.transform(features_test)

#ColumnTransformer returns numpy arrays. Convert the features to
dataframes
features_train_norm = pd.DataFrame(features_train_norm, columns =
features_train.columns)
features_test_norm = pd.DataFrame(features_test_norm, columns =
features_test.columns)

# -----> your code here below
my_ct = ColumnTransformer([('scale', StandardScaler(), ['age', 'bmi',
'children'])], remainder='passthrough')
#fit the scaler to the training data and convert from numpy arrays to
pandas frame
features_train_scale = my_ct.fit_transform(features_train)
#applied the trained scaler on the test data and convert from numpy arrays
to pandas frame
features_test_scale = my_ct.transform(features_test)

#ColumnTransformer returns numpy arrays. Convert the features to
dataframes
features_train_scale = pd.DataFrame(features_train_scale, columns =
features_train.columns)
features_test_scale = pd.DataFrame(features_test_scale, columns =
features_test.columns)

#print the summary statistics
print(features_train_scale.describe())
print(features_test_scale.describe())
```

## Implementing Neural Networks

# Neural network model: tf.keras.Sequential

Now that we have our data preprocessed we can start building the
neural network
Preview: Docs Loading link description
model. The most frequently used model in TensorFlow is Keras *Sequential*. A sequential model, as the name suggests, allows us to create models layer-by-layer in a step-by-step fashion. This model can have only one input tensor and only one output tensor.
To design a sequential model, we first need to import `Sequential` from `keras.models`:

```
from tensorflow.keras.models import Sequential
```

Copy to Clipboard
To improve readability, we will design the model in a separate Python function called `design_model()`. The following command initializes a Sequential model instance `my_model`:

```
my_model = Sequential(name="my first model")
```

Copy to Clipboard
`name` is an optional argument to any model in Keras.
Finally, we invoke our function in the main program with:

```
my_model = design_model(features_train)
```

Copy to Clipboard
The model's layers are accessed via the `layers` attribute:

```
print(my_model.layers)
```

Copy to Clipboard
As expected, the list of layers is empty. In the next exercise, we will start adding layers to our model.

**Instructions**

Checkpoint 1 Passed

1.

In the `design_model()` function:

remove `pass`

initialize an instance of `Sequential()` and assign it to a variable called `model`

For example, the following code initializes an instance of `Sequential()` and assigns it to a variable called `my_model`:

```
my_model = Sequential()
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Return the model instance `model` from the `design_model()` function.

The following code returns the value of a local variable called `a` from a function called `my_func()`:

```
def my_func():
  a = 5
  return a
```

Copy to Clipboard

Checkpoint 3 Passed

3.

In the main program, using the `layers` attribute, print the layers of the model instance `model`.

For example, the following code uses the `layers` attribute to print the layers of the model instance `my_model`:

```
print(my_model.layers)
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras import layers


def design_model(features):
  model = Sequential()
  return model

dataset = pd.read_csv('insurance.csv') #load the dataset
features = dataset.iloc[:,0:6] #choose first 7 columns as features
labels = dataset.iloc[:,-1] #choose the final column for prediction

features = pd.get_dummies(features) #one-hot encoding for categorical
variables
```

```
features_train, features_test, labels_train, labels_test =
train_test_split(features, labels, test_size=0.33, random_state=42) #split
the data into training and test data

#standardize
ct = ColumnTransformer([('standardize', StandardScaler(), ['age', 'bmi',
'children'])], remainder='passthrough')
features_train = ct.fit_transform(features_train)
features_test = ct.transform(features_test)

#invoke the function for our model design
model = design_model(features_train)

#print the layers
print(model.layers)
```

https://www.codecademy.com/paths/build-deep-learning-models-with-tensorflow/tracks/dlsp-getting-started-with-tensorflow/modules/dlsp-implementing-neural-networks/lessons/dl-neural-networks/exercises/introduction

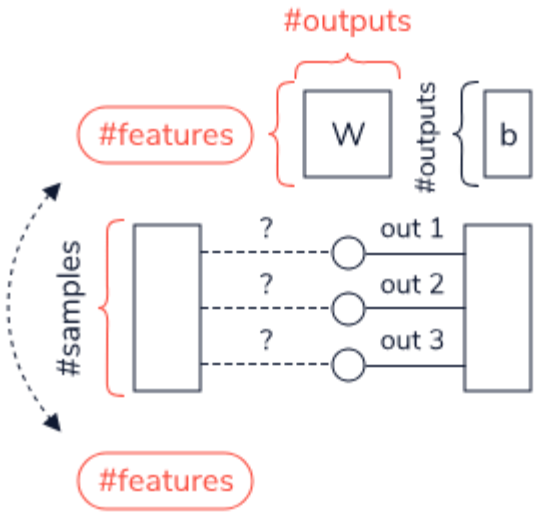
### Implementing Neural Networks

# Neural network model: layers

Layers are the building blocks of neural networks and can contain 1 or more neurons. Each layer is associated with parameters: weights, and bias, that are tuned during the learning. A fully-connected layer in which all neurons connect to all neurons in the next layer is created the following way in TensorFlow:

```
from tensorflow.keras import layers
# we chose 3 neurons here
layer = layers.Dense(3)
```

Copy to Clipboard




This layer looks like this graphically:

Pay attention to the dimensions of the weight and bias parameter matrices. Since we chose to create a layer with three neurons, the number of outputs of this layer is 3. Hence, the bias parameter would be a vector of (3, 1) dimensions. But what is the first dimension of the weights matrix? Without knowing how many features or input nodes are in the previous layer, we have no way of knowing! For that reason, with the following code:

```
print(layer.weights)
```

Copy to Clipboard

we get an empty array since no input layer is specified. However, if we write:

```
# 1338 samples, 11 features as in our dataset
input = tf.ones((1338, 11))
# a fully-connected layer with 3 neurons
layer = layers.Dense(3)
# calculate the outputs
output = layer(input)
# print the weights
print(layer.weights)
```

Copy to Clipboard

we get that the weight matrix has `shape = (11, 3)` and the bias matrix has `shape=(3,)`. Compare these weights with the diagram above to make sure you can associate the resulting shapes to it.

Fortunately, we don't have to worry about this. TensorFlow will determine the shapes of the weight matrix and bias matrix automatically the moment it encounters the first input.

**Instructions**

Checkpoint 1 Passed

**1.**

Change the number of samples in the `input` tensor from 1338 to 5000. How does this change affect the shape of the weight and bias vectors?

First, locate the following line of code in the `layers_demo.py` script:

```
input = tf.ones((1338, 11))
```

Copy to Clipboard

Second, replace 1338 (the number of samples) with 5000.

After you run the code, you will see that the weight and bias matrices didn't change their shape (their shape is independent of the number of samples) - see the diagram.

Checkpoint 2 Passed

**2.**

Now, change the number of features in `input` from 11 to 21. How does this change affect the shape of the weight and bias vectors?

First, locate the following line of code in the `layers_demo.py` script:

```
input = tf.ones((5000, 11))
```

Copy to Clipboard

Second, replace 11 (number of features) with 21.

Changing the number of features of the input data will modify the weight matrix. If you look into the diagram, the dimensions of a weight matrix are features and output. Since our features changed from 11 to 21, the weight matrix will have a new shape (21, 3). The bias array does not change its shape by changing the number of features.

Checkpoint 3 Passed

**3.**

Change the number of neurons in `layer` (below where `input` is defined) from 3 to 10. How does this change affect the shape of the weight and bias vectors?

First, locate the following code in the `layers_demo.py` script:

```
input = tf.ones((5000, 21))
# a fully-connected layer with 3 neurons
layer = layer.Dense(3)
```

Copy to Clipboard

Second, replace 3 (number of neurons/outputs) with 10 where `layer` is defined.

Changing the number of neurons in a layer changes the shape of the weight matrix from (21,3) to (21, 10), and bias from (3,) to (10,). Compare this to the diagram we drew for you.

```
import tensorflow as tf
from tensorflow.keras import layers
layer = layers.Dense(3) #3 is the number we chose

print(layer.weights) #we get empty weight and bias arrays because tensorflow
doesn't know what the shape is of the input to this layer

# 1338 samples, 11 features as in our dataset
input = tf.ones((5000, 21))
# a fully-connected layer with 3 neurons
layer = layers.Dense(10)
# calculate the outputs
output = layer(input)
# print the weights
print(layer.weights)
```

### Implementing Neural Networks

# Neural network model: input layer

Inputs to a
neural network
Preview: Docs A neural network, inspired by the human brain, is a model that learns from data to make decisions and solve complex problems.
are usually not considered the actual transformative layers. They are merely placeholders for data. In Keras, an input for a neural network can be specified with a `tf.keras.layers.InputLayer` object.
The following code initializes an input layer for a `DataFrame` `my_data` that has 15 columns:

```
from tensorflow.keras.layers import InputLayer
my_input = InputLayer(input_shape=(15,))
```

Copy to Clipboard
Notice that the `input_shape` parameter has to have its first dimension equal to the number of features in the data. You don't need to specify the second dimension: the number of samples or batch size.
The following code avoids hard-coding with using the `.shape` property of the `my_data` `DataFrame`:

```
#get the number of features/dimensions in the data
num_features = my_data.shape[1]
#without hard-coding
my_input = tf.keras.layers.InputLayer(input_shape=(num_features,))
```

Copy to Clipboard
The following code adds this input layer to a model instance `my_model`:

```
my_model.add(my_input)
```

Copy to Clipboard
The following code prints a useful summary of a model instance `my_model`:

```
print(my_model.summary())
```

Copy to Clipboard
As you can see, the summary shows that the total number of parameters is 0. This shows you that the input layer has no trainable parameters and is just a placeholder for data.
**Instructions**

Checkpoint 1 Passed

**1.**

In the `design_model()` function, create a variable called `num_features` and assign it the number of columns in the `features` `DataFrame` using the `.shape` property.

For example, the following code uses the `.shape` property of `DataFrame` to assign the number of columns in the `my_data` DataFrame to a variable called `num_features`:

```
num_features = my_data.shape[1]
```

Copy to Clipboard

Checkpoint 2 Passed

**2.**

In the `design_model()` function:

create a variable called `input`

assign `input` an instance of `InputLayer`

set the first dimension of the `input_shape` parameter equal to `num_features`

Then add the input layer to the model.

For example, the following code creates a variable called `input` and assigns it an instance of `InputLayer` such that the first dimension of the `input_shape` parameter is assigned the `num_features` variable representing the number of columns in the `my_data DataFrame`:

```
num_features = my_data.shape[1]
my_input = layers.InputLayer(input_shape=(num_features,))
```

Copy to Clipboard

To add `my_input` to `my_model`, you could do the following:

```
my_model.add(my_input)
```

Copy to Clipboard

Checkpoint 3 Passed

**3.**

At the bottom of **neural_networks.py**, use the `.summary()` method to print the summary of the model instance `model`.

The following code prints a useful summary of a model instance `my_model`:

```
print(my_model.summary())
```

```
import pandas as pd
import tensorflow as tf
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras import layers


def design_model(features):
  model = Sequential(name = "my_first_model")
  #get the number of features/dimensions in the data
  num_features = features.shape[1]
  #without hard-coding
  input = layers.InputLayer(input_shape=(num_features,))
  #adding the input layer
  model.add(input)
  return model


dataset = pd.read_csv('insurance.csv') #load the dataset
features = dataset.iloc[:,0:6] #choose first 7 columns as features
labels = dataset.iloc[:,-1] #choose the final column for prediction

features = pd.get_dummies(features) #one-hot encoding for categorical
variables
```

```
features_train, features_test, labels_train, labels_test =
train_test_split(features, labels, test_size=0.33, random_state=42) #split
the data into training and test data

#standardize
ct = ColumnTransformer([('standardize', StandardScaler(), ['age', 'bmi',
'children'])], remainder='passthrough')
features_train = ct.fit_transform(features_train)
features_test = ct.transform(features_test)

#invoke the function for our model design
model = design_model(features_train)
print(model.summary())
```

Implementing Neural Networks

# Neural network model: output layer

The output layer shape depends on your task. In the case of regression, we need one output for each sample. For example, if your data has 100 samples, you would expect your output to be a vector with 100 entries - a numerical prediction for each sample.

In our case, we are doing regression and wish to predict one number for each data point: the medical cost billed by health insurance indicated in the `charges` column in our data. Hence, our output layer has only one neuron.

The following command adds a layer with one neuron to a model instance `my_model`:

```
from tensorflow.keras.layers import Dense
my_model.add(Dense(1))
```

Copy to Clipboard

Notice that you don't need to specify the input shape of this layer since Tensorflow with Keras can automatically infer its shape from the previous layer.

**Instructions**

Checkpoint 1 Passed

**1.**

In a single command, create and add an output layer to the model instance `model` as an instance of `tensorflow.keras.layers.Dense`.

For example, to add an output layer to `my_model`:

```
my_model.add(Dense(1))
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import InputLayer
from tensorflow.keras.layers import Dense
```

```
def design_model(features):
  model = Sequential(name = "my_first_model")
  num_features = features.shape[1]
  input = InputLayer(input_shape=(num_features,))
  model.add(input) #add the input layer
  #your code
  model.add(Dense(1))
  return model


dataset = pd.read_csv('insurance.csv') #load the dataset
features = dataset.iloc[:,0:6] #choose first 7 columns as features
labels = dataset.iloc[:,-1] #choose the final column for prediction

features = pd.get_dummies(features) #one-hot encoding for categorical
variables
features_train, features_test, labels_train, labels_test =
train_test_split(features, labels, test_size=0.33, random_state=42) #split
the data into training and test data

#standardize
ct = ColumnTransformer([('standardize', StandardScaler(), ['age', 'bmi',
'children'])], remainder='passthrough')
features_train = ct.fit_transform(features_train)
features_test = ct.transform(features_test)

#invoke the function for our model design
model = design_model(features_train)
print(model.summary())
```

```
Model: "my_first_model"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense (Dense)                (None, 1)                 10
=================================================================
Total params: 10
Trainable params: 10
Non-trainable params: 0
_________________________________________________________________
None
```

## Implementing Neural Networks

# Neural network model: hidden layers

So far we have added one input layer and one output layer to our model. If you think about it, our model currently represents a linear regression. To capture more complex or non-linear interactions among the inputs and outputs neural networks, we'll need to incorporate hidden layers.

The following command adds a hidden layer to a model instance `my_model`:

```
from tensorflow.keras.layers import Dense
my_model.add(Dense(64, activation='relu'))
```

Copy to Clipboard

We chose 64 (2^6) to be the number of neurons since it makes optimization more efficient due to the binary nature of computation.

With the `activation` parameter, we specify which activation function we want to have in the output of our hidden layer. There are a number of activation functions such as `softmax`, `sigmoid`, but ReLU (`relu`) (Rectified Linear Unit) is very effective in many applications and we'll use it here.

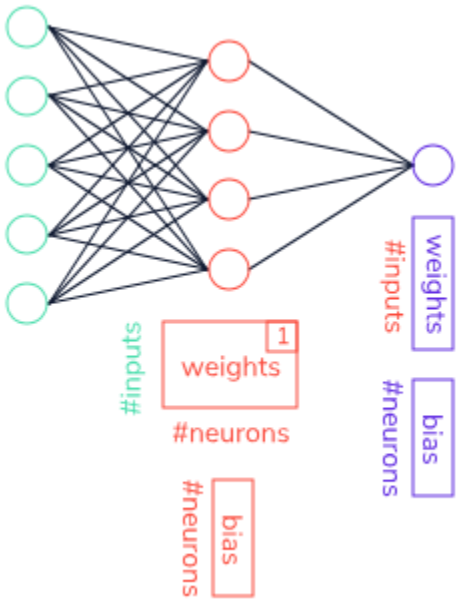
Adding more layers to a

neural network

Preview: Docs Loading link description

naturally increases the number of parameters to be tuned. With every layer, there are associated weight and bias vectors.

In the diagram below, we show the size of parameter vectors with each layer. In our case, the 1st layer's weight matrix (red) has shape (11, 64) because we feed 11 features to 64 hidden neurons. The output layer (purple) has the weight matrix of shape (64, 1) because we have 64 input units and 1 neuron in the final layer.




**Instructions**

Checkpoint 1 Passed

**1.**

In the `design_model()` function, in a single command, add a new hidden layer to the model instance `model` with the following parameters:

- 128 hidden units
- a `relu` activation function

The following code adds a new hidden layer to a model instance `my_model` with 64 hidden units and a `relu` activation function:

```
from tensorflow.keras.layers import Dense
my_model.add(Dense(64, activation = 'relu'))
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import InputLayer
from tensorflow.keras.layers import Dense


def design_model(features):
  model = Sequential(name = "my_first_model")
  input = InputLayer(input_shape=(features.shape[1],))
  #add the input layer
  model.add(input)
  #your code here
  model.add(Dense(128, activation = 'relu'))
  #adding an output layer to our model
  model.add(Dense(1))
  return model


dataset = pd.read_csv('insurance.csv') #load the dataset
features = dataset.iloc[:,0:6] #choose first 7 columns as features
labels = dataset.iloc[:,-1] #choose the final column for prediction

features = pd.get_dummies(features) #one-hot encoding for categorical
variables
features_train, features_test, labels_train, labels_test =
train_test_split(features, labels, test_size=0.33, random_state=42) #split
the data into training and test data

#standardize
ct = ColumnTransformer([('standardize', StandardScaler(), ['age', 'bmi',
'children'])], remainder='passthrough')
features_train = ct.fit_transform(features_train)
features_test = ct.transform(features_test)

#invoke the function for our model design
model = design_model(features_train)
```

```
#print the model summary here
print(model.summary())


Model: "my_first_model"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense (Dense)                (None, 128)               1280
_________________________________________________________________
dense_1 (Dense)              (None, 1)                 129
=================================================================
Total params: 1,409
Trainable params: 1,409
Non-trainable params: 0
_________________________________________________________________
None
```

Implementing Neural Networks

# Optimizers

As we mentioned, our goal is for the network to effectively adjust its weights or parameters in order to reach the best performance. Keras offers a variety of optimizers such as `SGD` (Stochastic Gradient Descent optimizer), `Adam`, `RMSprop`, and others.
We'll start by introducing the Adam optimizer:

```
from tensorflow.keras.optimizers import Adam
opt = Adam(learning_rate=0.01)
```

Copy to Clipboard

The learning rate determines how big of jumps the optimizer makes in the parameter space (weights and bias) and it is considered a *hyperparameter* that can be also tuned. While model parameters are the ones that the model uses to make predictions, hyperparameters determine the learning process (learning rate, number of iterations, optimizer type).
If the learning rate is set too high, the optimizer will make large jumps and possibly miss the solution. On the other hand, if set too low, the learning process is too slow and might not

converge to a desirable solution with the allotted time. Here we'll use a value of 0.01, which is often used.

Once the optimizer algorithm is chosen, a model instance `my_model` is compiled with the following code:

```
my_model.compile(loss='mse',  metrics=['mae'], optimizer=opt)
```

Copy to Clipboard

`loss` denotes the measure of learning success and the lower the loss the better the performance. In the case of regression, the most often used loss function is the Mean Squared Error `mse` (the average squared difference between the estimated values and the actual value). Additionally, we want to observe the progress of the Mean Absolute Error (`mae`) while training the model because MAE can give us a better idea than `mse` on how far off we are from the true values in the units we are predicting. In our case, we are predicting `charge` in dollars and MAE will tell us how many dollars we're off, on average, from the actual values as the network is being trained.

**Instructions**

Checkpoint 1 Passed

**1.**

In the `design_model()` function, create an instance of `Adam` optimizer with 0.01 learning rate and assign the result to a variable called `opt`.

For example, the following code creates an instance of `Adam` optimizer with 0.01 learning rate and assigns the result to a variable called `my_opt`:

```
from tensorflow.keras.optimizers import Adam

my_opt = Adam(learning_rate = 0.01)
```

Copy to Clipboard

Checkpoint 2 Passed

**2.**

In the `design_model()` function, use the `.compile()` method to compile the model instance `model` with:

- the `mse` loss
- `mae` metrics
- `opt` as the optimizer

For example, the following code uses the `.compile()` method to compile the model instance `my_model` with the `mse` loss, with `mae` metrics and optimizer instance `my_opt`:

```
my_model.compile(loss='mse',  metrics=['mae'], optimizer=my_opt)
```

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import InputLayer
from tensorflow.keras.layers import Dense
```

```
from tensorflow.keras.optimizers import Adam


def design_model(features):
  model = Sequential(name = "my_first_model")
  input = InputLayer(input_shape=(features.shape[1],))
   #add an input layer
  model.add(input)
  #add a hidden layer with 128 neurons
  model.add(Dense(128, activation='relu'))
  #add an output layer
  model.add(Dense(1))
  #your code here
  opt = Adam(learning_rate = 0.01)
  model.compile(loss = 'mse', metrics = ['mae'], optimizer = opt)
  return model


dataset = pd.read_csv('insurance.csv') #load the dataset
features = dataset.iloc[:,0:6] #choose first 7 columns as features
labels = dataset.iloc[:,-1] #choose the final column for prediction

features = pd.get_dummies(features) #one-hot encoding for categorical
variables
features_train, features_test, labels_train, labels_test =
train_test_split(features, labels, test_size=0.33, random_state=42) #split
the data into training and test data

#standardize
ct = ColumnTransformer([('standardize', StandardScaler(), ['age', 'bmi',
'children'])], remainder='passthrough')
features_train = ct.fit_transform(features_train)
features_test = ct.transform(features_test)

#invoke the function for our model design
model = design_model(features_train)
print(model.summary())
```

```
Model: "my_first_model"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
dense (Dense)                (None, 128)               1280
_________________________________________________________________
dense_1 (Dense)              (None, 1)                 129
=================================================================
```

```
Total params: 1,409
Trainable params: 1,409
Non-trainable params: 0
_________________________________________________________________
None
```

Implementing Neural Networks

# Training and evaluating the model

Now that we built the model we are ready to train the model using the training data.
The following command trains a model instance `my_model` using training data `my_data` and training labels `my_labels` :

```
my_model.fit(my_data, my_labels, epochs=50, batch_size=3, verbose=1)
```

Copy to Clipboard

`model.fit()` takes the following parameters:

- `my_data` is the training data set.
- `my_labels` are true labels for the training data points.
- `epochs` refers to the number of cycles through the full training dataset. Since training of neural networks is an iterative process, you need multiple passes through data. Here we chose 50 epochs, but how do you pick a number of epochs? Well, it is hard to give one answer since it depends on your dataset. Amongst others, this is a hyperparameter that can be tuned — which we'll cover later.
- `batch_size` is the number of data points to work through before updating the model parameters. It is also a hyperparameter that can be tuned.
- `verbose = 1` will show you the progress bar of the training.

When the training is finalized, we use the trained model to predict values for samples that the training procedure haven't seen: the *test set*.
The following commands evaluates the model instance `my_model` using the test data `my_data` and test labels `my_labels`:

```
val_mse, val_mae = my_model.evaluate(my_data, my_labels, verbose = 0)
```

Copy to Clipboard

In our case, `model.evaluate()` returns the value for our chosen loss metrics (`mse`) and for an additional metrics (`mae`).
So what is the final result? We should get ~$3884.21. This means that on average we're off with our prediction by around 3800 dollars. Is that a good result or a bad result?

Often you need an expert or domain knowledge to decide this. What is an acceptable error for the application? Is $3800 a big error when deciding on insurance charges? Can you do better and how? As you see, the process doesn't stop here.

**Instructions**

Checkpoint 1 Passed

**1.**

Using the `.fit()` method, train the model instance `model` with:

the training data `features_train`
training labels `labels_train`
40 epochs
batch size equal to 1
verbose set to true (1)

The following code uses the `.fit()` method to train the model instance `my_model` using the training data `my_data`, training labels `my_labels`, with 100 epochs, batch size equal to 1, and with verbose set to true (1):

```
my_model.fit(my_data, my_labels, epochs=100, batch_size=1, verbose=1)
```

Copy to Clipboard

Checkpoint 2 Passed

**2.**

Using the `.evaluate()` method, evaluate the model instance `model` with

the test data `features_test`
test labels `labels_test`
the `verbose` parameter set to false (0)

Assign the result to variables `val_mse` and `val_mae`, respectively.

For example, the following code uses the `.evaluate()` method to evaluate the model instance `my_model` using the test data `my_data`, test labels `my_labels`, and verbose parameter set to false (0) and assign the result to variables `val_mse` and `val_mae`, respectively:

```
val_mse, val_mae = my_model.evaluate(my_data, my_labels, verbose = 0)
```

```
import pandas as pd
import tensorflow
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import InputLayer
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

tensorflow.random.set_seed(35) #for the reproducibility of results

def design_model(features):
  model = Sequential(name = "my_first_model")
  #without hard-coding
  input = InputLayer(input_shape=(features.shape[1],))
```

```
  #add the input layer
  model.add(input)
  #add a hidden layer with 64 neurons
  model.add(Dense(128, activation='relu'))
  #add an output layer to our model
  model.add(Dense(1))
  opt = Adam(learning_rate=0.1)
  model.compile(loss='mse',  metrics=['mae'], optimizer=opt)
  return model

dataset = pd.read_csv('insurance.csv') #load the dataset
features = dataset.iloc[:,0:6] #choose first 7 columns as features
labels = dataset.iloc[:,-1] #choose the final column for prediction

features = pd.get_dummies(features) #one-hot encoding for categorical
variables
features_train, features_test, labels_train, labels_test =
train_test_split(features, labels, test_size=0.33, random_state=42) #split
the data into training and test data

#standardize
ct = ColumnTransformer([('standardize', StandardScaler(), ['age', 'bmi',
'children'])], remainder='passthrough')
features_train = ct.fit_transform(features_train)
features_test = ct.transform(features_test)

#invoke the function for our model design
model = design_model(features_train)
print(model.summary())

#fit the model to the training data using 20 epochs, and 1 batch size
model.fit(features_train, labels_train, epochs = 40, batch_size = 1,
verbose = 1)

#evaluate the model on the test data
val_mse, val_mae = model.evaluate(features_test, labels_test, verbose = 0)

print("MAE: ", val_mae)


23205430.0000 - mae: 2884.0330
MAE:  2549.854736328125
```

## Implementing Neural Networks

# Summary

Congrats! You have built your
neural network
Preview: Docs Loading link description
, trained it, and evaluated it using TensorFlow with Keras. To remind you, these are the concepts you learned in this lesson:

- Preparing the data for learning:
  - separating features from labels using array slicing
  - determining the shape of your data
  - preprocessing the categorical variables using one-hot encoding
  - splitting the data into training and test sets
  - scaling the numerical features
- Designing a Sequential model by chaining `InputLayer()` and the `tf.keras.layers.Dense` layers. `InputLayer()` was used as a placeholder for the input data. The output layer in this case needed one neuron since we need a prediction of a single value in the regression. And finally, hidden layers were added with the `relu` activation function to handle complex dependencies in the data.
- Choosing an optimizer using `keras.optimizers` with a specific learning rate hyperparameter.
- Training the model - using `model.fit()` to train the model on the training data and training labels.
- Setting the values for the learning hyperparameters: number of epochs and batch sizes.
- Evaluating the model using `model.evaluate()` on the test data.

You might be wondering, what do I do with the plethora of hyperparameters? Or why if I use different random states I receive different results? Plus, how I can guarantee that my good performance isn't just good luck?
And you are right! This is not the full story. In machine learning, we tweak the hyperparameters using a better evaluation methodology — something we'll cover next.

```
import pandas as pd
import tensorflow
from sklearn.model_selection import train_test_split
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import InputLayer
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

tensorflow.random.set_seed(35) #for the reproducibility of results

def design_model(features):
  model = Sequential(name = "my_first_model")
  #without hard-coding
  input = InputLayer(input_shape=(features.shape[1],))
  #add the input layer
  model.add(input)
```

```
  #add a hidden layer with 64 neurons
  model.add(Dense(128, activation='relu'))
  #add an output layer to our model
  model.add(Dense(1))
  opt = Adam(learning_rate=0.1)
  model.compile(loss='mse',  metrics=['mae'], optimizer=opt)
  return model

dataset = pd.read_csv('insurance.csv') #load the dataset
features = dataset.iloc[:,0:6] #choose first 7 columns as features
labels = dataset.iloc[:,-1] #choose the final column for prediction

features = pd.get_dummies(features) #one-hot encoding for categorical
variables
features_train, features_test, labels_train, labels_test =
train_test_split(features, labels, test_size=0.33, random_state=42) #split
the data into training and test data

#standardize
ct = ColumnTransformer([('standardize', StandardScaler(), ['age', 'bmi',
'children'])], remainder='passthrough')
features_train = ct.fit_transform(features_train)
features_test = ct.transform(features_test)

#invoke the function for our model design
model = design_model(features_train)
print(model.summary())

#fit the model to the training data using 20 epochs, and 1 batch size
model.fit(features_train, labels_train, epochs = 40, batch_size = 1,
verbose = 1)

#evaluate the model on the test data
val_mse, val_mae = model.evaluate(features_test, labels_test, verbose = 0)

print("MAE: ", val_mae)
```

https://www.codecademy.com/paths/build-deep-learning-models-with-tensorflow/tracks/dlsp-getting-started-with-tensorflow/modules/dlsp-implementing-neural-networks/lessons/dl-neural-networks/exercises/dl-summary

# Hyperparameter Tuning

In order to understand this tutorial about hyperparameter tuning refer first to the following:

- **Data = insurance.csv**
- **MODEL.py**

### MODEL.py

```
import tensorflow as tf
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import Normalizer

tf.random.set_seed(42) #for reproducibility of result we always use the same seed for random
number generator

dataset = pd.read_csv("insurance.csv") #read the dataset

def design_model(X, learning_rate):
    model = Sequential(name="my_first_model")
    input = tf.keras.Input(shape=(X.shape[1],))
    model.add(input)
    model.add(layers.Dense(128, activation='relu'))
    model.add(layers.Dense(64, activation='relu'))
    model.add(layers.Dense(24, activation='relu'))
    model.add(layers.Dense(1))
    opt = tf.keras.optimizers.Adam(learning_rate = learning_rate)
    model.compile(loss='mse', metrics=['mae'], optimizer=opt)
    return model

def fit_model(f_train, l_train, learning_rate, num_epochs):
    #build the model
    model = design_model(features_train, learning_rate)

    #train the model on the training data
    es = EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience = 20)
    history = model.fit(features_train, labels_train, epochs=num_epochs, batch_size= 16,
verbose=0, validation_split = 0.2, callbacks = [es])

    print(str(history.history.keys()))
    # plot learning curves
    fig, axs = plt.subplots(1, 2, gridspec_kw={'hspace': 1, 'wspace': 0.8})
    (ax1, ax2) = axs
    ax1.plot(history.history['loss'], label='train')
    ax1.plot(history.history['val_loss'], label='validation')
    ax1.set_title('lrate=' + str(learning_rate), pad=-50)
    ax1.legend(loc="upper right")
    ax1.set_xlabel("# of epochs")
    ax1.set_ylabel("loss (mse)")
```

```
    ax2.plot(history.history['mae'], label='train')
    ax2.plot(history.history['val_mae'], label='validation')
    ax2.set_title('lrate=' + str(learning_rate), pad=-50)
    ax2.legend(loc="upper right")
    ax2.set_xlabel("# of epochs")
    ax2.set_ylabel("MAE")

    print("Final training MAE:", history.history['mae'][-1])
    print("Final validation MAE:", history.history['val_mae'][-1])


features = dataset.iloc[:,0:6] #choose first 7 columns as features
labels = dataset.iloc[:,-1] #choose the final column for prediction

features = pd.get_dummies(features) #one hot encoding for categorical variables
features_train, features_test, labels_train, labels_test = train_test_split(features, labels,
test_size=0.33, random_state=42)

#standardize
ct = ColumnTransformer([('standardize', StandardScaler(), ['age', 'bmi', 'children'])],
remainder='passthrough')
features_train = ct.fit_transform(features_train) #gives numpy arrays
features_test = ct.transform(features_test) #gives numpy arrays
```

**Hyperparameter Tuning**

# Introduction to hyperparameter tuning

We bet you were excited to build your first
neural network
Preview: Docs A neural network, inspired by the human brain, is a model that learns from data to make decisions and solve complex problems.
model, train, and evaluate it. But usually the story doesn't end there. Previously, we chose certain values for the following:

- the learning rate
- number of batches
- number of epochs
- number of units per hidden layer
- activation functions.

These are all hyperparameters. Why did we choose those numbers and values? Sometimes you start with a "hunch," by looking at what other machine learning engineers often use in their solutions.
Almost any machine learning project you start will have the pipeline depicted in the diagram on the right side.
First, we split data into the following sets:

- A training set to fit the model by updating the parameters (weights) of a neural network model. Previously, we randomly selected 67% (two thirds) of our data to be the training set.
- A *validation set* for checking the model's fit. It is a biased evaluation since the model is modified based on its performance on this set. We haven't set aside a validation set until now.
- A test set is used for an unbiased evaluation of the model. The test set should never be used for hyperparameter selection and tweaking! Rather, it is used to compare different models. For example, in

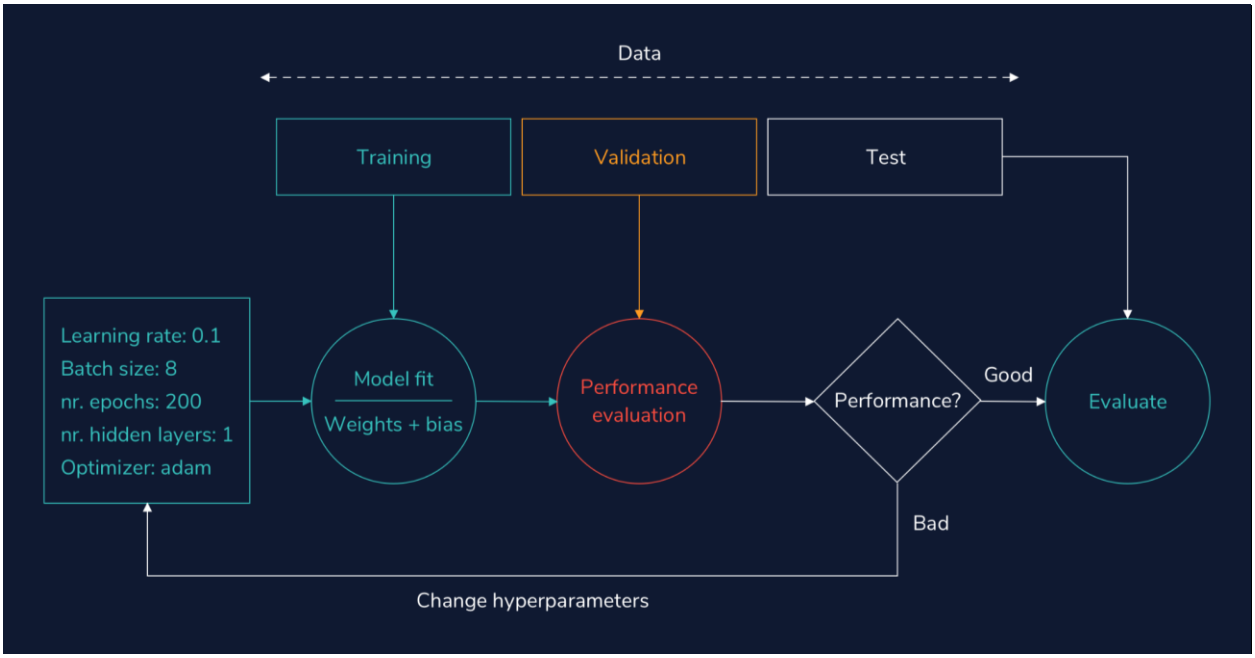
Kaggle competitions, the leaderboard is determined based on the performance on a test set. Previously, we allocated 33% of the total data to be the test set.

Second, we set hyperparameters to some initial values. We fit the model to the training data and check how it performs on the validation data by looking at accuracy for classification or mean absolute error for regression. Finally, if the performance is not satisfactory, we change the hyperparameters and repeat the steps. Otherwise, we evaluate the final model on the test set and report the results.

What percentage of our data should we set aside for our training, validation, and test sets? Well, it depends on the amount of data you have and the task you are performing. Usually, machine learning engineers choose 70% of data for training, 20% for validating, and 10% for testing. But other splits are possible.

**Instructions**

1. In the image, you'll see how we use the three datasets and our hyperparameters to adjust and evaluate our model's performance:
    - We use training data to adjust the weights and biases of our model to change its fit.
    - We use validation data to evaluate the model's performance.
    - If the validation performance is good, we can use our test data to check if our model still performs well with a completely new set of data.
    - If the validation performance isn't good, we tweak our hyperparameters before retraining the model:
    - the learning rate
    - batch size
    - number of epochs
    - number of hidden layers
    - optimizer




## Hyperparameter Tuning

# Using a validation set for hyperparameter tuning

Using the training data to choose hyperparameters might lead to overfitting to the training data meaning the model learns patterns specific to the training data that would not apply to new data. For that reason, hyperparameters are chosen on a held-out set called *validation set* . In TensorFlow Keras, validation split can be specified as a parameter in the `.fit()` function:

```
my_model.fit(data, labels, epochs = 20, batch_size = 1, verbose = 1,  validation_split
= 0.2)
```

Copy to Clipboard

where `validation_split` is a float between 0 and 1, denoting a fraction of the training data to be used as validation data. In the example above, 20% of the data would be allocated for validation. It is usually a small fraction of the training data. The model will set apart this fraction of the training data, will not train on it, and will evaluate the loss and any model metrics on this data at the end of each epoch.

**Instructions**

Checkpoint 1 Passed

**1.**

Use the `.fit()` function to fit the model instance `model` to the training data `features_train` and training features `labels_train` with 40 `epochs`, batch size set to 8, verbose set to true (1), and validation split set to 33%.

The following code uses the `.fit()` function to fit the model instance `my_model` to the training data `data` and training features `labels` with 20 `epochs`, batch size set to 1, verbose set to true (1), and validation split set to 20%:

```
my_model.fit(data, labels, epochs = 20, batch_size = 1, verbose = 1,
validation_split = 0.2)
```

Copy to Clipboard

Replace `my_model` with `model`, `data` with `features_train`, `labels` with `labels_train`, `epochs = 20` with `epochs = 40`, `batch_size = 1` with `batch_size = 8` and `validation_split = 0.2` with `validation_split = 0.33`.

```
#see model.py file for more details
from model import design_model, features_train, labels_train

model = design_model(features_train, learning_rate = 0.01)
#your code here
model.fit(features_train, labels_train, epochs = 40, batch_size = 8, verbose = 1,
validation_split = 0.33)
```

Data = insurance.csv

```
 model.py

import tensorflow as tf
import pandas as pd
from tensorflow.keras.models import Sequential
from tensorflow.keras import layers
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import Normalizer
import matplotlib.pyplot as plt
```

```
tf.random.set_seed(42) #for reproducibility of result we always use the same seed for random number generator
dataset = pd.read_csv("insurance.csv") #read the dataset
dataset.to_csv("test.csv", index = False)

def design_model(X, learning_rate):  #our function to design the model
    model = Sequential(name="my_first_model")
    input = tf.keras.Input(shape=(X.shape[1],))
    model.add(input)
    model.add(layers.Dense(64, activation='relu')) #addinf one hidden layer
    model.add(layers.Dense(1))
    opt = tf.keras.optimizers.Adam(learning_rate = learning_rate) #setting the learning rate of Adam to the one
specified in the function parameter
    model.compile(loss='mse', metrics=['mae'], optimizer=opt)
    return model

features = dataset.iloc[:,0:6] #choose first 7 columns as features
labels = dataset.iloc[:,-1] #choose the final column for prediction
features = pd.get_dummies(features) #one hot encoding for categorical variables
features_train, features_test, labels_train, labels_test = train_test_split(features, labels, test_size=0.33,
random_state=42)

#standardize
ct = ColumnTransformer([('standardize', StandardScaler(), ['age', 'bmi', 'children'])],
remainder='passthrough')
features_train = ct.fit_transform(features_train) #gives numpy arrays
features_test = ct.transform(features_test) #gives numpy arrays
```

### Hyperparameter Tuning

# Manual tuning: learning rate

Neural networks are trained with the gradient descent algorithm and one of the most important hyperparameters in the network training is the *learning rate*. The learning rate determines how big of a change you apply to the network weights as a consequence of the error gradient calculated on a batch of training data.

A larger learning rate leads to a faster learning process at a cost to be stuck in a suboptimal solution (local minimum). A smaller learning rate might produce a good suboptimal or global solution, but it will take it much longer to converge. In the extremes, a learning rate too large will lead to an unstable learning process oscillating over the epochs. A learning rate too small may not converge or get stuck in a local minimum.

It can be helpful to test different learning rates as we change our hyperparameters. A learning rate of 1.0 leads to oscillations, 0.01 may give us a good performance, while 1e-07 is too small and almost nothing is learned within the allotted time.

**Instructions**

Checkpoint 1 Passed

**1.**

In the code on the right, we created a list of different learning rates we would like to check. For each learning rate, we fit the model and plot the loss on the training set (blue) and the validation set (orange).

Run the code on the right. Since we are trying out multiple learning rates, you will need to wait a bit to see the results.

Checkpoint 2 Enabled

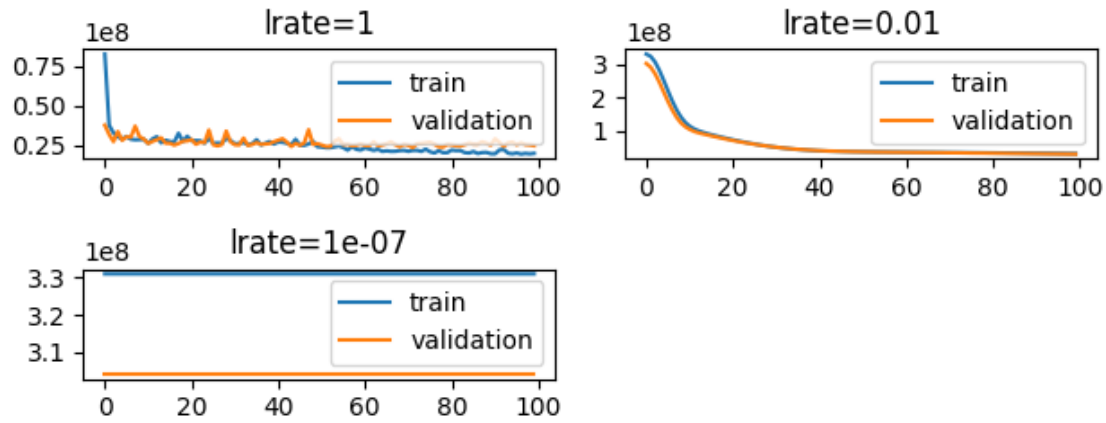
**2.**

Change the learning rates list `learning_rates` to contain the following learning rates: 1E-3, 1E-4, and 1E-7 instead. Which one gives the best performance?

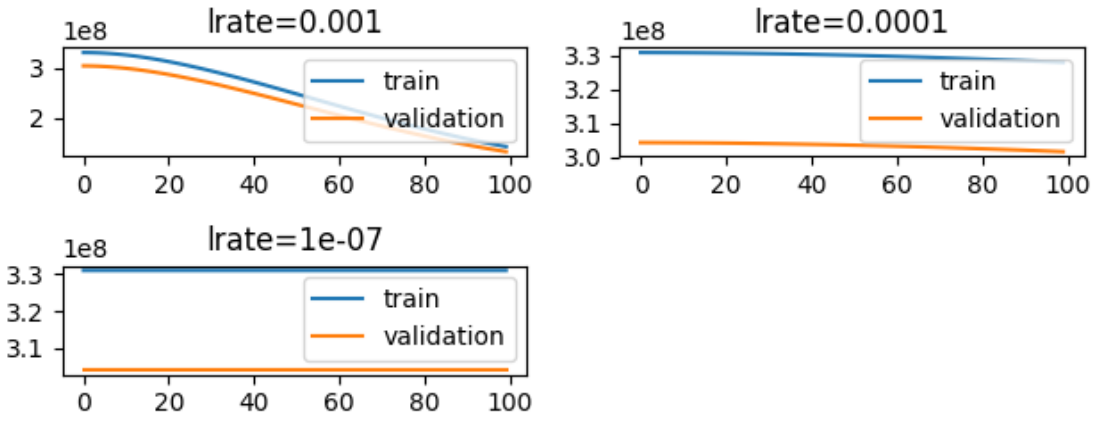
```
#see model.py file for more details
from model import design_model, features_train, labels_train
import matplotlib.pyplot as plt
```

```
def fit_model(f_train, l_train, learning_rate, num_epochs, bs):
    #build the model
    model = design_model(f_train, learning_rate)
    #train the model on the training data
    history = model.fit(f_train, l_train, epochs = num_epochs, batch_size = bs,
verbose = 0, validation_split = 0.2)
    # plot learning curves
    plt.plot(history.history['loss'], label='train')
    plt.plot(history.history['val_loss'], label='validation')
    plt.title('lrate=' + str(learning_rate))
    plt.legend(loc="upper right")


#make a list of learning rates to try out
learning_rates = [1, 1E-2, 1E-7]
learning_rates = [ 1E-3, 1E-4, 1E-7] #new learning rates
#fixed number of epochs
num_epochs = 100
#fixed number of batches
batch_size = 10

for i in range(len(learning_rates)):
  plot_no = 420 + (i+1)
  plt.subplot(plot_no)
  fit_model(features_train, labels_train, learning_rates[i], num_epochs, batch_size)

plt.tight_layout()
plt.show()
plt.savefig('static/images/my_plot.png')
print("See the plot on the right with learning rates", learning_rates)
import app #don't worry about this. This is to show you the plot in the browser.
```

```
learning_rates = [1, 1E-2, 1E-7
```




```
learning_rates = [ 1E-3, 1E-4, 1E-7] #new learning rates
```




**Hyperparameter Tuning**

# Manual tuning: batch size

The *batch size* is a hyperparameter that determines how many training samples are seen before updating the network's parameters (weight and bias matrices).
When the batch contains all the training examples, the process is called batch gradient descent. If the batch has one sample, it is called the stochastic gradient descent. And finally, when 1 < batch size < number of training points, is called mini-batch gradient descent. An advantage of using batches is for GPU computation that can parallelize neural network computations.
How do we choose the batch size for our model? On one hand, a larger batch size will provide our model with better gradient estimates and a solution close to the optimum, but this comes at a cost of computational efficiency and good generalization performance. On the other hand, smaller batch size is a poor estimate of the gradient, but the learning is performed faster. Finding the "sweet spot" depends on the dataset and the problem, and can be determined through hyperparameter tuning.
For this experiment, we fix the learning rate to 0.01 and try the following batch sizes: 1, 2, 10, and 16. Notice how small batch sizes have a larger variance (more oscillation in the learning curve).
Want to improve the performance with a larger batch size? A good trick is to increase the learning rate!

**Instructions**

Checkpoint 1 Failed, try again

**1.**

Modify the `batches` list to include the following batch sizes: 4, 32, 64. Rerun the code and observe the plots.

Checkpoint 2 Step instruction is unavailable until previous steps are completed

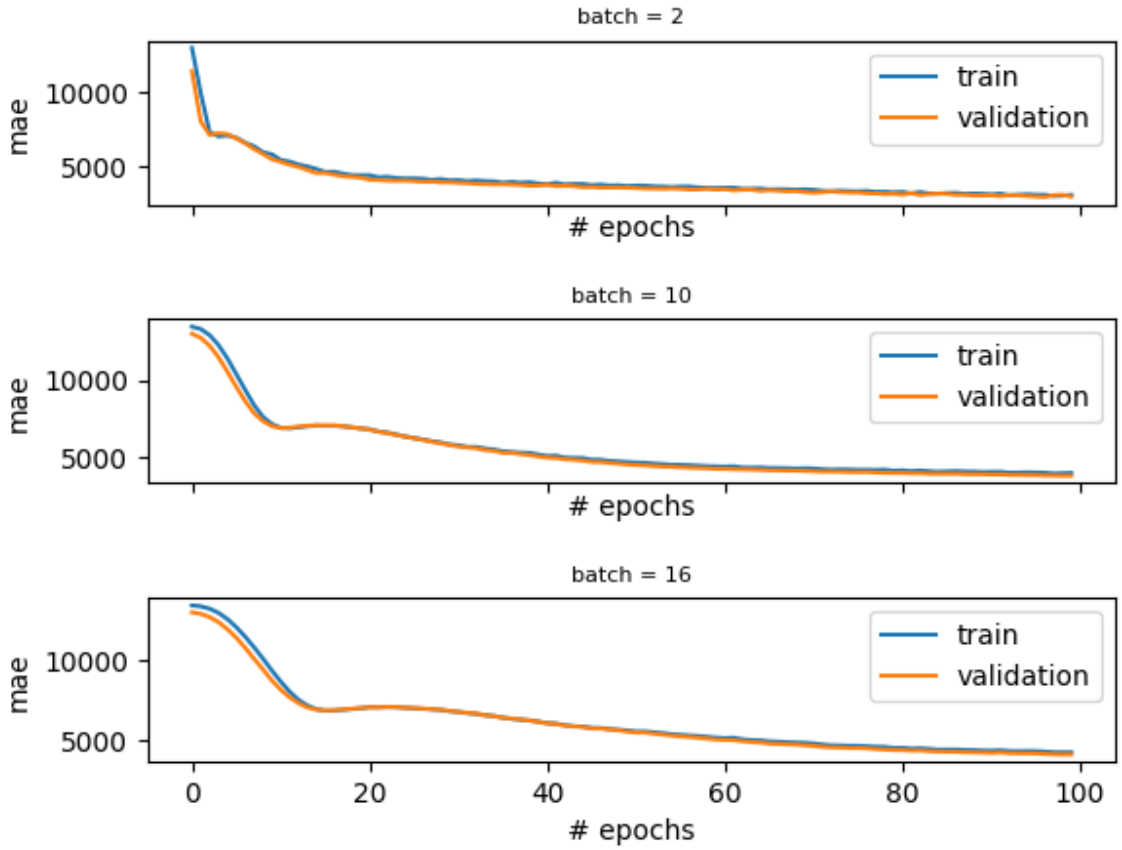
**2.**

In the previous checkpoint, you might have noticed bad performance for larger batch sizes (32 and 64). When having performance issues with larger batches it might help to increase the learning rate. Modify the value for the learning rate by assigning 0.1 to the `learning_rate` variable. Rerun the code and observe the plots.

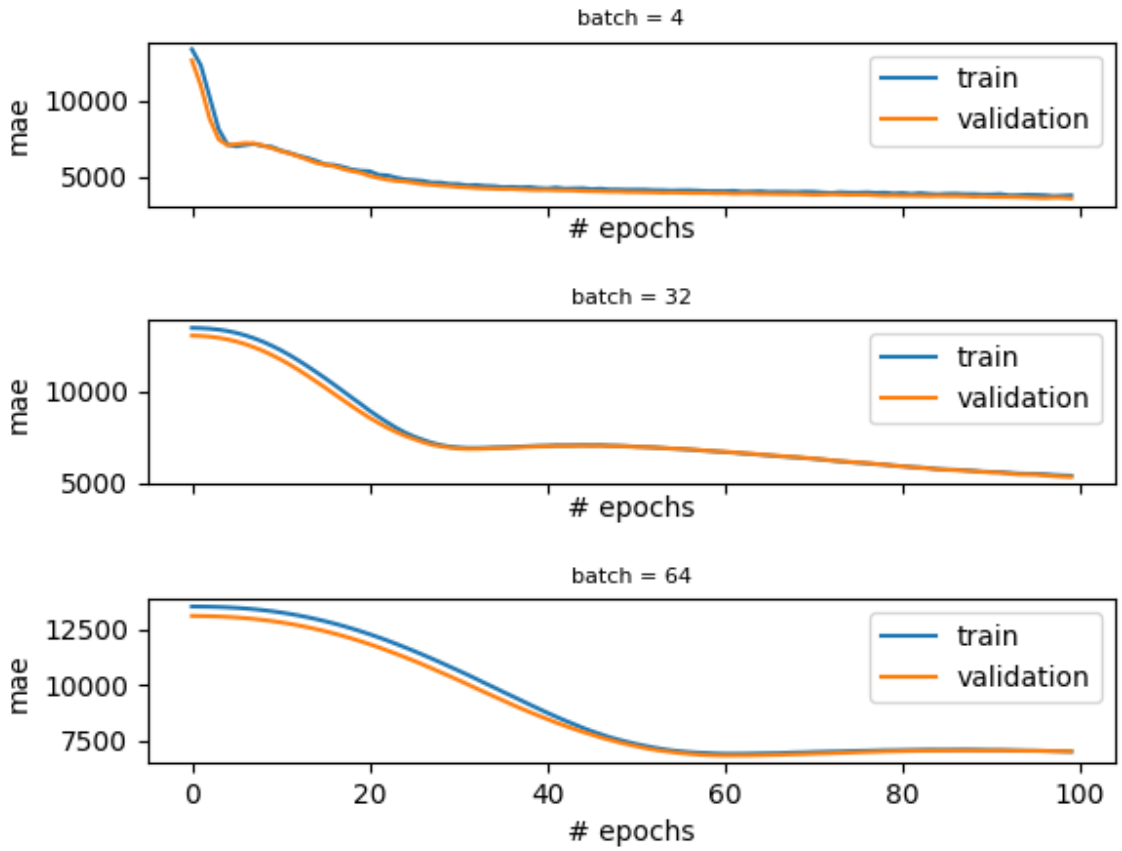
```
from model import features_train, labels_train, design_model
import matplotlib.pyplot as plt

def fit_model(f_train, l_train, learning_rate, num_epochs, batch_size, ax):
    model = design_model(features_train, learning_rate)
    #train the model on the training data
```

```
    history = model.fit(features_train, labels_train, epochs=num_epochs, batch_size =
batch_size, verbose=0, validation_split = 0.3)
    # plot learning curves
    ax.plot(history.history['mae'], label='train')
    ax.plot(history.history['val_mae'], label='validation')
    ax.set_title('batch = ' + str(batch_size), fontdict={'fontsize': 8, 'fontweight':
'medium'})
    ax.set_xlabel('# epochs')
    ax.set_ylabel('mae')
    ax.legend()

#fixed learning rate
learning_rate = 0.01
#fixed number of epochs
num_epochs = 100
#we choose a number of batch sizes to try out
batches = [2, 10, 16]
print("Learning rate fixed to:", learning_rate)

#plotting code
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, sharex='col', sharey='row',
                    gridspec_kw={'hspace': 0.7, 'wspace': 0.4}) #preparing axes
for plotting
axes = [ax1, ax2, ax3]

#iterate through all the batch values
for i in range(len(batches)):
  fit_model(features_train, labels_train, learning_rate, num_epochs, batches[i],
axes[i])

plt.savefig('static/images/my_plot.png')
print("See the plot on the right with batch sizes", batches)
import app #don't worry about this. This is to show you the plot in the browser.
```

```
batches = [2, 10, 16]
```



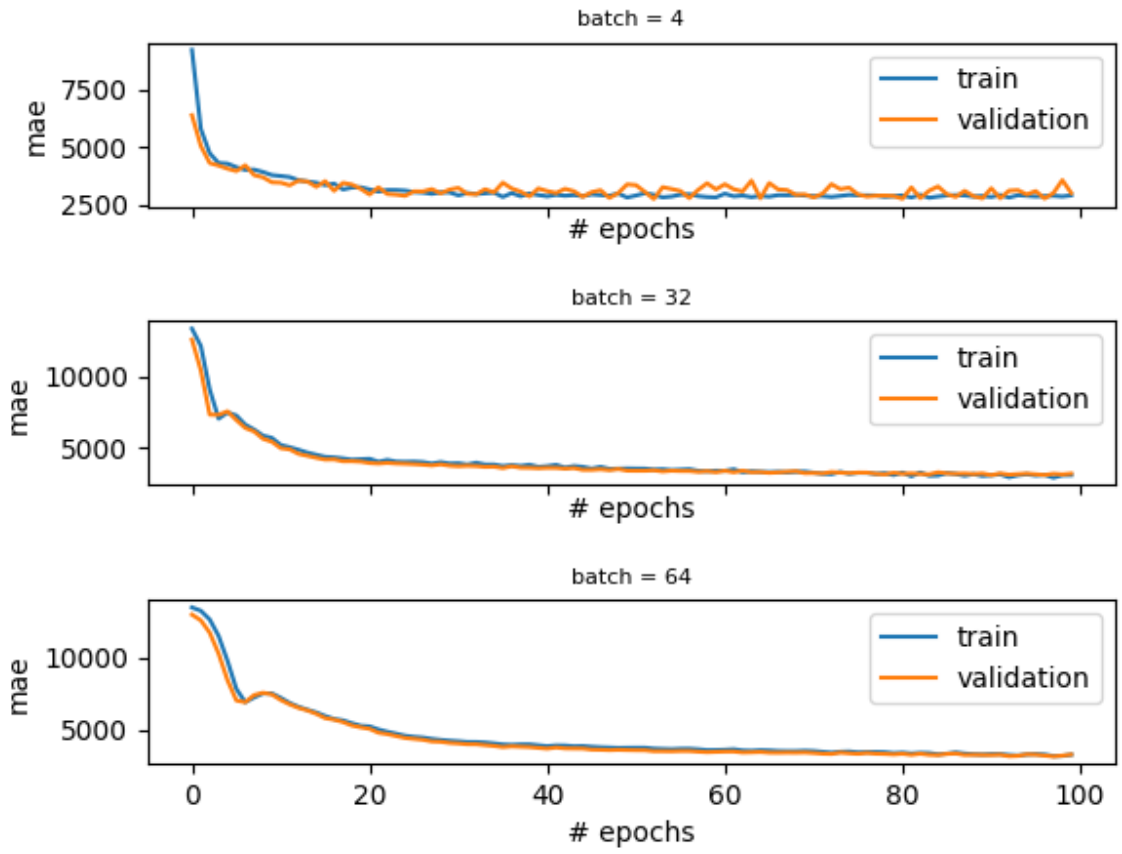

```
batches = [4, 32, 64] #new batches
```




```
from model import features_train, labels_train, design_model
import matplotlib.pyplot as plt

def fit_model(f_train, l_train, learning_rate, num_epochs, batch_size, ax):
    model = design_model(features_train, learning_rate)
    #train the model on the training data
    history = model.fit(features_train, labels_train, epochs=num_epochs, batch_size = batch_size,
verbose=0, validation_split = 0.3)
    # plot learning curves
    ax.plot(history.history['mae'], label='train')
    ax.plot(history.history['val_mae'], label='validation')
    ax.set_title('batch = ' + str(batch_size), fontdict={'fontsize': 8, 'fontweight': 'medium'})
    ax.set_xlabel('# epochs')
    ax.set_ylabel('mae')
    ax.legend()

#fixed learning rate
learning_rate = 0.1
#fixed number of epochs
num_epochs = 100
#we choose a number of batch sizes to try out
batches = [4, 32, 64]
print("Learning rate fixed to:", learning_rate)

#plotting code
fig, (ax1, ax2, ax3) = plt.subplots(3, 1, sharex='col', sharey='row',
                        gridspec_kw={'hspace': 0.7, 'wspace': 0.4}) #preparing axes for plotting
```

```
axes = [ax1, ax2, ax3]

#iterate through all the batch values
for i in range(len(batches)):
  fit_model(features_train, labels_train, learning_rate, num_epochs, batches[i], axes[i])

plt.savefig('static/images/my_plot.png')
print("See the plot on the right with batch sizes", batches)
import app #don't worry about this. This is to show you the plot in the browser.
```

```
#fixed learning rate                batches
learning_rate = 0.1;                batches = [4, 32, 64]
```




**Hyperparameter Tuning**

# Manual tuning: epochs and early stopping

Being an iterative process, gradient descent iterates many times through the training data. The number of *epochs* is a hyperparameter representing the number of complete passes through the training dataset. This is typically a large number (100, 1000, or larger). If the data is split into batches, in one epoch the optimizer will see all the batches.

How do you choose the number of epochs? Too many epochs can lead to overfitting, and too few to underfitting. One trick is to use *early stopping*: when the training performance reaches the plateau or starts degrading, the learning stops.

To illustrate we can introduce some overfitting by increasing the number of parameters in the neural network

Preview: Docs A neural network, inspired by the human brain, is a model that learns from data to make decisions and solve complex problems.

model and get the following plot:

We know we are overfitting because the validation error at first decreases but eventually starts increasing. The final validation MAE is ~3034, while the training MAE is ~1000. That's a big difference. We see that the training could have been stopped earlier (around epoch 50).

We can specify early stopping in TensorFlow with Keras by creating an `EarlyStopping` callback and adding it as a parameter when we fit our model.

```
from tensorflow.keras.callbacks import EarlyStopping

stop = EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience=40)

history = model.fit(features_train, labels_train, epochs=num_epochs, batch_size=16,
verbose=0, validation_split=0.2, callbacks=[stop])
```

Copy to Clipboard

Here, we include the following:

- `monitor = val_loss`, which means we are monitoring the validation loss to decide when to stop the training
- `mode = min`, which means we seek minimal loss
- `patience = 40`, which means that if the learning reaches a plateau, it will continue for 40 more epochs in case the plateau leads to improved performance

**Instructions**

Checkpoint 1 Enabled

**1.**

In the `fit_model()` method, just before calling `model.fit()`, create an instance of `EarlyStopping` that monitors the validation loss (`val_loss`), seeks minimal loss, that is verbose, and has patience equal to 20. Assign the result to a variable called `es`.

To create an instance of `EarlyStopping` we do the following:

```
stop = EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience = 40)
```

Copy to Clipboard

Replace `stop` with `es`, and the value of `patience` with 20.

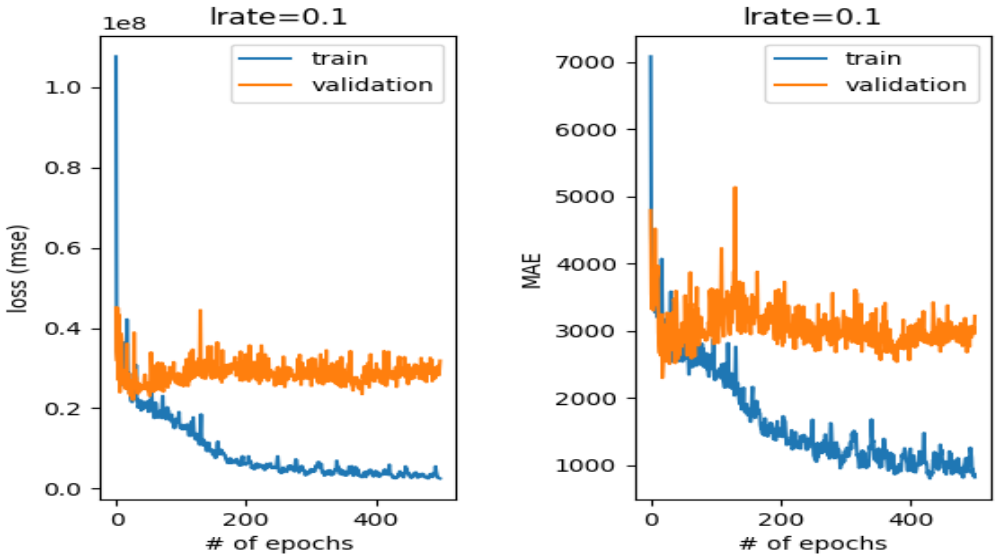
Checkpoint 2 Step instruction is unavailable until previous steps are completed

**2.**

Now that you have an instance of `EarlyStopping` assigned to `es`, you need to pass the instance as a callback function to the `.fit()` method. We left an empty list for you there: `callback = []`. Put `es` as a single element in the `callback`. Run the code. The early stopping epoch should be 47. Please verify.

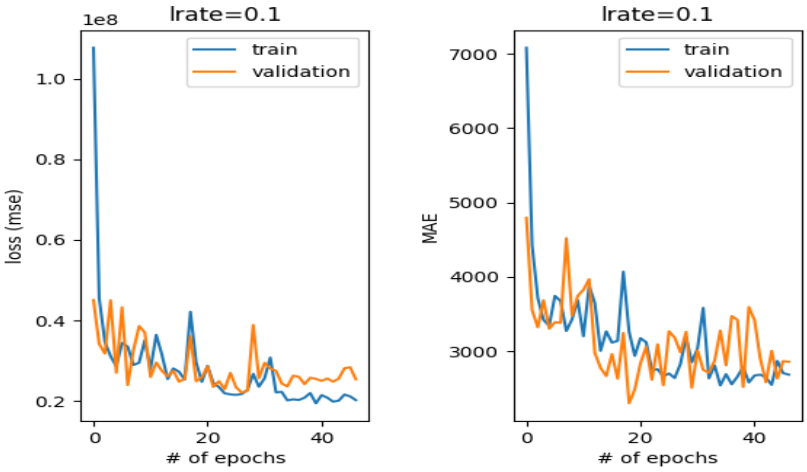
```
from model import features_train, labels_train, design_model
import matplotlib.pyplot as plt
from tensorflow.keras.callbacks import EarlyStopping

def fit_model(f_train, l_train, learning_rate, num_epochs):
    #build the model: to see the specs go to model.pyl we increased the number of
hidden neurons
    #in order to introduce some overfitting
    model = design_model(features_train, learning_rate)
    #train the model on the training data
```

```
    #your code here
    history = model.fit(features_train, labels_train, epochs=num_epochs, batch_size=
16, verbose=0, validation_split = 0.2, callbacks = [])
    return history


#using the early stopping in fit_model
learning_rate = 0.1
num_epochs = 500
history = fit_model(features_train, labels_train, learning_rate, num_epochs)

#plotting
fig, axs = plt.subplots(1, 2, gridspec_kw={'hspace': 1, 'wspace': 0.5})
(ax1, ax2) = axs
ax1.plot(history.history['loss'], label='train')
ax1.plot(history.history['val_loss'], label='validation')
ax1.set_title('lrate=' + str(learning_rate))
ax1.legend(loc="upper right")
ax1.set_xlabel("# of epochs")
ax1.set_ylabel("loss (mse)")

ax2.plot(history.history['mae'], label='train')
ax2.plot(history.history['val_mae'], label='validation')
ax2.set_title('lrate=' + str(learning_rate))
ax2.legend(loc="upper right")
ax2.set_xlabel("# of epochs")
ax2.set_ylabel("MAE")

print("Final training MAE:", history.history['mae'][-1])
print("Final validation MAE:", history.history['val_mae'][-1])

plt.savefig('static/images/my_plot.png')
import app #don't worry about this. This is to show you the plot in the browser.
```




```
from model import features_train, labels_train, design_model
```

```
import matplotlib.pyplot as plt
from tensorflow.keras.callbacks import EarlyStopping

def fit model(f train, l train, learning rate, num epochs):
    #build the model: to see the specs go to model.pyl we increased the number of hidden neurons
    #in order to introduce some overfitting
    model = design model(features train, learning rate)
    #train the model on the training data
    #your code here
    es = EarlyStopping(monitor='val loss', mode='min', verbose=1, patience = 20)
    history = model.fit(features_train, labels_train, epochs=num_epochs, batch_size= 16,
verbose=0, validation_split = 0.2, callbacks = [es])
    return history


#using the early stopping in fit model
learning_rate = 0.1
num_epochs = 500
history = fit_model(features_train, labels_train, learning_rate, num_epochs)

#plotting
fig, axs = plt.subplots(1, 2, gridspec kw={'hspace': 1, 'wspace': 0.5})
(ax1, ax2) = axs
ax1.plot(history.history['loss'], label='train')
ax1.plot(history.history['val loss'], label='validation')
ax1.set_title('lrate=' + str(learning_rate))
ax1.legend(loc="upper right")
ax1.set_xlabel("# of epochs")
ax1.set_ylabel("loss (mse)")

ax2.plot(history.history['mae'], label='train')
ax2.plot(history.history['val_mae'], label='validation')
ax2.set title('lrate=' + str(learning rate))
ax2.legend(loc="upper right")
ax2.set_xlabel("# of epochs")
ax2.set_ylabel("MAE")

print("Final training MAE:", history.history['mae'][-1])
print("Final validation MAE:", history.history['val mae'][-1])

plt.savefig('static/images/my_plot.png')

Epoch 00047: early stopping
Final training MAE: 2685.82568359375
Final validation MAE: 2859.48974609375
```




## Hyperparameter Tuning

# Manual tuning: changing the model

We saw in the previous exercise that if you have a big model and you train too long, you might overfit. Let us see the opposite - having a too simple model.
In the code on the right, we compare a one-layer
neural network
Preview: Docs Loading link description
and a model with a single hidden layer. The models look like this:

```
def one_layer_model(X, learning_rate):
    ...
    model.add(input)
    model.add(layers.Dense(1))
    ...
```

Copy to Clipboard

and

```
def more_complex_model(X, learning_rate):
    ...
    model.add(input)
    model.add(layers.Dense(64, activation='relu'))
    model.add(layers.Dense(1))
```

Copy to Clipboard

When we run the learning we get the learning curves depicted on the far right.
If you observe Plot #1 for the model with no hidden layers you will see an issue: the validation curve is below the training curve. This means that the training curve can get better at some point, but the model complexity doesn't allow it. This phenomenon is called underfitting. You can also notice that no early stopping occurs here since the performance of this model is bad all the time.
Plot #2 is for the model with a single hidden layer. You can observe a well-behaving curve with the early stopping at epoch 38 and we have a much better result. Nice!
How do we choose the number of hidden layers and the number of units per layer? That is a tough question and there is no good answer. The rule of thumb is to start with one hidden layer and add as many units as we have features in the dataset. However, this might not always work. We need to try things out and observe our learning curve.

**Instructions**

Checkpoint 1 Enabled

**1.**

Decrease the number of hidden units of the model from 64 to 8 in the `more_complex_model()` method. When does the early stopping happen?

The following code adds a hidden layer with 64 neurons to our model:

```
model.add(layers.Dense(64, activation='relu'))
```

Copy to Clipboard

Replace 64 with 8.

It was shown that larger models (64) units might converge faster.

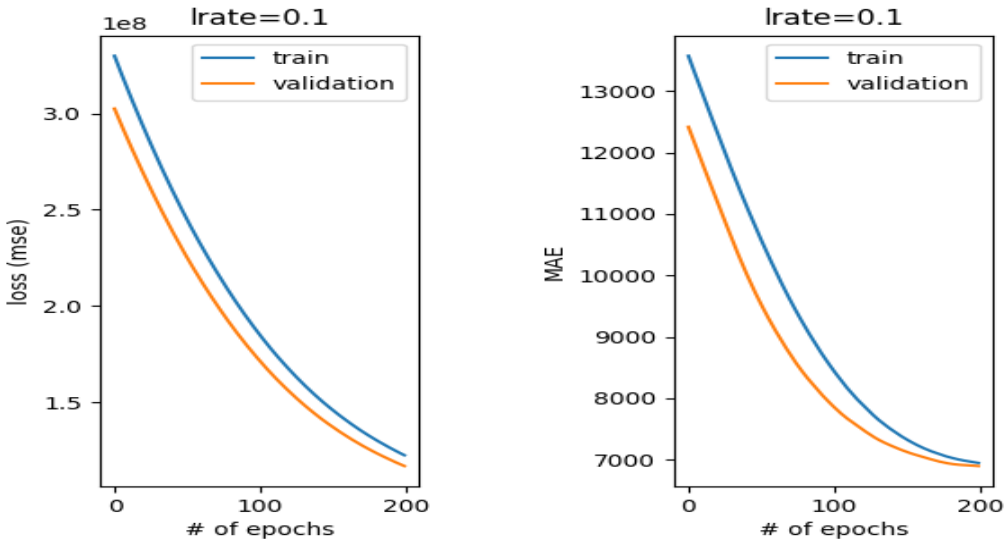
```
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras import layers
from tensorflow.keras.callbacks import EarlyStopping
import matplotlib.pyplot as plt
from model import features_train, labels_train

def more_complex_model(X, learning_rate):
    model = Sequential(name="my_first_model")
    input = tf.keras.Input(shape=(X.shape[1],))
    model.add(input)
```

```
    model.add(layers.Dense(64, activation='relu'))
    model.add(layers.Dense(1))
    opt = tf.keras.optimizers.Adam(learning_rate = learning_rate)
    model.compile(loss='mse', metrics=['mae'], optimizer=opt)
    return model

def one_layer_model(X, learning_rate):
    model = Sequential(name="my_first_model")
    input = tf.keras.Input(shape=(X.shape[1],))
    model.add(input)
    model.add(layers.Dense(1))
    opt = tf.keras.optimizers.Adam(learning_rate = learning_rate)
    model.compile(loss='mse', metrics=['mae'], optimizer=opt)
    return model

def fit_model(model, f_train, l_train, learning_rate, num_epochs):
    #train the model on the training data
    es = EarlyStopping(monitor='val_loss', mode='min', verbose=1, patience = 20)
    history = model.fit(features_train, labels_train, epochs=num_epochs, batch_size=
2, verbose=0, validation_split = 0.2, callbacks = [es])
    return history

def plot(history):
    # plot learning curves
    fig, axs = plt.subplots(1, 2, gridspec_kw={'hspace': 1, 'wspace': 0.8})
    (ax1, ax2) = axs
    ax1.plot(history.history['loss'], label='train')
    ax1.plot(history.history['val_loss'], label='validation')
    ax1.set_title('lrate=' + str(learning_rate))
    ax1.legend(loc="upper right")
    ax1.set_xlabel("# of epochs")
    ax1.set_ylabel("loss (mse)")

    ax2.plot(history.history['mae'], label='train')
    ax2.plot(history.history['val_mae'], label='validation')
    ax2.set_title('lrate=' + str(learning_rate))
    ax2.legend(loc="upper right")
    ax2.set_xlabel("# of epochs")
    ax2.set_ylabel("MAE")
    print("Final training MAE:", history.history['mae'][-1])
    print("Final validation MAE:", history.history['val_mae'][-1])

learning_rate = 0.1
num_epochs = 200

#fit the more simple model
print("Results of a one layer model:")
history1 = fit_model(one_layer_model(features_train, learning_rate), features_train,
labels_train, learning_rate, num_epochs)
plot(history1)
plt.savefig('static/images/my_plot1.png')

#fit the more complex model
print("Results of a model with hidden layers:")
history2 = fit_model(more_complex_model(features_train, learning_rate),
features_train, labels_train, learning_rate, num_epochs)
plot(history2)
plt.savefig('static/images/my_plot2.png')
```

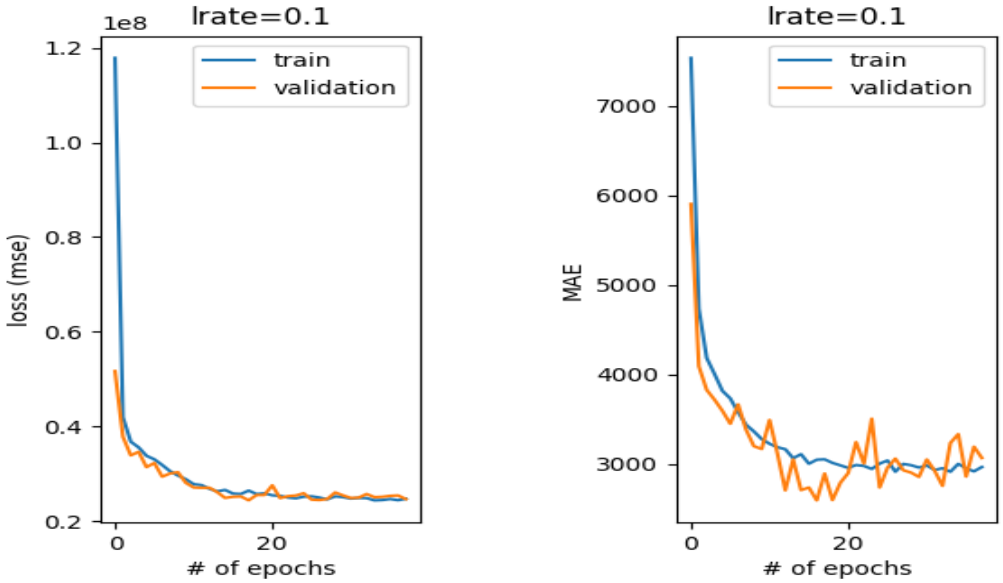
```
import app #don't worry about this. This is to show you the plot in the browser.
```

# Hyperparameter tuning: model size

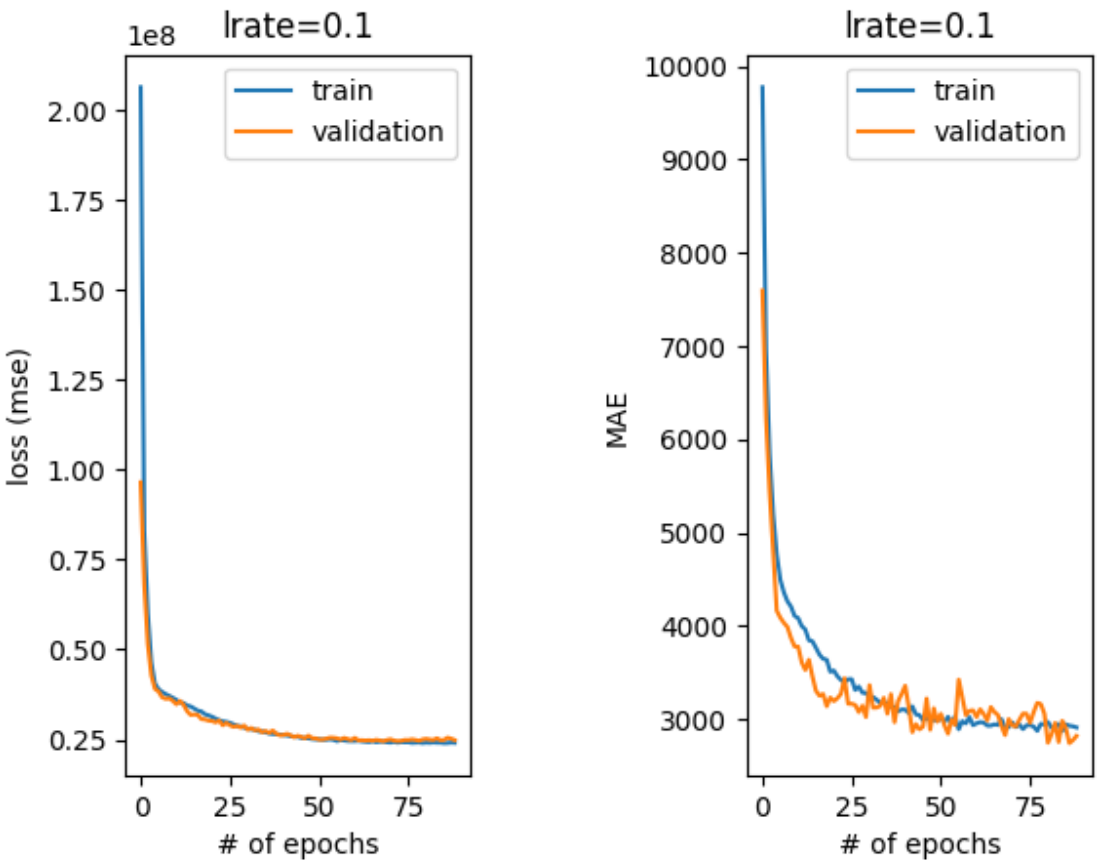
**Plot #1: a model without hidden layers**




**Plot #2: a model with one hidden layer (64 NEURONS)**



**Plot #2: a model with one hidden layer (8 NEURONS)**




**Hyperparameter Tuning**

# Towards automated tuning: grid and random search

So far we've been manually setting and adjusting hyperparameters to train and evaluate our model. If we didn't like the result, we changed the hyperparameters to some other values. However, this is rather cumbersome; it would be nice if we could make these changes in a systematic and automated way. Fortunately, there are some strategies for automated hyperparameter tuning, including the following two. *Grid search*, or exhaustive search, tries every combination of desired hyperparameter values. If, for example, we want to try learning rates of 0.01 and 0.001 and batch sizes of 10, 30, and 50, grid search will try six combinations of parameters (0.01 and 10, 0.01 and 30, 0.01 and 50, 0.001 and 10, and so on). This obviously gets very computationally demanding when we increase the number of values per hyperparameter or the number of hyperparameters we want to tune.

On the other hand, *Random Search* goes through random combinations of hyperparameters and doesn't try them all.

**Grid search in Keras**

To use `GridSearchCV` from scikit-learn for regression we need to first wrap our [neural network](#)

Preview: Docs Loading link description

model into a `KerasRegressor`:

`model = KerasRegressor(build_fn=design_model)`

Then we need to setup the desired hyperparameters grid (we don't use many values for the sake of speed):

```
batch_size = [10, 40]
epochs = [10, 50]
param_grid = dict(batch_size=batch_size, epochs=epochs)
```

Copy to Clipboard

Finally, we initialize a `GridSearchCV` object and fit our model to the data:

```
grid = GridSearchCV(estimator = model, param_grid=param_grid, scoring =
make_scorer(mean_squared_error, greater_is_better=False))
grid_result = grid.fit(features_train, labels_train, verbose = 0)
```

Copy to Clipboard

Notice that we initialized the `scoring` parameter with scikit-learn's `.make_scorer()` method. We're evaluating our hyperparameter combinations with a mean squared error making sure that `greater_is_better` is set to `False` since we are searching for a set of hyperparameters that yield us the smallest error.

**Randomized search in Keras**

We first change our hyperparameter grid specification for the randomized search in order to have more options:

```
param_grid = {'batch_size': sp_randint(2, 16), 'nb_epoch': sp_randint(10,
100)}
```

Copy to Clipboard

Randomized search will sample values for `batch_size` and `nb_epoch` from uniform distributions in the interval [2, 16] and [10, 100], respectively, for a fixed number of iterations. In our case, 12 iterations:

```
grid = RandomizedSearchCV(estimator = model, param_distributions=param_grid,
scoring = make_scorer(mean_squared_error, greater_is_better=False), n_iter =
12)
```

Copy to Clipboard

We cover only simpler cases here, but you can set up `GridSearchCV` and `RandomizedSearchCV` to tune over any hyperparameters you can think of: optimizers, number of hidden layers, number of neurons per layer, and so on.

**Instructions**

Checkpoint 1 Enabled

**1.**

Change the `batch_size` array to [6, 64] to try other batch size values. Rerun the script and observe what is the best combination for grid search in the output that looks something like this:

```
Best: -239970538.248960 using {'batch_size': 3, 'nb_epoch': 61}
```

```
from sklearn.model_selection import GridSearchCV
```

```
from sklearn.model_selection import RandomizedSearchCV
from scipy.stats import randint as sp_randint
from keras.wrappers.scikit_learn import KerasRegressor
from sklearn.metrics import mean_squared_error
from sklearn.metrics import make_scorer
from model import design_model, features_train, labels_train

def do_grid_search():
  batch_size = [6, 64]
  epochs = [10, 50]
  model = KerasRegressor(build_fn=design_model)
  param_grid = dict(batch_size=batch_size, epochs=epochs)
  grid = GridSearchCV(estimator = model, param_grid=param_grid, scoring =
make_scorer(mean_squared_error, greater_is_better=False),return_train_score =
True)
  grid_result = grid.fit(features_train, labels_train, verbose = 0)
  print(grid_result)
  print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))

  means = grid_result.cv_results_['mean_test_score']
  stds = grid_result.cv_results_['std_test_score']
  params = grid_result.cv_results_['params']
  for mean, stdev, param in zip(means, stds, params):
      print("%f (%f) with: %r" % (mean, stdev, param))

  print("Traininig")
  means = grid_result.cv_results_['mean_train_score']
  stds = grid_result.cv_results_['std_train_score']
  for mean, stdev, param in zip(means, stds, params):
      print("%f (%f) with: %r" % (mean, stdev, param))

#------------- RANDOMIZED SEARCH --------------
def do_randomized_search():
  param_grid = {'batch_size': sp_randint(2, 16), 'nb_epoch': sp_randint(10, 100)}
  model = KerasRegressor(build_fn=design_model)
  grid = RandomizedSearchCV(estimator = model, param_distributions=param_grid,
scoring = make_scorer(mean_squared_error, greater_is_better=False), n_iter = 12)
  grid_result = grid.fit(features_train, labels_train, verbose = 0)
  print(grid_result)
  print("Best: %f using %s" % (grid_result.best_score_, grid_result.best_params_))

  means = grid_result.cv_results_['mean_test_score']
  stds = grid_result.cv_results_['std_test_score']
  params = grid_result.cv_results_['params']
  for mean, stdev, param in zip(means, stds, params):
      print("%f (%f) with: %r" % (mean, stdev, param))

print("-------------- GRID SEARCH --------------------")
do_grid_search()
print("-------------- RANDOMIZED SEARCH --------------------")
do_randomized_search()
```

**Hyperparameter Tuning**

# Regularization: dropout

*Regularization* is a set of techniques that prevent the learning process to completely fit the model to the training data which can lead to overfitting. It makes the model simpler, smooths out the learning curve, and hence makes it more 'regular'. There are many techniques for regularization such as simplifying the model, adding weight regularization, weight decay, and so on. The most common regularization method is *dropout*.

Dropout is a technique that randomly ignores, or "drops out" a number of outputs of a layer by setting them to zeros. The dropout rate is the percentage of layer outputs set to zero (usually between 20% to 50%).

In Keras, we can add a dropout layer by introducing the `Dropout` layer.

Let's recreate our overfitting network having too many layers and too many neurons per layer in the `design_model_no_dropout()` method in **insurance_tuning.py**. For this model, we get the learning curve depicted in Figure 1. The validation error gets worse, which indicates the trend of overfitting.

Next, we introduce dropout layers in the `design_model_dropout()` method in **insurance_tuning.py**. Our model looks like this:

```
model.add(input)
model.add(layers.Dense(128, activation = 'relu'))
model.add(layers.Dropout(0.1))
model.add(layers.Dense(64, activation = 'relu'))
model.add(layers.Dropout(0.2))
model.add(layers.Dense(24, activation = 'relu'))
#your code here!
model.add(layers.Dense(1))
```

Copy to Clipboard

For this model, we get the learning curve in Figure 2. The validation MAE we get with the dropout is lower than without it. Note that the validation error is also lower than the training error in this case. One of the explanations might be that the dropout is used only during training, and the full network is used during the validation/testing with layers' output scaled down by the dropout rate.

**Instructions**

Checkpoint 1 Enabled

**1.**

In the `design_model_dropout()` method, after the hidden layer with 24 neurons, add another dropout method as an instance of `tensorflow.keras.layers.Dropout` with the dropout rate set to 0.3.

The following code adds another dropout method as an instance of `tensorflow.keras.layers.Dropout` with the dropout rate set to 0.2:

```
model.add(layers.Dropout(0.2))
```

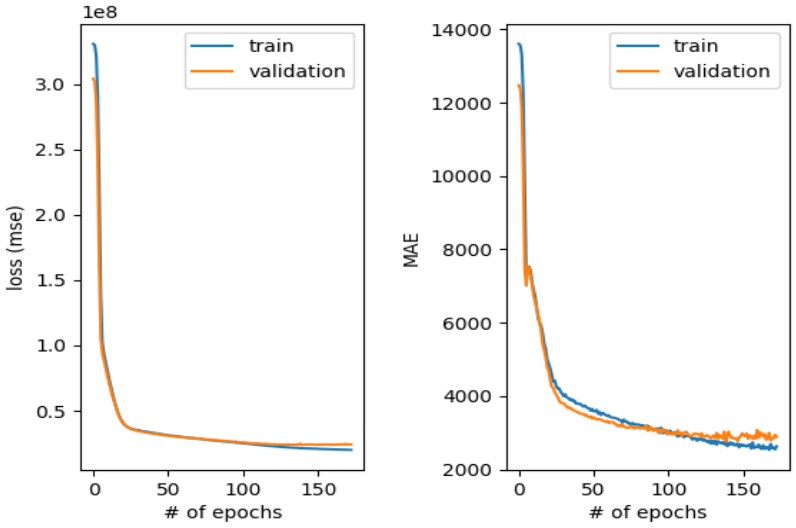
Copy to Clipboard

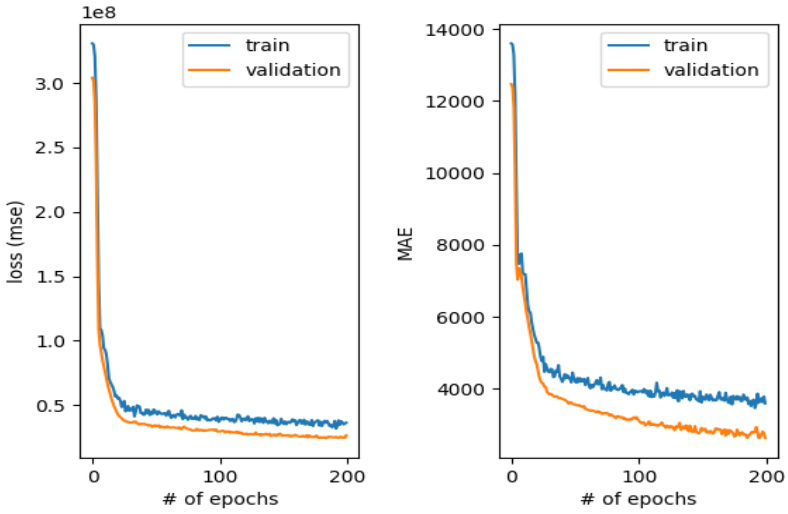
Replace 0.2 with 0.3.

```
from model import features_train, labels_train, fit_model
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras import layers
from tensorflow.keras.callbacks import EarlyStopping
from plotting import plot
import matplotlib.pyplot as plt

def design_model_dropout(X, learning_rate):
    model = Sequential(name="my_first_model")
    input = tf.keras.Input(shape=(X.shape[1],))
    model.add(input)
    model.add(layers.Dense(128, activation='relu'))
    model.add(layers.Dropout(0.1))
```

```
    model.add(layers.Dense(64, activation='relu'))
    model.add(layers.Dropout(0.2))
    model.add(layers.Dense(24, activation='relu'))
    #------your code here!------
    model.add(layers.Dropout(0.3))

    model.add(layers.Dense(1))
    opt = tf.keras.optimizers.Adam(learning_rate = learning_rate)
    model.compile(loss='mse', metrics=['mae'], optimizer=opt)
    return model

def design_model_no_dropout(X, learning_rate):
    model = Sequential(name="my_first_model")
    input = layers.InputLayer(input_shape=(X.shape[1],))
    model.add(input)
    model.add(layers.Dense(128, activation='relu'))
    model.add(layers.Dense(64, activation='relu'))
    model.add(layers.Dense(24, activation='relu'))
    model.add(layers.Dense(1))
    opt = tf.keras.optimizers.Adam(learning_rate = learning_rate)
    model.compile(loss='mse', metrics=['mae'], optimizer=opt)
    return model

#using the early stopping in fit_model
learning_rate = 0.001
num_epochs = 200
#train the model without dropout
history1 = fit_model(design_model_no_dropout(features_train, learning_rate),
features_train, labels_train, learning_rate, num_epochs)
#train the model with dropout
history2 = fit_model(design_model_dropout(features_train, learning_rate), features_train,
labels_train, learning_rate, num_epochs)

plot(history1, 'static/images/no_dropout.png')
plot(history2, 'static/images/with_dropout.png')
```

# Regularization: dropout






**Hyperparameter Tuning**

# Baselines: how to know the performance is reasonable?

Why do we need a *baseline*? For example, we have data consisting of 90% dog images, and 10% cat images. An algorithm that predicts the majority class for each data point, will have 90% accuracy on this dataset! That might sound good, but predicting the majority class is hardly a useful classifier. We need to perform better.
A baseline result is the simplest possible prediction. For some problems, this may be a random result, and for others, it may be the most common class prediction. Since we are focused on a regression task in this lesson, we can use averages or medians of the class distribution known as central tendency measures as the result for all predictions.
Scikit-learn provides `DummyRegressor`, which serves as a baseline regression algorithm. We'll choose `mean` (average) as our central tendency measure.

```
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error
dummy_regr = DummyRegressor(strategy="mean")
dummy_regr.fit(features_train, labels_train)
y_pred = dummy_regr.predict(features_test)
MAE_baseline = mean_absolute_error(labels_test, y_pred)
print(MAE_baseline)
```

Copy to Clipboard

The result of the baseline is $9,190, and we definitely did better than this (around $3,000) in our previous experiments in this lesson.

**Instructions**

Checkpoint 1 Enabled

**1.**

Change the value of the `strategy` parameter in the `DummyRegressor` from `mean` to `median`.

The following code initializes a DummyRegressor object:

```
dummy_regr = DummyRegressor(strategy="mean")
```

Copy to Clipboard

Change `mean` to `median`.

```
#see model.py file for more details
from model import features_train, labels_train, features_test, labels_test
from sklearn.dummy import DummyRegressor
from sklearn.metrics import mean_absolute_error


dummy_regr = DummyRegressor(strategy="median")
dummy_regr.fit(features_train, labels_train)
y_pred = dummy_regr.predict(features_test)
MAE_baseline = mean_absolute_error(labels_test, y_pred)
print(MAE_baseline)
```

### Hyperparameter Tuning

## Summary

In this lesson, you learned how to both manually and automatically choose hyperparameters of the neural network
Preview: Docs Loading link description
training procedure in order to select a model with the best predictive performance on a validation set. The hyperparameters we covered in this lesson are

- learning rate
- batch size
- number of epochs
- model size (number of hidden layers/neurons and number of parameters)
- regularization (dropout)

We discussed the concepts of underfitting (having a too simple model to capture data patterns) and overfitting (having a model with too many parameters that learns the training data too well and is unable to generalize). We discussed methods to combat overfitting such as regularization. To avoid underfitting we increased the complexity of our model.
Besides data preprocessing, hyperparameter tuning is probably the most costly and intensive process of neural network training. We covered how to set up grid seach and randomized search in Keras in order to automate the process of hyperparameter tuning.
We also showed you how to check the performance of your model against a simple baseline. Baselines give you an idea of whether your model has a reasonable performance.
In the end, we hope you see how all of the hyperparameters interplay and how they can influence the performance of the network.

```
from model import features_train, labels_train, fit_model
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras import layers
from tensorflow.keras.callbacks import EarlyStopping
from plotting import plot
import matplotlib.pyplot as plt

def design_model_dropout(X, learning_rate):
    model = Sequential(name="my_first_model")
    input = tf.keras.Input(shape=(X.shape[1],))
    model.add(input)
    model.add(layers.Dense(128, activation='relu'))
    model.add(layers.Dropout(0.1))
    model.add(layers.Dense(64, activation='relu'))
    model.add(layers.Dropout(0.2))
    model.add(layers.Dense(24, activation='relu'))
```

```
    #------your code here!------


    model.add(layers.Dense(1))
    opt = tf.keras.optimizers.Adam(learning_rate = learning_rate)
    model.compile(loss='mse', metrics=['mae'], optimizer=opt)
    return model

def design_model_no_dropout(X, learning_rate):
    model = Sequential(name="my_first_model")
    input = layers.InputLayer(input_shape=(X.shape[1],))
    model.add(input)
    model.add(layers.Dense(128, activation='relu'))
    model.add(layers.Dense(64, activation='relu'))
    model.add(layers.Dense(24, activation='relu'))
    model.add(layers.Dense(1))
    opt = tf.keras.optimizers.Adam(learning_rate = learning_rate)
    model.compile(loss='mse', metrics=['mae'], optimizer=opt)
    return model

#using the early stopping in fit_model
learning_rate = 0.001
num_epochs = 200
#train the model without dropout
history1 = fit_model(design_model_no_dropout(features_train, learning_rate),
features_train, labels_train, learning_rate, num_epochs)
#train the model with dropout
history2 = fit_model(design_model_dropout(features_train, learning_rate), features_train,
labels_train, learning_rate, num_epochs)

plot(history1, 'static/images/no_dropout.png')

plot(history2, 'static/images/with_dropout.png')

import app #don't worry about this. This is to show you the plot in the browser.
```

**Regularization: dropout**

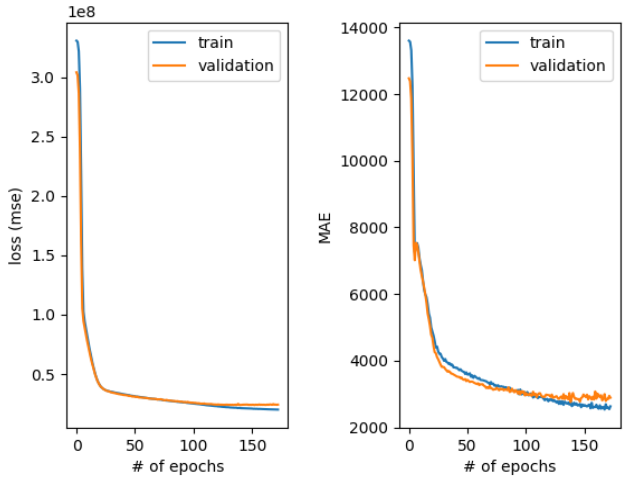



**Figure 1: learning curves without dropout**

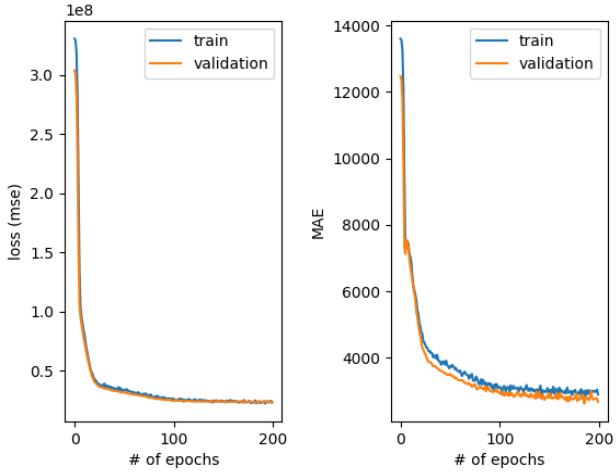



**Figure 2: learning curves with dropout**

# How to set up a deep learning environment

**This article gives detailed instructions on how to set up a deep learning environment.**

*TensorFlow* (TF), developed by Google Brain, is the most well-known library used in production for deep learning models and it has a very large community. However, TensorFlow has a steep learning curve. On the other hand, *Keras* is a high-level API built on top of TensorFlow. It is more user-friendly and easy to use compared to TF, and you can familiarize yourself more quickly because it's more "pythonic". So why not just use Keras alone? You can. But in case you want to have more control over your model network, have access to better debugging, or develop novel networks and conduct some deep learning research in the future, TensorFlow is the way.

In this tutorial, we will help you set up a TensorFlow deep learning environment. Keras API is included in TensorFlow, but you can also download and use it separately.

## Some considerations before installing: using GPU or not

A Graphics Processing Unit (GPU) is a microprocessing chip designed to handle graphics in computing environments and can have many more cores than a Central Processing Unit (CPU). More cores of GPUs allow for the better computation of multiple parallel processes. If you have access to GPU, how can you use it?

On your computer, if you installed TensorFlow using `pip`, the newest version will have the GPU support. For releases 1.15 and older, CPU and GPU packages are separate.

`pip install tensorflow` for CPU

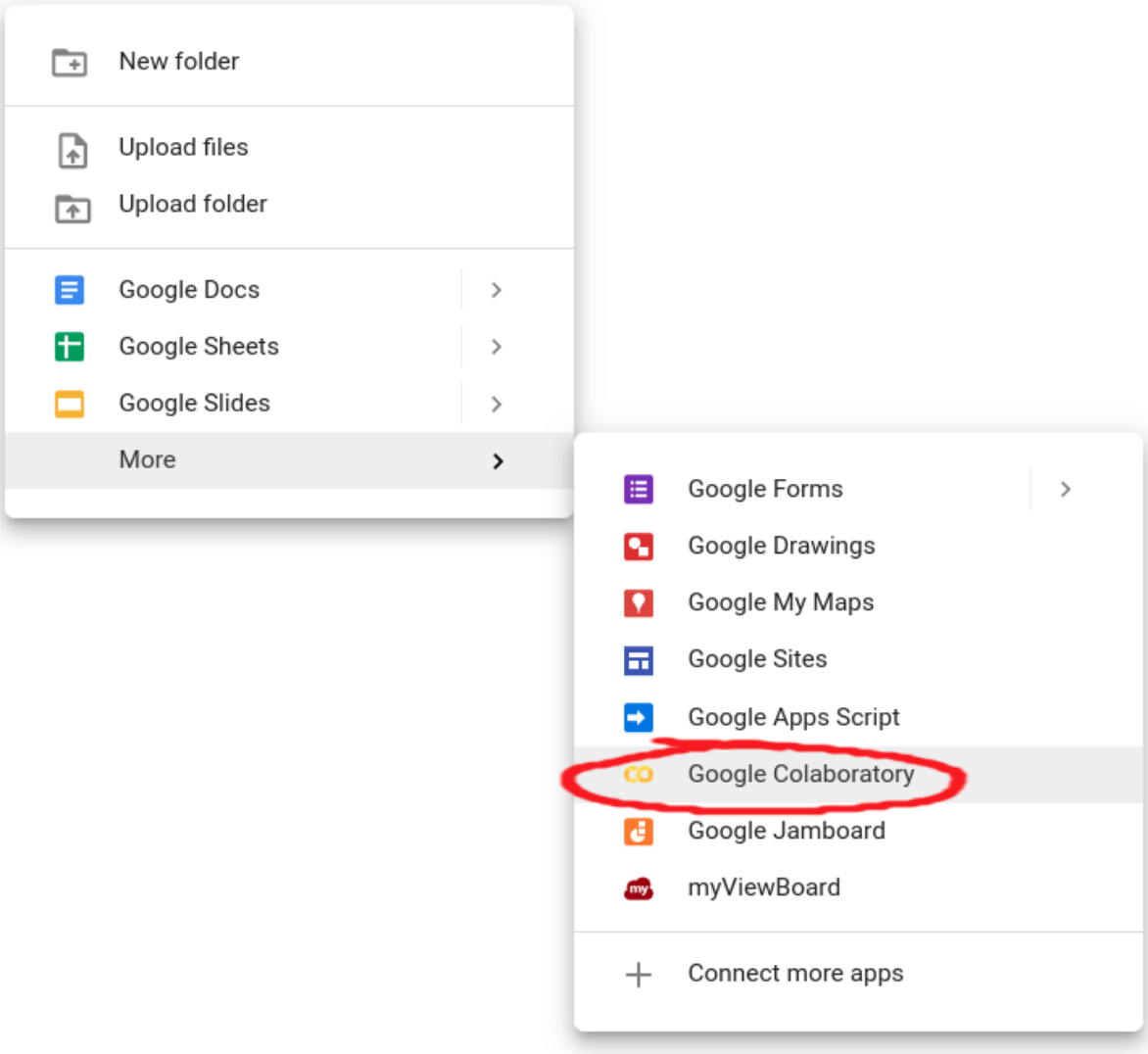
`pip install tensorflow-gpu` for GPU

However, it is not that simple since you need to also install some NVIDIA® software requirements. We suggest following TensorFlow's installation instructions for GPU support.

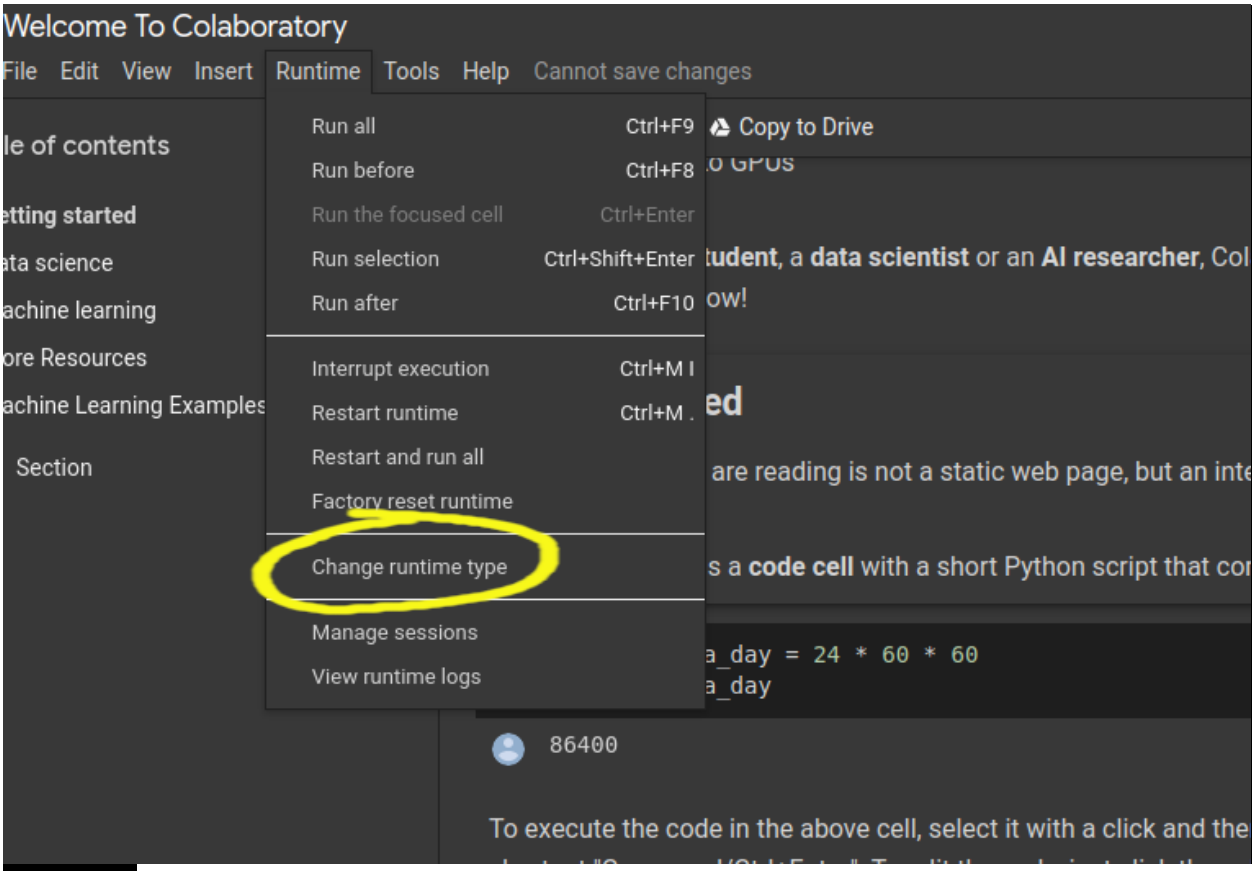
## Different ways to using TensorFlow

There are multiple ways for you to run deep learning algorithms using TensorFlow:

- **(Easy-breezy) Use Google Colaboratory (Colab).** This is the easiest option for some initial exploration. No installation is necessary - run TensorFlow code directly in the browser with Colaboratory, Google's Jupyter Notebook environment that requires no setup to use and runs entirely in the cloud. There are two ways to start a Colaboratory notebook:
  – Visit Colab
  – From your Google Drive: *Right Mouse Click* > *More* > *Colaboratory* to start a new Colab Notebook.




Switching to using a GPU is easy in Colaboratory. In *Runtime* > *Change runtime type*, select GPU, and there you are. You even have access to a special Tensor Processing Unit (TPU), an application-specific circuit developed by Google specifically for neural network machine learning.




**Your computer, your rules.** To install TensorFlow on your computer, you first need to have Python installed (version 3.5–3.8) and a Python's package manager *pip* (version >19.0.), which should have come with your Python version. TensorFlow is installed in the same way as any other package you add to your Python. It's always good to install new large packages in a virtual environment to keep it separate from your base distribution and we will provide commands for that as well.

**(Suggested) Installing with *pip*:**

Create a virtual environment with *venv* (install it if needed):

```
python3 -m venv my_tensorflow
```

Activate the virtual environment:

```
source my_tensorflow/bin/activate
```

First upgrade pip:

```
pip install --upgrade pip
```

Use *pip* to install TensorFlow in your virtual environment:

```
pip install --upgrade tensorflow
```

Note: with the new versions of TF (version > 2.0) by installing with *pip*, you don't need to specifically install the GPU supported version: *tensorflow-gpu*.

**The procedure with Anaconda/Miniconda:**

Create a virtual environment using the following command:

`conda create -n my_tensorflow` (instead of *my_tensorflow* you can give any name)

Activate the environment:

```
conda activate my_tensorflow
```

Now that you are in `my_tensorflow` virtual environment you can download TensorFlow (for CPU)

with:

```
conda install -c anaconda tensorflow
```

If you wish to use TensorFlow with GPU support use:

```
conda install -c anaconda tensorflow-gpu
```

You can complete the steps above in Anaconda Navigator which provides a Graphical User Interface (GUI) if you don't want to use the command line. Be careful about the distinction between the CPU and GPU version when using Anaconda.

- **Takes some footwork: use Docker.** Docker is a software platform for building applications based on containers — small and lightweight execution environments that make shared use of the operating system kernel but otherwise run in isolation from one another. For this, you need to install Docker on your machine. Depending on your operating system, procedures can differ. Using Docker is the easiest way to set up GPU support.

## How to check if you successfully downloaded TensorFlow?

You can test your installation by entering Python's interactive mode and typing the following code:

```
import tensorflow as tf

print(tf.__version__) #will print the version of your TensorFlow
```

The community often uses `tf` as a module handle, and by also using it will enable you to copy-paste some code from the Internet as you are learning, and it will already be compatible with your import handle.

## How to access the functionalities of Keras?

As we mentioned, Keras is built on top of TensorFlow. You can access its functionalities directly if you install the Keras Python package:

```
import keras
```

or through TensorFlow:

```
from tensorflow import keras
```

---

**Happy coding!**

# Review: Getting Started with TensorFlow

**Review what's covered in Getting Started with TensorFlow.**

## Getting Started with TensorFlow

### Goals of this Unit

Congratulations! The goal of this unit was to take the abstract concepts about deep learning you have learned and translate them into Python using TensorFlow. You have coded your own neural networks and fine-tuned them to create powerful deep learning regression models.

Having completed this unit, you are now able to:

- Translate abstract deep learning concepts into Python
- Design a neural network from scratch
- Use deep learning models to solve regression problems
- Tune the hyperparameters of your model and improve its performance through iterative testing

Happy coding!

# Classification

# Introduction: Classification

**See what's covered in Classification.**

## Classification

### Goals of this Unit

The goal of this unit is to apply the deep learning concepts you have explored to learn a classification model. In this section, you will work with tabular data and image data to diversify your deep learning skillset and gain an array of neural network concepts at your disposal. With these in hand, you will be able to work on a wider array of projects such as classifying pictures of galaxies!

After this unit, you will be able to:

- Use your deep learning knowledge to perform classification
- Work with different types of data, including images
- Increase your deep learning skillset to tackle a wider array of projects

Happy coding!

# Regression vs. Classification

**Learn about the two types of Supervised Learning algorithms.**

Machine Learning is a set of many different techniques that are each suited to answering different types of questions.

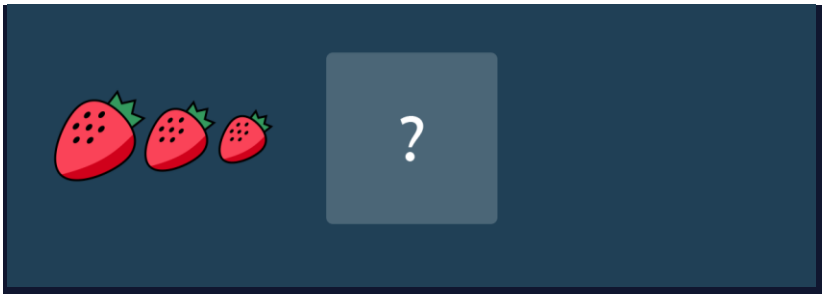
One way of categorizing machine learning algorithms is by using the kind output they produce. In terms of output, two main types of machine learning models exist: those for regression and those for classification.

### Regression

Regression is used to predict outputs that are *continuous*. The outputs are quantities that can be flexibly determined based on the inputs of the model rather than being confined to a set of possible labels.

For example:

- Predict the height of a potted plant from the amount of rainfall
- Predict salary based on someone's age and availability of high-speed internet
- Predict a car's MPG (miles per gallon) based on size and model year



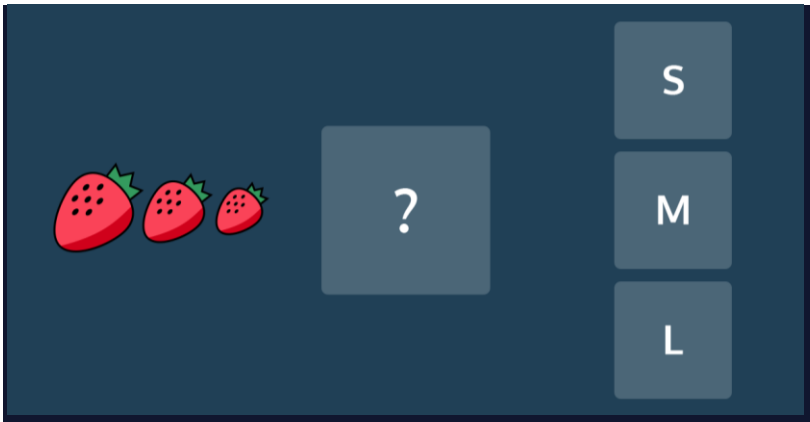

Linear regression is the most popular regression algorithm. It is often underrated because of its relative simplicity. In a business setting, it could be used to predict the likelihood that a customer will churn or the revenue a customer will generate. More complex models may fit this data better, at the cost of losing simplicity.

## Classification

Classification is used to predict a *discrete label*. The outputs fall under a finite set of possible outcomes. Many situations have only two possible outcomes. This is called *binary classification* (True/False, 0 or 1, 🌭 / not 🌭).
For example:

- Predict whether an email is spam or not
- Predict whether it will rain or not
- Predict whether a user is a power user or a casual user

There are also two other common types of classification: *multi-class classification* and *multi-label classification*. Multi-class classification has the same idea behind binary classification, except instead of two possible outcomes, there are three or more.
For example:




An important note about binary and multi-class classification is that in both, each outcome has one specific label. However, in multi-label classification, there are multiple possible labels for each outcome. This is useful for customer segmentation, image categorization, and sentiment analysis for understanding text. To perform these classifications, we use models like Naive Bayes, K-Nearest Neighbors, SVMs, as well as various deep learning models.

An example of multi-label classification is shown below. Here, a cat and a bird are both identified in a photo showing a classification model with more than one label as a result.


Choosing a model is a critical step in the Machine Learning process. It is important that the model fits the question at hand. When you choose the right model, you are already one step closer to getting meaningful and interesting results.

# Classification

# Introduction

Sorting the world into categories — classifying — is an important task you do every day. Human beings have always classified things and benefited largely from that:

- We classified mushrooms into edible and inedible for survival.
- We classified concepts so that we could label them with different words.
- Biologists classify animals to better understand them, astronomers classify planets into habitable or not
- You probably classify different kinds of music into enjoyable or not.

It may be of no surprise then that classifying is an important task within computer science as well. Classification is a technique to predict to which class each instance or example belongs. Supervised classification learns the prediction model from the labeled data. In this lesson, we will talk about two types of classification:

Binary

Preview: Docs Loading link description

classification results in a decision that is yes or no. For example, we want to classify whether a medical case is positive or negative, or whether an image contains a hotdog or not a hotdog.

Multi-class classification categorizes examples in one of several categories. For example, predicting whether air quality is poor, moderate, or severe, or classifying text into topics such as sports, entertainment, politics, and so on.

Because neural networks are very effective for high-dimensionality problems and able to model complex relationships between

variables

Preview: Docs Loading link description

, they are often used for classification tasks.

In this lesson, we will teach you how to perform classification in TensorFlow with Keras. The lesson will mostly focus on the following aspects:

- How to prepare data for classification with neural networks?
- Which loss function(s) to use for classification and why?
- Which activation function(s) to use in the output layer of the classification model?
- How many neurons to use in the output layer of the classification model?
- Which evaluation metric is used for classification?

**Classification**

# Cross-entropy

Before we continue loading the data and designing the model, we need to talk about *cross-entropy*, an important concept for evaluating classification model training. Cross-entropy is a score that summarizes

the average difference between the actual and predicted probability distributions for all classes. The goal is to minimize the score, with a perfect cross-entropy value is 0.

For example, consider a problem with three classes, each having three examples in the data classified in class 1, class 2, and class 3, respectively. They are represented with one-hot encoding.

Let the true distribution for each example be:

```
#the first class is set to probability 1, all others are 0; this example
belongs to class #1
ex_1_true = [1, 0, 0]
#the second class is set to probability 1, all others are 0;this example
belongs to class #2
ex_2_true = [0, 1, 0]
#the third class is set to probability 1, all others are 0;this example
belongs to class #3
ex_3_true = [0, 0, 1]
```

Copy to Clipboard

Now imagine a predictive model that gave us the following predictions:

```
#the highest probability is given to class #1
ex_1_predicted = [0.7, 0.2, 0.1]
#the highest probability is given to class #2
ex_2_predicted = [0.1, 0.8, 0.1]
#the highest probability is given to class #3
ex_3_predicted = [0.2, 0.2, 0.6]
```

Copy to Clipboard

If we compare the true and predicted distributions above, they seem to be rather different numbers, but there is a good pattern here: each example's predicted distribution gives the highest probability to the label the example actually belongs to. This means the distributions are similar and the cross-entropy should be small. When we calculate cross-entropy for the example above, we get 0.364, which is rather good and close to 0.

Now, consider a bad predictive model that gives the highest probability to a wrong label every time:

```
#the highest probability given to class #3, true labels is class #1
ex_1_predicted_bad = [0.1, 0.1, 0.7]
#the highest probability given to class #1, true labels is class #2
ex_2_predicted_bad = [0.8, 0.1, 0.1]
#the highest probability given to class #1, true labels is class #3
ex_3_predicted_bad = [0.6, 0.2, 0.2]
```

Copy to Clipboard

When we calculate the cross-entropy for these examples, we get 2.036, which is rather bad.

If we take cross-entropy between two identical true distributions, we get perfect probabilities and cross-entropy equal to 0.

Run the code on the right to see this in practice. To calculate cross-entropy between two distributions we are using the `log_loss()` function in scikit-learn, which is equivalent to calculating cross-entropy.

**Instructions**

Checkpoint 1 Passed

**1.**

Use the `log_loss()` function to calculate cross-entropy between `true_labels` and `true_labels`.

The following code calculates cross-entropy between two distributions `distr1` and `distr2`:

```
from sklearn.metrics import log_loss
ll = log_loss(distr1, distr2)
```

Copy to Clipboard

Replace `distr1` with `true_labels` and `distr2` with `true_labels`. You should get cross-entropy equal to 0.

```
from sklearn.metrics import log_loss

#the first class is set to probability 1, all others are 0; this example belongs
to class #1
ex_1_true = [1, 0, 0]
#the second class is set to probability 1, all others are 0;this example belongs
to class #2
ex_2_true = [0, 1, 0]
#the third class is set to probability 1, all others are 0;this example belongs to
class #3
ex_3_true = [0, 0, 1]

#the highest probability is given to class #1
ex_1_predicted = [0.7, 0.2, 0.1]
#the highest probability is given to class #2
ex_2_predicted = [0.1, 0.8, 0.1]
#the highest probability is given to class #3
ex_3_predicted = [0.2, 0.2, 0.6]

#the highest probability given to class #3, true labels is class #1
ex_1_predicted_bad = [0.1, 0.1, 0.7]
#the highest probability given to class #1, true labels is class #2
ex_2_predicted_bad = [0.8, 0.1, 0.1]
#the highest probability given to class #1, true labels is class #3
ex_3_predicted_bad = [0.6, 0.2, 0.2]

true_labels = [ex_1_true, ex_2_true, ex_3_true]
predicted_labels = [ex_1_predicted, ex_2_predicted, ex_3_predicted]
predicted_labels_bad = [ex_1_predicted_bad, ex_2_predicted_bad,
ex_3_predicted_bad]

ll = log_loss(true_labels, predicted_labels)
print('Average Log Loss (good prediction): %.3f' % ll)

ll = log_loss(true_labels, predicted_labels_bad)
print('Average Log Loss (bad prediction): %.3f' % ll)

#your code here
ll = log_loss(true_labels, true_labels)
print('(TODO)Average Log Loss (true prediction): %.3f' % ll)

RESULTS:
Average Log Loss (good prediction): 0.364
Average Log Loss (bad prediction): 2.036
(TODO)Average Log Loss (true prediction): 0.000
```

**Classification**

# Loading and analyzing the data

Let's walk through an example of how to load, analyze, and split the data.
Assume we have a dataset, stored in the **train_glass.csv** (training data) and **test_glass.csv** (test data) files, about various products made of glass. Using the **train_glass.csv** file, we want to learn a model that can predict which glass item can be constructed given the proportion of various elements such as Aluminium (Al), Magnesium (Mg), and Iron (Fe). We then want to evaluate the model on the test data.
To load the training data into a pandas DataFrame, we do the following:

```
import pandas as pd
data_train = pd.read_csv("train_glass.csv")
```

Copy to Clipboard

The following command lists all features with accompanying types about the columns:

```
print(data_train.info())
```

Copy to Clipboard

The output looks something like this:

```
#   Column    Non-Null Count   Dtype
---  ------   --------------   -----
0   Al        300 non-null     float64
1   Mg        300 non-null     float64
3   Fe        300 non-null     float64
4   item      300 non-null     object
```

Copy to Clipboard

We see that `Al`, `Mg`, and `Fe` are numeric columns, and `item` is an object column containing strings. We would like to predict the `item` column.
The following commands show us which categories we have in the `item` column and what their distribution is:

```
from collections import Counter
print('Classes and number of values in the
dataset:',Counter(data_train["item"]))
```

Copy to Clipboard

which gives us the following output:

```
Classes and number of values in the dataset:
Counter({'lamps': 75, 'tableware': 125, 'containers': 100})
```

Copy to Clipboard

This tells us that we have three categories to predict: "lamps", "tableware", and "containers", and how many samples we have in our training data for each.

Lastly, we need to split the features in our data into the label (y-variable we are trying to predict) and the x-variables (predictor variables) by doing the following:

```
train_y = data_train["item"]
train_x = data_train[['Al', 'Mg', 'Fe']]
```

Copy to Clipboard

For this exercise (and the remainder of this lesson), you will be working with the `air_quality_train.csv` and `air_quality_test.csv` data sets. You will create a classification model that will predict the air quality dependent on the different element compounds found in the air (e.g. Carbon monoxide, Benzene). The label column or y-variable is `Air_Quality` and has the following scale from worst to best air quality, `['Severe', 'Very Poor', 'Poor', 'Moderate', 'Satisfactory', 'Good']`.

**Instructions**

Checkpoint 1 Passed

**1.**

Using `pd.read_csv()`, load and save the `air_quality_train.csv` data set to an object named `train_data`. Then load and save `air_quality_test.csv` as `test_data`.

In the previous example above, we used this code to load and save the data:

```
data_train = pd.read_csv("train_glass.csv")
```

Copy to Clipboard

Checkpoint 2 Passed

**2.**

Let's take a look at the column features of the training data set. Use `.info()` to print the non-null count (the observations that are not null) and the data type of each column in `train_data`.

In the previous example above, `print(data_train.info())` was used to view the column information.

Checkpoint 3 Passed

**3.**

Next, use `Counter()` to print the class distribution for `Air_Quality` from the training set.

Checkpoint 4 Passed

**4.**

Save all the predictor variables to the object `x_train`. The predictor variables are all the variables except `Air_Quality`.

Here is the list of the predictor variables, `'PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI'`.

Checkpoint 5 Passed

**5.**

Then save the label (y-variable) to the object `y_train`.

In the next exercise, we will split the testing data and continue with our classification model.

```
import pandas as pd
from collections import Counter

# 1- read in and save the training and testing data
train_data = pd.read_csv("air_quality_train.csv")
test_data = pd.read_csv("air_quality_test.csv")
```

```
# 2- print columns and their respective types of train_data
print(train_data.info())

# 3- print the class distribution
print(Counter(train_data["Air_Quality"]))

# 4- extract the predictor features from the training data
x_train = train_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2',
'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI']]

# 5- extract the label column (y-variable) from the training data
y_train = train_data["Air_Quality"]

#we will do the same for the test features in the next exercise
```

**Classification**

# Preparing the data

When using categorical cross-entropy — the loss function necessary for multiclass classification problems — in TensorFlow with Keras, one needs to convert all the categorical features and labels into one-hot encoding vectors. Previously, when we had features encoded as strings, we used the `pandas.get_dummies()` function. This works well for features, but it's not very usable for labels. The problem is that `get_dummies()` creates a separate column for each category, and you cannot predict for multiple columns.

A better approach is to convert the label vectors to integers ranging from 0 to the number of classes by using `sklearn.preprocessing.LabelEncoder`:

```
from sklearn.preprocessing import LabelEncoder
le=LabelEncoder()
train_y=le.fit_transform(train_y.astype(str))
test_y=le.transform(test_y.astype(str))
```

Copy to Clipboard

We first fit the transformer to the training data using the `LabelEncoder.fit_transform()` method

Preview: Docs Loading link description

, and then fit the trained transformer to the test data using the `LabelEncoder.transform()` method.

We can print the resulting mappings with:

```
integer_mapping = {l: i for i, l in enumerate(le.classes_)}
print(integer_mapping)
```

Copy to Clipboard

We get the following output:

```
{'lamps': 0, 'tableware': 1, 'containers': 2}
```

Copy to Clipboard

Each category is mapped to an integer, from 0 to 2 (because we have three categories).

Now that we have labels as integers, we can use a Keras function called `to_categorical()` to convert them into one-hot-encodings — the format we need for our cross-entropy loss:

```
train_y = tensorflow.keras.utils.to_categorical(train_y, dtype = 'int64')
test_y = tensorflow.keras.utils.to_categorical(test_y, dtype = 'int64')
```

Copy to Clipboard

**Instructions**

Checkpoint 1 Passed

**1.**

Use the `LabelEncoder.fit_transform()` method to encode the label vector `y_train` into integers and assign the result back to the `y_train` variable.

The following code uses the `LabelEncoder.fit_transform()` method to encode the label vector `my_vector` into integers and assign the result back to the `my_vector` variable:

```
my_vector = le.fit_transform(my_vector.astype(str))
```

Copy to Clipboard

Replace `my_vector` with `y_train`.

Checkpoint 2 Passed

**2.**

Use the `le.transform()` method to encode the label vector `y_test` into integers, where `le` is the instance of `LabelEncoder` trained in the previous step, and assign the result back to `y_test`.

The following code uses the `le.transform()` method to encode the label vector `my_vector` into integers and assign the result back to `my_vector`:

```
my_vector = le.transform(my_vector.astype(str))
```

Copy to Clipboard

Replace `my_vector` with `y_test`.

Checkpoint 3 Passed

**3.**

Using the `tensorflow.keras.utils.to_categorical()` function, convert the integer encoded label vector `y_train` into a one-hot encoding vector and assign the result back into the `y_train` variable.

The following code uses the `tensorflow.keras.utils.to_categorical()` function, convert the integer encoded label vector `my_vector` into a one-hot encoding vector and assign the result back into the `my_vector` variable:

```
my_vector = tensorflow.keras.utils.to_categorical(my_vector, dtype =
'int64')
```

Copy to Clipboard

Replace `my_vector` with `y_train`.

Checkpoint 4 Passed

**4.**

Using the `tensorflow.keras.utils.to_categorical()` function, convert the integer encoded label vector `y_test` into a one-hot encoding vector and assign the result back into the `y_test` variable.

The following code uses the `tensorflow.keras.utils.to_categorical()` function, convert the integer encoded label vector `my_vector` into a one-hot encoding vector and assign the result back into the `my_vector` variable:

```
my_vector = tensorflow.keras.utils.to_categorical(my_vector, dtype =
'int64')
```

Copy to Clipboard

Replace `my_vector` with `y_test`.

```
import pandas as pd
from collections import Counter
from sklearn.preprocessing import LabelEncoder
import tensorflow
#your code here

train_data = pd.read_csv("air_quality_train.csv")
test_data = pd.read_csv("air_quality_test.csv")

#print columns and their respective types
print(train_data.info())
#print the class distribution
print(Counter(train_data["Air_Quality"]))
#extract the features from the training data
x_train = train_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2',
'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the training data
y_train = train_data["Air_Quality"]
#extract the features from the test data
x_test = test_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3',
'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the test data
y_test = test_data["Air_Quality"]

#encode the labels into integers
le = LabelEncoder()
y_train = le.fit_transform(y_train.astype(str))
y_test = le.transform(y_test.astype(str))
print(y_train[:10) > [5 2 5 1 3 1 3 2 3 2]

#print the integer mappings
integer_mapping = {l: i for i, l in enumerate(le.classes_)}
print("The integer mapping:\n", integer_mapping)
 > The integer mapping:
 {'Good': 0, 'Moderate': 1, 'Poor': 2, 'Satisfactory': 3, 'Severe': 4, 'Very Poor': 5}

#convert the integer encoded labels into binary vectors
y_train = tensorflow.keras.utils.to_categorical(y_train, dtype = 'int64')
y_test = tensorflow.keras.utils.to_categorical(y_test, dtype = 'int64')

print(y_train[:10) >
 [[0 0 0 0 0 1]
 [0 0 1 0 0 0]
 [0 0 0 0 0 1]
 [0 1 0 0 0 0]
 [0 0 0 1 0 0]
 [0 1 0 0 0 0]
 [0 0 0 1 0 0]
 [0 0 1 0 0 0]
 [0 0 0 1 0 0]
 [0 0 1 0 0 0]]
```

## Classification

# Designing a deep learning model for classification

To initialize a Keras Sequential model in TensorFlow, we do the following:

```
from tensorflow.keras.models import Sequential
my_model = Sequential()
```

Copy to Clipboard

The process is the following:

set the input layer
set the hidden layers
set the output layer.

To add the input layer, we use `keras.layers.InputLayer` the following way:

```
from tensorflow.keras.layers import  InputLayer
my_model.add(InputLayer(input_shape=(data_train.shape[1],)))
```

Copy to Clipboard

For now, we will only add one hidden layer using `keras.layers.Dense`:

```
from tensorflow.keras.layers import  Dense
my_model.add(Dense(8, activation='relu'))
```

Copy to Clipboard

This layer has eight hidden units and uses a rectified linear unit (`relu`) as the activation function.

Finally, we need to set the output layer. For regression, we don't use any activation function in the final layer because we needed to predict a number without any transformations. However, for classification, the desired output is a vector of categorical probabilities.

To have this vector as an output, we need to use the `softmax` activation function that outputs a vector with elements having values between 0 and 1 and that sum to 1 (just as all the probabilities of all outcomes for a random variable must sum up to 1). In the case of a

binary

Preview: Docs Loading link description

classification problem, a `sigmoid` activation function can also be used in the output layer but paired with the `binary_crossentropy` loss.

Since we have 3 classes to predict in our glass production data, the final `softmax` layer must have 3 units:

```
my_model.add(Dense(3, activation='softmax')) #the output layer is a softmax
with 3 units
```

Copy to Clipboard

**Instructions**

Checkpoint 1 Passed

**1.**

To your model declaration `model` add an input layer using `tensorflow.keras.layers.InputLayer`.

The following code adds an input layer to a model called `my_model` as an instance of `tensorflow.keras.layers.InputLayer`:

```
my_model.add(InputLayer(input_shape=(data_train.shape[1],)))
```

Copy to Clipboard

Replace `my_model` with `model`, and `data_train` with `x_train`.

Checkpoint 2 Passed

**2.**

To your model instance `model` add a hidden layer using `tensorflow.keras.layers.Dense` with 10 neurons and `relu` activation function.

The following code adds a hidden layer as an instance of `tensorflow.keras.layers.Dense` with 10 neurons and a `relu` activation function to a model instance called `my_model`:

```
from tensorflow.keras.layers import  Dense
my_model.add(Dense(8, activation='relu'))
```

Copy to Clipboard

Replace `my_model` with `model` and 8 with 10.

Checkpoint 3 Passed

**3.**

To your model, add an output layer as an instance of `tensorflow.keras.layers.Dense` with `softmax` as the activation function, and the number of hidden units corresponding to the number of classes in the air quality data.

The following code adds an output layer as an instance of `tensorflow.keras.layers.Dense` with `softmax` as the activation function, and the number of hidden units equal to 3 classes:

```
from tensorflow.keras.layers import Dense
my_model.add(Dense(3, activation='softmax'))
```

Copy to Clipboard

Replace `my_model`, and 3 with 6. That is how many classes we have in the Air Quality data.

```
import pandas as pd
from collections import Counter
from sklearn.preprocessing import LabelEncoder
import tensorflow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import  InputLayer
from tensorflow.keras.layers import  Dense
#your code here

train_data = pd.read_csv("air_quality_train.csv")
test_data = pd.read_csv("air_quality_test.csv")

#print columns and their respective types
print(train_data.info())
#print the class distribution
print(Counter(train_data["Air_Quality"]))
#extract the features from the training data
x_train = train_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2',
'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the training data
y_train = train_data["Air_Quality"]
#extract the features from the test data
x_test = test_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3',
'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the test data
y_test = test_data["Air_Quality"]
```

```
#encode the labels into integers
le = LabelEncoder()
#convert the integer encoded labels into binary vectors
y_train=le.fit_transform(y_train.astype(str))
y_test=le.transform(y_test.astype(str))
#convert the integer encoded labels into binary vectors
y_train = tensorflow.keras.utils.to_categorical(y_train, dtype = 'int64')
y_test = tensorflow.keras.utils.to_categorical(y_test, dtype = 'int64')

#design the model
model = Sequential()
#add the input layer
model.add(InputLayer(input_shape=(x_train.shape[1],)))
#add a hidden layer
model.add(Dense(10, activation='relu'))
#add an output layer
model.add(Dense(6, activation='softmax'))
```

**Classification**

# Setting the optimizer

Now that we've had a brief introduction to cross-entropy, we'll see how to use it with our model.
First, to specify the use of cross-entropy when optimizing the model, we need to set the `loss` parameter
Preview: Docs Loading link description
to `categorical_crossentropy` of the `Model.compile()` method
Preview: Docs Loading link description
.
Second, we also need to decide which metrics to use to evaluate our model. For classification, we usually use accuracy. Accuracy calculates how often predictions equal labels and is expressed in percentages. We will use this metric for our problem.
Finally, we will use Adam as our optimizer because it's effective here and is commonly used.
To compile the model with all the specifications mentioned above we do the following:

```
my_model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

Copy to Clipboard
We are now ready to train our model.
**Instructions**

Checkpoint 1 Enabled

**1.**

Compile your model instance `model` using the `categorical_crossentropy` loss, `adam` optimizer, and `accuracy` as the metrics.

The following code compiles a model instance `my_model` using the `categorical_crossentropy` loss, `adam` optimizer, and `accuracy` as the metrics:

```
my_model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

Copy to Clipboard

Replace `my_model` with `model`.

```
import pandas as pd
from collections import Counter
from sklearn.preprocessing import LabelEncoder
import tensorflow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import  InputLayer
from tensorflow.keras.layers import  Dense
#your code here

train_data = pd.read_csv("air_quality_train.csv")
test_data = pd.read_csv("air_quality_test.csv")

#print columns and their respective types
print(train_data.info())
#print the class distribution
print(Counter(train_data["Air_Quality"]))
#extract the features from the training data
x_train = train_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2',
'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the training data
y_train = train_data["Air_Quality"]
#extract the features from the test data
x_test = test_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3',
'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the test data
y_test = test_data["Air_Quality"]

#encode the labels into integers
le = LabelEncoder()
#convert the integer encoded labels into binary vectors
y_train=le.fit_transform(y_train.astype(str))
y_test=le.transform(y_test.astype(str))
#convert the integer encoded labels into binary vectors
y_train = tensorflow.keras.utils.to_categorical(y_train, dtype = 'int64')
y_test = tensorflow.keras.utils.to_categorical(y_test, dtype = 'int64')

#design the model
model = Sequential()
#add the input layer
model.add(InputLayer(input_shape=(x_train.shape[1],)))
#add a hidden layer
model.add(Dense(10, activation='relu'))
#add an output layer
model.add(Dense(6, activation='softmax'))

#compile the model
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

**Classification**

# Train and evaluate the classification model

To train a model instance `my_model` on the training data `my_data` and training labels `my_labels` we do the following:

```
my_model.fit(my_data, my_labels, epochs=10, batch_size=1, verbose=1)
```

Copy to Clipboard

With the command above, we set the number of

epochs

Preview: Docs Loading link description

to 10 and the batch size to 1. To see the progress of the training we set verbose to true (1).

After the model is trained, we can evaluate it using the unseen test data `my_test` and test labels `test_labels`:

```
loss, acc = my_model.evaluate(my_test, test_labels, verbose=0)
```

Copy to Clipboard

We take two outputs out of the `.evaluate()` function:

- the value of the loss (`categorical_crossentropy`)
- accuracy (as set in the `metrics` parameter

  Preview: Docs Loading link description

  of `.compile()`).

**Instructions**

Checkpoint 1 Enabled

**1.**

Using the `Model.fit()` function, train your model instance `model` with the training data `x_train` and labels `y_train`, using 20 epochs, batch size of 4, and verbose set to 1.

Note: Running this in the LE will take almost a full minute!

The following code trains a model instance `my_model` with the training data `my_data` and labels `my_labels`, using 100 epochs, batch size of 6, and verbose set to 1:

```
my_model.fit(my_data, my_labels, epochs = 100, batch_size = 6, verbose
= 1)
```

Copy to Clipboard

Replace `my_model` with `model`, `my_data` with `x_train`, `my_labels` with `y_train`, `epochs = 100` with `epochs = 20`, and `batch_size = 6` with `batch_size = 4`.

```
import pandas as pd
from collections import Counter
from sklearn.preprocessing import LabelEncoder
import tensorflow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import  InputLayer
from tensorflow.keras.layers import  Dense
#your code here

train_data = pd.read_csv("air_quality_train.csv")
```

```
test_data = pd.read_csv("air_quality_test.csv")

#print columns and their respective types
print(train_data.info())
#print the class distribution
print(Counter(train_data["Air_Quality"]))
#extract the features from the training data
x_train = train_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2',
'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the training data
y_train = train_data["Air_Quality"]
#extract the features from the test data
x_test = test_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3',
'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the test data
y_test = test_data["Air_Quality"]

#encode the labels into integers
le = LabelEncoder()
#convert the integer encoded labels into binary vectors
y_train=le.fit_transform(y_train.astype(str))
y_test=le.transform(y_test.astype(str))
#convert the integer encoded labels into binary vectors
y_train = tensorflow.keras.utils.to_categorical(y_train, dtype = 'int64')
y_test = tensorflow.keras.utils.to_categorical(y_test, dtype = 'int64')

#design the model
model = Sequential()
#add the input layer
model.add(InputLayer(input_shape=(x_train.shape[1],)))
#add a hidden layer
model.add(Dense(10, activation='relu'))
#add an output layer
model.add(Dense(6, activation='softmax'))

#compile the model
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])

#train and evaluate the model
model.fit(x_train, y_train, epochs = 20, batch_size = 4, verbose = 1)
```

Classification

# Additional evaluation statistics

Sometimes having only accuracy reported is not enough or adequate. Accuracy is often used when data is balanced, meaning it contains an equal or almost equal number of samples from all the classes. However, oftentimes data comes imbalanced. For example in medicine, the rate of a disease is low. In these cases, reporting another metric such as F1-score is more adequate.

Frequently, we want to consider both false negatives and false positives when evaluating our model. Maybe we want to find a model that is a balance between precision and recall. For example, if we are building a relevancy classifier for Twitter that spots mentions about toys, it might be equally bad to miss a (relevant) toy mention as to misclassify a non-toy (irrelevant) mention. Luckily, an F1-score is a helpful way to evaluate our model that takes both precision and recall into account with the harmonic mean.

To observe the F1-score of a trained model instance `my_model`, amongst other metrics, we use `sklearn.metrics.classification_report`:

```
import numpy as np
from sklearn.metrics import classification_report
yhat_classes = np.argmax(my_model.predict(my_test), axis=1)
y_true = np.argmax(my_test_labels, axis=1)
print(classification_report(y_true, yhat_classes))
```

Copy to Clipboard

In the code above we do the following:

- predict classes for all test cases `my_test` using the `.predict()` method
  Preview: Docs Loading link description
  and assign the result to the `yhat_classes` variable.
- using `.argmax()` convert the one-hot-encoded labels `my_test_labels` into the index
  Preview: Docs Loading link description
  of the class the sample belongs to. The index corresponds to our class encoded as an integer.
- use the `.classification_report()` method to calculate all the metrics.

**Instructions**

Checkpoint 1 Passed

**1.**

Using the `Model.predict()` method, get the predictions for your test data `x_test` using the trained model instance `model`. Assign the result to a variable called `y_estimate`.

Note: Running this in the LE will take almost a full minute!

The following code uses the `Model.predict()` method to get the predictions for data instance `X` using the trained model instance `my_model`:

```
y_hat = my_model.predict(X)
```

Copy to Clipboard

Replace `y_hat` with `y_estimate`, `my_model` with `model`, and `X` with `x_test`.

Checkpoint 2 Passed

**2.**

Using `np.argmax()` convert the one-hot encoded labels `y_estimate` into the index of the class each sample in the test data belongs to with the `axis` parameter set to 1. Assign the result to `y_estimate`.

Note: Running this in the LE will take almost a full minute!

The following code uses `np.argmax()` convert the one-hot encoded labels `my_labels` into the index of the class each sample in the test data belongs to:

```
y_true = np.argmax(y_true, axis=1)
```

Copy to Clipboard

Replace `y_true` with `y_estimate`.

Checkpoint 3 Passed

**3.**

Using `np.argmax()` convert the one-hot encoded labels `y_test` into the index of the class each sample in the test data belongs to with the `axis` parameter set to 1. Assign the result to `y_true`.
Note: Running this in the LE will take almost a full minute!

The following code uses `np.argmax()` convert the one-hot encoded labels `my_labels` into the index of the class each sample in the test data belongs to:

```
Y = np.argmax(Y_bin, axis=1)
```

Copy to Clipboard

Replace `Y` with `y_true` and `Y_bin` with `y_test`.

Checkpoint 4 Passed

**4.**

Using `sklearn.metrics.classification_report`, print additional metrics, such as F1-score calculated between the true `y_true` and estimated test data labels `y_estimate`.
Note: Running this in the LE will take almost a full minute!

The following code uses `sklearn.metrics.classification_report` to print additional metrics, such as F1-score calculated between the true `y_true` and estimated test data labels `y_hat`:

```
print(classification_report(y_true, y_hat))
```

Copy to Clipboard

Replace `y_hat` with `y_estimate`.

```
import pandas as pd
from collections import Counter
from sklearn.preprocessing import LabelEncoder
import tensorflow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import  InputLayer
from tensorflow.keras.layers import  Dense
from sklearn.metrics import classification_report
import numpy as np
#your code here

train_data = pd.read_csv("air_quality_train.csv")
test_data = pd.read_csv("air_quality_test.csv")

#print columns and their respective types
print(train_data.info())
#print the class distribution
print(Counter(train_data["Air_Quality"]))
#extract the features from the training data
x_train = train_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2',
'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the training data
y_train = train_data["Air_Quality"]
#extract the features from the test data
x_test = test_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3',
'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the test data
y_test = test_data["Air_Quality"]

#encode the labels into integers
```

```
le = LabelEncoder()
#convert the integer encoded labels into binary vectors
y_train=le.fit_transform(y_train.astype(str))
y_test=le.transform(y_test.astype(str))
#convert the integer encoded labels into binary vectors
y_train = tensorflow.keras.utils.to_categorical(y_train, dtype = 'int64')
y_test = tensorflow.keras.utils.to_categorical(y_test, dtype = 'int64')

#design the model
model = Sequential()
#add the input layer
model.add(InputLayer(input_shape=(x_train.shape[1],)))
#add a hidden layer
model.add(Dense(10, activation='relu'))
#add an output layer
model.add(Dense(6, activation='softmax'))

#compile the model
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])

#train and evaluate the model
model.fit(x_train, y_train, epochs = 20, batch_size = 16, verbose = 1)

#get additional statistics
y_estimate = model.predict(x_test)
y_estimate = np.argmax(y_estimate, axis = 1)
y_true = np.argmax(y_test, axis = 1)
print(classification_report(y_true, y_estimate))
```

**Classification**

# Classification loss alternative: sparse crossentropy

As we saw before, categorical cross-entropy requires that we first integer-encode our categorical labels and then convert them to one-hot encodings using `to_categorical()`. There is another type of loss – sparse categorical cross-entropy – which is a computationally modified categorical cross-entropy loss that allows you to leave the integer labels as they are and skip the entire procedure of encoding.
Sparse categorical cross-entropy is mathematically identical to categorical cross-entropy but introduces some computational shortcuts that save time in memory as well as computation because it uses a single integer for a class, rather than a whole vector. This is especially useful when we have data with many classes to predict.
We can specify the use of the sparse categorical crossentropy in the `.compile()` method:

```
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

Note the following changes: we make sure that our labels are just integer encoded using the `LabelEncoder()` but not converted into one-hot-encodings using `.to_categorical()`. Hence, we [comment](#)
Preview: Docs Loading link description
out the code that uses `.to_categorical()`.

**Instructions**

Checkpoint 1 Passed

**1.**

Using the `#` symbol for comments, comment out the following lines of code (Line 36 and Line 37):

```
y_train = tensorflow.keras.utils.to_categorical(y_train, dtype = 'int64')
y_test = tensorflow.keras.utils.to_categorical(y_test, dtype = 'int64')
```

Copy to Clipboard

Checkpoint 2 Passed

**2.**

Modify the existing code on the right by changing the `model.compile()` method to use `sparse_categorical_crossentropy` as loss. Run the code.
Note: Running this in the LE will take almost a full minute!

The following code uses compiles a model instance `my_model` with `sparse_categorical_crossentropy` as loss:

```
my_model.compile(loss='sparse_categorical_crossentropy', optimizer='adam', metrics=['accuracy'])
```

Copy to Clipboard

Replace `my_model` with `model`.

```
import pandas as pd
from collections import Counter
from sklearn.preprocessing import LabelEncoder
import tensorflow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import  InputLayer
from tensorflow.keras.layers import  Dense
from sklearn.metrics import classification_report
import numpy as np
#your code here

train_data = pd.read_csv("air_quality_train.csv")
test_data = pd.read_csv("air_quality_test.csv")

#print columns and their respective types
print(train_data.info())
#print the class distribution
print(Counter(train_data["Air_Quality"]))
#extract the features from the training data
x_train = train_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the training data
y_train = train_data["Air_Quality"]
#extract the features from the test data
```

```
x_test = test_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO',
'SO2', 'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the test data
y_test = test_data["Air_Quality"]

#encode the labels into integers
le = LabelEncoder()
#convert the integer encoded labels into binary vectors
y_train=le.fit_transform(y_train.astype(str))
y_test=le.transform(y_test.astype(str))
#convert the integer encoded labels into binary vectors
#we comment it here because we need only integer labels for
#sparse cross-entropy
#y_train = tensorflow.keras.utils.to_categorical(y_train, dtype = 'int64')
#y_test = tensorflow.keras.utils.to_categorical(y_test, dtype = 'int64')

#design the model
model = Sequential()
#add the input layer
model.add(InputLayer(input_shape=(x_train.shape[1],)))
#add a hidden layer
model.add(Dense(10, activation='relu'))
#add an output layer
model.add(Dense(6, activation='softmax'))

#compile the model
model.compile(loss='sparse_categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])

#train and evaluate the model
model.fit(x_train, y_train, epochs = 20, batch_size = 16, verbose = 0)

#get additional statistics
y_estimate = model.predict(x_test, verbose=0)
y_estimate = np.argmax(y_estimate, axis=1)
print(classification_report(y_test, y_estimate))
```

**Classification**

# Tweak the model

Now that we have run our code several times, we might be wondering if the model can be further improved.
The first thing we can try is to increase the number of
epochs
Preview: Docs An epoch is one complete pass of the entire training dataset by the learning algorithm to update the model's weights.
. Having 20 epochs, as we previously had, is usually not enough. Try changing the number of epochs, for example, to 40 and see what happens. Increasing the number of epochs naturally makes the learning longer, but as you probably observed, the results are often much better.

Other hyperparameters you might consider changing are: the batch size number of hidden layers number of units per hidden layer the learning rate of the optimizer the optimizer and so on.

**Instructions**

Checkpoint 1 Enabled

**1.**

Change the number of epochs from 20 to 30. Rerun the code and observe the results.

Note: Running this in the LE will take some time!

The following code sets the number of epochs for training a model instance `my_model` to 100:

```
my_model.fit(x_train, y_train, epochs = 20, batch_size = 8, verbose = 0)
```

Copy to Clipboard

Replace `epochs = 20` with `epochs = 30`.

```
import pandas as pd
from collections import Counter
from sklearn.preprocessing import LabelEncoder
import tensorflow
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import  InputLayer
from tensorflow.keras.layers import  Dense
from sklearn.metrics import classification_report
import numpy as np

train_data = pd.read_csv("air_quality_train.csv")
test_data = pd.read_csv("air_quality_test.csv")

#print columns and their respective types
print(train_data.info())
#print the class distribution
print(Counter(train_data["Air_Quality"]))
#extract the features from the training data
x_train = train_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2',
'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the training data
y_train = train_data["Air_Quality"]
#extract the features from the test data
x_test = test_data[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2', 'O3',
'Benzene', 'Toluene', 'Xylene', 'AQI']]
#extract the label column from the test data
y_test = test_data["Air_Quality"]

#encode the labels into integers
le = LabelEncoder()
#convert the integer encoded labels into binary vectors
y_train=le.fit_transform(y_train.astype(str))
y_test=le.transform(y_test.astype(str))
#convert the integer encoded labels into binary vectors
y_train = tensorflow.keras.utils.to_categorical(y_train, dtype = 'int64')
y_test = tensorflow.keras.utils.to_categorical(y_test, dtype = 'int64')

#design the model
model = Sequential()
#add the input layer
model.add(InputLayer(input_shape=(x_train.shape[1],)))
```

```
#add a hidden layer
model.add(Dense(10, activation='relu'))
#add an output layer
model.add(Dense(6, activation='softmax'))

#compile the model
model.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])

#train and evaluate the model
model.fit(x_train, y_train, epochs = 30, batch_size = 16, verbose = 0)

#get additional statistics
y_estimate = model.predict(x_test)
y_estimate = np.argmax(y_estimate, axis = 1)
y_true = np.argmax(y_test, axis = 1)
print(classification_report(y_true, y_estimate))
```

### Classification

# Summary

Congrats! You just created your first classification model using tabular data. Moreover, you performed multi-class classification! In this lesson, you learned:

- The task of classification and what the main difference is between the binary
- Preview: Docs Loading link description
- and multi-class classification
- How to calculate cross-entropy in practice as well as how to interpret it and use it for classification.
- How to analyze your data using `pandas` functionalities, and see the distribution of the categories using `collections.Counter()`, which might be useful for seeing if the data is imbalanced.
- How to prepare the data for classification by encoding the labels using `sklearn.preprocessing.LabelEncoder()` and converting them to one-hot encoding format necessary for the loss function using `tensorflow.keras.utils.to_categorical()`.
- How to design a TensorFlow with Keras deep learning model to perform classification focusing on the final (output) layer that needs to have a softmax activation function.
- How to initialize the optimizer by using the `categorical_cross_entropy` loss and accuracy as the learning metrics.
- How to train and evaluate your model.
- How to use an alternative loss function `sparse_categorical_crossentropy` that allows you to keep your labels integer encoded and skip converting them into one-hot encoding.
- How to tweak the model to see if the performance can be improved.

Build Deep Learning Models with TensorFlow

# Classification

In this project, you will use a dataset from Kaggle to predict the survival of patients with heart failure from serum creatinine and ejection fraction, and other factors such as age, anemia, diabetes, and so on.

Cardiovascular diseases (CVDs) are the number 1 cause of death globally, taking an estimated 17.9 million lives each year, which accounts for 31% of all deaths worldwide. Heart failure is a common event caused by CVDs, and this dataset contains 12 features that can be used to predict mortality by heart failure.

Most cardiovascular diseases can be prevented by addressing behavioral risk factors such as tobacco use, unhealthy diet and obesity, physical inactivity, and harmful alcohol use using population-wide strategies.

People with cardiovascular disease or who are at high cardiovascular risk (due to the presence of one or more risk factors such as hypertension, diabetes, hyperlipidemia, or already established disease) need early detection and management wherein a machine learning model can be of great help.

## Tasks

0/26 complete

Mark the tasks as complete by checking them off

## Loading the data

**1.**

Using `pandas.read_csv()`, load the data from **heart_failure.csv** to a pandas DataFrame object. Assign the resulting DataFrame to a variable called `data`.

The following code loads a CSV formatted data stored in **my_file.csv** and assigns the resulting DataFrame to a variable called `my_csv`:

```
import pandas as pd
my_csv = pd.read_csv('my_file.csv')
```

Copy to Clipboard

Replace `my_csv` with `data`, and `my_file.csv` with `heart_failure.csv`.

**2.**

Use the `DataFrame.info()` method to print all the columns and their types of the DataFrame instance `data`.

To print all the columns and their types for a DataFrame instance `my_data`, we do the following:

```
print(my_data.info())
```

Copy to Clipboard

Replace `my_data` with `data`.

**3.**

Print the distribution of the `death_event` column in the `data` DataFrame class using `collections.Counter`. This is the column you will need to predict.

The following code prints the class distribution of a column `my_class` in a DataFrame called `data` using `collections.Counter`:

```
from collections import Counter
print('Classes and number of values in the
dataset',Counter(data['my_class']))
```

Copy to Clipboard

Replace `my_class` with `death_event`, which is the column in our data we want to predict.

**4.**

Extract the label column `death_event` from the `data` `DataFrame` and assign the result to a variable called `y`.

The following code extracts a column called `my_column` from a pandas `DataFrame` called `my_data` and assigns the result to a variable called `my_label`:

```
my_label = my_data["my_column"]
```

Copy to Clipboard

Replace `my_label` with `y`, `my_data`with `data`, and `my_column` with `death_event`.

**5.**

Extract the features columns `['age','anaemia','creatinine_phosphokinase','diabetes','ejection_fraction','high_blood_pressure','platelets','serum_creatinine','serum_sodium','sex','smoking','time']` from the DataFrame instance `data` and assign the result to a variable called `x`.

The following code extracts a list of features ["ft1", "ft2", "ft3"] from a DataFrame instance called `my_data` and assigns the result to a variable called `my_features`:

```
my_features = my_data[["ft1", "ft2", "ft3"]]
```

Copy to Clipboard

Replace `my_features` with `x`, `my_data` with `data`, and `["ft1", "ft2", "ft3"]` with `['age','anaemia','creatinine_phosphokinase','diabetes','ejection_fraction','high_blood_pressure','platelets','serum_creatinine','serum_sodium','sex','smoking','time']`.

# Data preprocessing

**6.**

Use the `pandas.get_dummies()` function to convert the categorical features in the DataFrame instance `x` to one-hot encoding vectors and assign the result back to variable `x`.

The following code converts categorical features in a DataFrame instance called `my_features` and assigns the result back to `my_features`:

```
my_features  = pd.get_dummies(my_features)
```

Copy to Clipboard

Replace `my_features` with `x`.

**7.**

Use the `sklearn.model_selection.train_test_split()` method to split the data into training features, test features, training labels, and test labels, respectively. To the `test_size` parameter assign the percentage of data you wish to put in the test data, and use any value for the `random_state` parameter. Store the results of the function to `X_train`, `X_test`, `Y_train`, `Y_test` variables, making sure you use this order.

The following code uses the `sklearn.model_selection.train_test_split()` method to split the features data `x` and label data `y` into training features, test features, training labels, and test labels:

```
X_train, X_test, Y_train, Y_test = train_test_split(my_features,
my_labels, test_size = 0.3, random_state = 0)
```

Copy to Clipboard

Replace `my_features` with `x` and `my_labels` with `y`.

**8.**

Initialize a ColumnTransformer object by using `StandardScaler` to scale the numeric features in the dataset: `['age','creatinine_phosphokinase','ejection_fraction','platelets','serum_creatinine','serum_sodium','time']`. Assign the resulting object to a variable called `ct`.

The following code initializes a ColumnTransformer instance `my_transformer` and uses `Normalizer()` to normalize numeric features called "ft1", "ft2", and "ft3":

```
from sklearn.preprocessing import Normalizer
my_transformer = ColumnTransformer([("numeric", Normalizer(), ["ft1",
"ft2", "ft3"])])
```

Copy to Clipboard

Replace `my_transformer` with `ct`, `Normalizer()` with `StandardScaler()`, and `["ft1", "ft2", "ft3"]` with `['age','creatinine phosphokinase','ejection fraction','platelets','serum creatinine','serum_sodium','time']`.

**9.**

Use the `ColumnTransformer.fit_transform()` function to train the scaler instance `ct` on the training data `X_train` and assign the result back to `X_train`.

The following code trains an instance of ColumnTransformer called `my_transformer` on the training data `my_train` and assigns the result back to `my_train`:

```
my_train = my_transformer.fit_transform(my_train)
```

Copy to Clipboard

Replace `my_train` with `X_train`, and `my_transformer` with `ct`.

**10.**

Use the `ColumnTransformer.transform()` to scale the test data instance `X_test` using the trained scaler `ct`, and assign the result back to `X_test`.

The following code scales the test data instance `my_test` using a trained scaler `my_transformer`, and assigns the result back to `my_test`:

```
my_test = my_transformer.transform(my_test)
```

Copy to Clipboard

Replace `my_test` with `X_test` and `my_transformer` with `ct`.

# Prepare labels for classification

**11.**

Initialize an instance of `LabelEncoder` and assign it to a variable called `le`.

The following code initializes an instance of `LabelEncoder` and assigns it to a variable called `enc`:

```
enc = LabelEncoder()
```

Copy to Clipboard

Replace `enc` with `le`.

**12.**

Using the `LabelEncoder.fit_transform()` function, fit the encoder instance `le` to the training labels `Y_train`, while at the same time converting the training labels according to the trained encoder.

The following code uses the `LabelEncoder.fit_transform()` function to fit the encoder instance `enc` to the `my_labels` labels:

```
my_labels = enc.fit_transform(my_labels.astype(str))
```

Copy to Clipboard

Replace `my_labels` with `Y_train` and `enc` with `le`.

**13.**

Using the `LabelEncoder.transform()` function, encode the test labels `Y_test` using the trained encoder `le`.

The following code uses the `LabelEncoder.transform()` function to encode the `my_labels` labels using the trained encoder `enc`:

```
my_labels = enc.transform(my_labels.astype(str))
```

Copy to Clipboard

Replace `my_labels` with `Y_test` and `enc` with `le`.

**14.**

Using the `tensorflow.keras.utils.to_categorical()` function, transform the encoded training labels `Y_train` into a binary vector and assign the result back to `Y_train`.

The following code uses the `tensorflow.keras.utils.to_categorical()` function to transform the encoded labels `my_labels` into a binary vector:

```
from tensorflow.keras.utils import to_categorical
my_labels = to_categorical(my_labels)
```

Copy to Clipboard

Replace `my_labels` with `Y_train`.

**15.**

Using the `tensorflow.keras.utils.to_categorical()` function, transform the encoded test labels `Y_test` into a binary vector and assign the result back to `Y_test`.

The following code uses the `tensorflow.keras.utils.to_categorical()` function to transform the encoded labels `my_labels` into a binary vector:

```
from tensorflow.keras.utils import to_categorical
my_labels = to_categorical(my_labels)
```

Copy to Clipboard

Replace `my_labels` with `Y_test`.

# Design the model

**16.**

Initialize a `tensorflow.keras.models.Sequential` model instance called `model`.

The following code initializes a `tensorflow.keras.models.Sequential` model instance called `md`:

```
from tensorflow.keras.models import Sequential
md = Sequential()
```

Copy to Clipboard

Replace `md` with `model`.

**17.**

Create an input layer instance of `tensorflow.keras.layers.InputLayer` and add it to the model instance `model` using the `Model.add()` function.

The following code creates an input layer instance of `tensorflow.keras.layers.InputLayer` and adds it to the model instance `md` using the `Model.add()` function:

```
md.add(InputLayer(input_shape=(my_data.shape[1],)))
```

Copy to Clipboard

Replace `md` with `model` and `my_data` with `X_train`.

**18.**

Create a hidden layer instance of `tensorflow.keras.layers.Dense` with `relu` activation function and 12 hidden neurons, and add it to the model instance `model`.

The following code creates a hidden layer instance of `tensorflow.keras.layers.Dense` with `relu` activation function and 64 hidden neurons, and adds it to the model instance `md`:

```
from tensorflow.keras.layers import Dense
md.add(Dense(64, activation='relu'))
```

Copy to Clipboard

Replace `md` with `model` and 64 with 12.

**19.**

Create an output layer instance of `tensorflow.keras.layers.Dense` with a `softmax` activation function (because of classification) with the number of neurons corresponding to the number of classes in the dataset.

The following code creates an output layer instance of `tensorflow.keras.layers.Dense` with a `softmax` activation function with the number of neurons corresponding to the number of classes in a dataset that has 6 possible labels:

```
md.add(Dense(6, activation='softmax'))
```

Copy to Clipboard

Replace `md` with `model` and 6 with 2 (your dataset has two classes).

**20.**

Using the `Model.compile()` function, compile the model instance `model` using the `categorical_crossentropy` loss, `adam` optimizer and `accuracy` as metrics.

The following code compiles the model instance `md` using the `Model.compile()` function, `categorical_crossentropy` loss, `adam` optimizer and `accuracy` as metrics.

```
md.compile(loss='categorical_crossentropy', optimizer='adam',
metrics=['accuracy'])
```

Copy to Clipboard

Replace `md` with `model`.

# Train and evaluate the model

**21.**

Using the `Model.fit()` function, fit the model instance `model` to the training data `X_train` and training labels `Y_train`. Set the number of epochs to 100 and the batch size parameter to 16.

The following code uses the `Model.fit()` function to fit the model instance `md` to the data `my_data` and labels `my_labels` with 50 epochs and the batch size parameter set to 2:

```
md.fit(my_data, my_labels, epochs = 50, batch_size = 2, verbose=1)
```

Copy to Clipboard

Replace `md` with `model`, `my_data` with `X_train` and `my_labels` with `Y_train`.

**22.**

Using the `Model.evaluate()` function, evaluate the trained model instance `model` on the test data `X_test` and test labels `Y_test`. Assign the result to a variable called `loss` (representing the final loss value) and a variable called `acc` (representing the accuracy metrics), respectively.

The following code uses the `Model.evaluate()` function to evaluate the trained model instance `md` on the test data `my_data` and test labels `my_labels`, and assigns the result to a variable called `loss` and a variable called `acc`, respectively:

```
loss, acc = md.evaluate(my_data, my_labels, verbose=0)
print("Loss", loss, "Accuracy:", acc)
```

Copy to Clipboard

Replace `md` with `model`, `my_data` with `X_test` and `my_labels` with `Y_test`.

# Generating a classification report

**23.**

Use the `Model.predict()` to get the predictions for the test data `X_test` with the trained model instance `model`. Assign the result to a variable called `y_estimate`.

The following code uses the `Model.predict_classes()` to get the predictions for the test data `my_data` with the trained model instance `md`, and assigns the result to a variable called `y_hat`:

```
y_hat = md.predict(my_data, verbose=0)
```

Copy to Clipboard

Replace `y_hat` with `y_estimate`, `md` with `model` and `my_data` with `X_test`.

**24.**

Use the `numpy.argmax()` method to select the indices of the true classes for each label encoding in `y_estimate`. Assign the result to a variable called `y_estimate`.

**25.**

Use the `numpy.argmax()` method to select the indices of the true classes for each label encoding in `Y_test`. Assign the result to a variable called `y_true`.

The following code uses the `numpy.argmax()` method to select the indices of the true classes for each label encoding in `my_data` and assign the result to a variable called `y_true`:

```
import numpy as np
y_true = np.argmax(my_data, axis=1)
```

Copy to Clipboard

Replace `my_data` with `Y_test`.

**26.**

Print additional metrics, such as F1-score, using the `sklearn.metrics.classification_report()` function by providing it with `y_true` and `y_estimate` vectors as input parameters.

The following code prints additional metrics, such as F1-score, using the `sklearn.metrics.classification_report()` function by providing it with `y_true` and `y_hat` vectors as input parameters:

```
print(classification_report(y_true, y_hat))
```

Copy to Clipboard

Replace `y_hat` with `y_estimate`.

```
import pandas as pd
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.model_selection import train_test_split
from collections import Counter
from sklearn.compose import ColumnTransformer
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, InputLayer
from sklearn.metrics import classification_report
from tensorflow.keras.utils import to_categorical
import numpy as np
```

# Image Classification

## Introduction to Image Classification

Neural networks are perfectly suited for *image classification*: the task of finding the complex patterns in pixels necessary to map an image to its label. As a result, image classification is a common application of deep learning.

In this lesson, we will learn how neural networks can be applied to image classification. To do this, we will be using *convolutional layers*: layers designed to process image data by focusing on local relationships between features.

Pneumonia, the infection of lung tissue, is responsible for over two and a half million deaths worldwide. A radiologist's diagnosis is often critical for the discovery and treatment of pneumonia. However, these doctors are often in short supply.

In this lesson, we will train a *Convolutional Neural Network (CNN)* to automatically classify chest X-rays, showcasing the potential for deep learning in medical domains.

**Instructions**

We will be using the Chest X-Ray (Pneumonia) dataset: approximately 6000 chest x-rays, each annotated as 'NORMAL' or 'PNEUMONIA.'

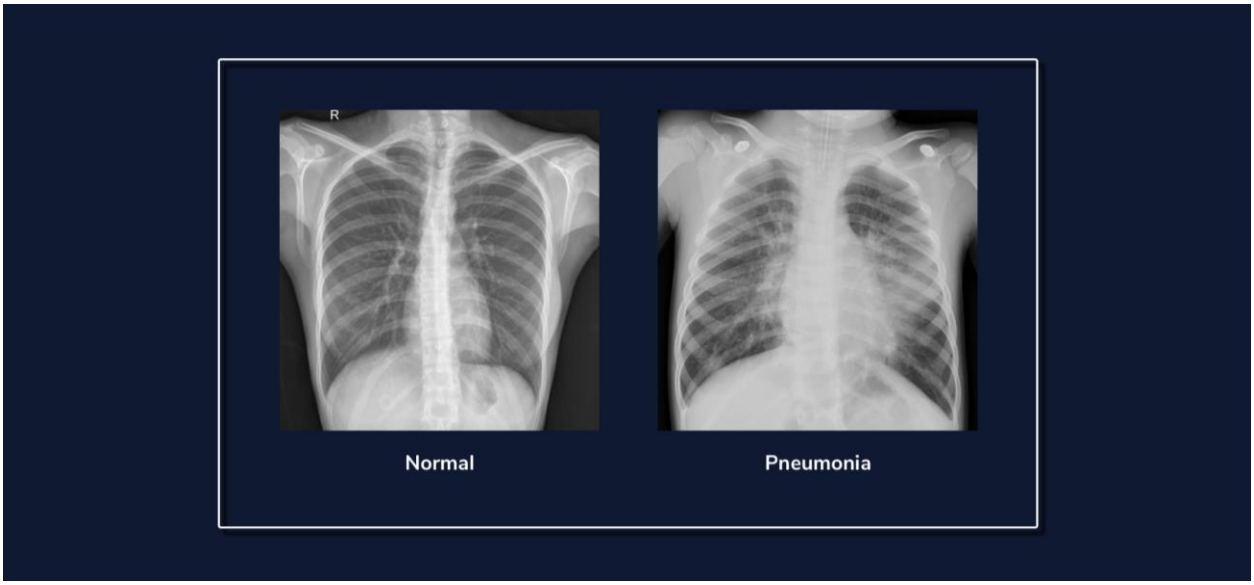
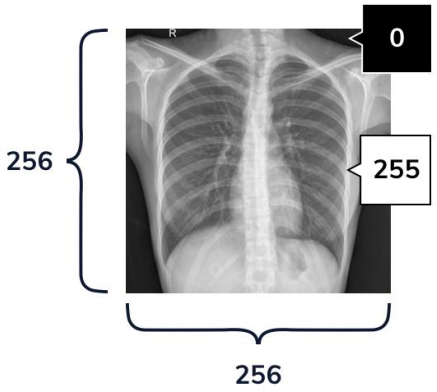


**Image Classification**

# Preprocessing Image Data

Our goal is to pass these X-ray images into our network, and to classify them according to their respective labels. At a high-level, this is very similar to how we would approach classifying non-image data.

But for our image data, our features are going to come from image pixels. Each image will be 256 pixels tall and 256 pixels wide, and each pixel has a value between 0 (black) - 255 (white).




While loading and preprocessing image data can be a bit tricky, Keras provides us with a few tools to make the process less burdensome. Of these, the `ImageDataGenerator` class is the most critical. We can use `ImageDataGenerator`s to load images from a file path, and to preprocess them. We can constuctor an `ImageDataGenerator` using the following code:

```
my_image_data_generator = ImageDataGenerator()
```

Copy to Clipboard

Beyond just loading our images, the `ImageDataGenerator` can also preprocess our data. We do this by passing additional arguments to the constructor.

There are a few ways to preprocess image data, but we will focus on the most important step: *pixel normalization*. Because neural networks struggle with large integer values, we want to rescale our raw pixel values between `0` and `1`. Our pixels have values in `[0,255]`, so we can normalize pixels by dividing each pixel by `255.0`.

We can also use our `ImageDataGenerator` for *data augmentation*: generating more data without collecting any new images. A common way to augment image data is to flip or randomly shift each image by small amounts. Because our dataset is only a few hundred images, we'll also use the `ImageDataGenerator` to randomly shift images during training.

For example, we can define another `ImageDataGenerator` and set its `vertical_flip`

parameter

Preview: Docs Loading link description

to be `True`.

```
my_augmented_image_data_generator = ImageDataGenerator( vertical_flip
= True )
```

Copy to Clipboard

If we use this `ImageDataGenerator` to load images, it will randomly flip some of those images upside down.

**Instructions**

Checkpoint 1 Passed

**1.**

Right now, we have defined `training_data_generator`, an `ImageDataGenerator` that can only load image data.
Let's modify `training_data_generator` so that it will also automatically perform pixel normalization. To do this, we need to set its `rescale` keyword argument equal to `1.0/255.`

Checkpoint 2 Passed

**2.**

Let's modify `training_data_generator`'s constructor so that we can use the `ImageDataGenerator` for data augmentation. We can do this by specifying that we want to randomly shift our training data each time we iterate through it.
Define the following parameters in the constructor:

Set `zoom_range` to be `0.2`. This will randomly increase or decrease the size of the image by up to 20%.

Set `rotation_range` to be `15`. This randomly rotates the image between [-15,15] degrees.

Set `width_shift_range` equal to `0.05`. This shift the image along its width by up to +/- 5%

Set `height_shift_range` to be `0.05`. This shifts the image along its height by up to +/- 5%

Try adding the above keyword arguments to `training_data_generator`'s constructor. Don't forget to include commas between each of these arguments!
When you are finished, try running the code. You should see the attributes that we just set (and a few other defaults) in the output terminal!

For example, if we wanted a zoom range of plus or minus 30%, we could set the `zoom_range` value like this:

```
zoom_range = 0.3
```

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator

#Creates an ImageDataGenerator:
training_data_generator = ImageDataGenerator(
       rescale=1.0/255,
        #Randomly increase or decrease the size of the image by up to 20%
        zoom_range=0.2,

        #Randomly rotate the image between -15,15 degrees
        rotation_range=15,
```

```
        #Shift the image along its width by up to +/- 5%
        width_shift_range=0.05,

        #Shift the image along its height by up to +/- 5%
        height_shift_range=0.05
)



#Prints its attributes:
print(training_data_generator.__dict__)
```

**Image Classification**

# Loading Image Data

Now, we can use the `ImageDataGenerator` object that we just created to load and batch our data, using its `.flow_from_directory()`

method

Preview: Docs Loading link description

.

Let's consider each of its arguments:

- `directory` : A string

  Preview: Docs Loading link description

  that defines the path to the folder containing our training data.

- `class_mode` : How we should represent the labels in our data. "For example, we can set this to `"categorical"` to return our labels as one-hot arrays, with a `1` in the correct class slot.

- `color_mode` : Specifies the type of image. For example, we set this to `"grayscale"` for black and white images, or to `"rgb"` (Red-Green-Blue) for color images.

- `target_size` : A tuple

  Preview: Docs Loading link description

  specifying the height and width of our image. Every image in the directory will be resized into this shape.

- `batch_size` : The batch size of our data.

The resulting `training_iterator` variable is a `DirectoryIterator` object. We can pass this object directly to `model.fit()` to train our model on our training data.

For example, the following code creates a `DirectoryIterator` that loads images from `"my_data_directory"` as 128 by 128 pixel color images in batches of 32:

```
training_data_generator.flow_from_directory(
```

```
"my_data_directory",
class_mode="categorical",
color_mode="rgb,
target_size=(128,128),
batch_size=32)
```

Copy to Clipboard

As the `training_data_generator` loads the data from the directory, it will automatically rescale the pixels by `1/255`, and apply the random transformations that we specified in the previous exercise.

**Instructions**

Checkpoint 1 Passed

**1.**

`data/train` is a folder that contains two subfolders:

**NORMAL** : Chest x-rays of patients without pneumonia.

**PNEUMONIA** : Chest x-rays of patients with pneumonia.

`flow_from_directory` will automatically label the images according to their subfolder.
Set `DIRECTORY` equal to `"data/train"`.

Checkpoint 2 Passed

**2.**

When we train our model, we will be using a categorical loss function, which will expect labels to be in a onehot format. For example, a picture in the PNEUMONIA folder should be labeled `[0,1]`, and picture in the NORMAL folder should be labeled `[1,0]`.
Set `CLASS_MODE` to be `"categorical"`to specify this behavior.

Checkpoint 3 Passed

**3.**

Lastly, we need to set `color_mode`. One might call our chest x-ray images “black and white,” but really these images are composed of pixels that can be any shade of gray, from white to black. The term for this type of image is grayscale. In contrast, color images are usually made of three stacked “images” or *channels*: one of Red pixels, another of Green pixels, and a third of Blue pixels.
Set `COLOR_MODE` to be `"grayscale"`. This tells the `DirectoryIterator` to load each image as gray pixels.
Once you are finished, uncomment line 11 to create the `training_iterator`.

Checkpoint 4 Passed

**4.**

Once we have used these parameters to create our `DirectoryIterator`, we can iterate over the training batches using the `.next()` method.
Copy these two lines of code over to the workspace to load a sample batch of data, and print its dimensions:

```
sample_batch_input,sample_batch_labels  =
training_iterator.next()


print(sample_batch_input.shape,sample_batch_labels.shape)
```

Copy to Clipboard

The sample_batch_input shape should be `( 32, 256, 256, 1)`. This is because there are 32 images in a batch, and each is a 256x256 pixel grayscale image. The last dimension is the number

of channels. Because these are grayscale images, there is only one channel representing light intensity.

Sample_batch_labels should be shape of `(32,2)`, because an image can be labeled Normal (`[1,0]`) or Pneumonia (`[0,1]`).

```
from preprocess import training_data_generator


DIRECTORY = "data/train"
CLASS_MODE = "categorical"
COLOR_MODE = "grayscale"
TARGET_SIZE = (256,256)
BATCH_SIZE = 32


#Creates a DirectoryIterator object using the above parameters:


training_iterator =
training_data_generator.flow_from_directory(DIRECTORY,class_mode=CLASS_MODE,color_
mode=COLOR_MODE,target_size=TARGET_SIZE,batch_size=BATCH_SIZE)
```

**Image Classification**

# Modifying our Feed-Forward Classification Model

We will now attempt to adapt a basic feed-forward classification model to classify images.

One way to classify image data is to treat an image as a vector of pixels. After all, we pass most data into our neural networks as feature vectors, so why not do the same here?

To do this, we need to:

1- Change the shape of our input layer model to accept our image data. Now, our input shape will be `(image height, image width, image channels)`. For example, if our data were composed of 512x512 pixel RGB images, we add an input shape as follows:

```
model.add(tf.keras.Input(shape=(512,512,3)))
```

Copy to Clipboard

2- Add a `Flatten()` layer to “flatten” our input image into a single vector. Kera’s `Flatten()` layer allows us to preserve the batch size of data, but combine the other dimensions of the image (height, width, image channels) into a single, lengthy feature vector. We can then pass this output to a `Dense()` layer.

**Instructions**

Checkpoint 1 Passed

**1.**

Use `model.add()` to add two layers to the model:

First, add an `tf.keras.Input()` layer. Remember to set its `shape` parameter for 256x256 grayscale images.

Second, add a `tf.keras.layers.Flatten()` layer after the input layer. This will automatically flatten our 256x256 pixel input images into 1-dimensional arrays of length 256x256=65536 features.

For example, for 128x128 pixel images, we would add the following to the beginning of our model:

```
model.add(tf.keras.Input(shape=(128,128,1)))
model.add(tf.keras.layers.Flatten())
```

**2.**

Let's also print out the model information using the `.summary` method.
Now, uncomment `model.summary()` below our model. Try running your code.
Note that even though our model only has three hidden layers, it already has a whopping 6.5 million trainable parameters. This means that we have over 1000x more parameters than data points! As a result this model will not only take a significant amount of compute to train, but it will also easily overfit to our training data.
The problem here is the size of our input. When we have flattened our image, we end up with an input of size 65536 features. We then need a matrix of size [65536 by 100] to transform this input into a layer with 100 units! This feed-forward model will struggle to learn meaningful combinations of so many features for those next hidden units.
Do not fret! Help is on the way. We will now introduce an alternative approach for extracting meaningful features from image data: *convolutional layers*. These layers allow us to scale down our input image into meaningful features while using only a fraction of the parameters required in a linear layer.

```
import tensorflow as tf

model = tf.keras.Sequential()

#Add an input layer that will expect grayscale input images of size
256x256:

model.add(tf.keras.Input(shape=(256,256,1)))

#Use a Flatten() layer to flatten the image into a single vector:

model.add(tf.keras.layers.Flatten())

model.add(tf.keras.layers.Dense(100,activation="relu"))
model.add(tf.keras.layers.Dense(50,activation="relu"))
model.add(tf.keras.layers.Dense(2,activation="softmax"))

#Print model information:
model.summary()
```

**Image Classification**

# A Better Alternative: Convolutional Neural Networks

*Convolutional Neural Networks (CNNs)* use layers specifically designed for image data. These layers capture local relationships between nearby features in an image.

Previously, in our feed-forward model, we multiplied our normalized pixels by a large weight matrix (of shape `(65536, 100)`) to generate our next set of features.

However, when we use a convolutional layer, we learn a set of smaller weight tensors, called *filters* (also known as *kernels*). We move each of these filters (i.e. *convolve* them) across the height and width of our input, to generate a new "image" of features. Each new "pixel" results from applying the filter to that location in the original image.

The interactive on the right demonstrates how we convolve a filter across a single image.

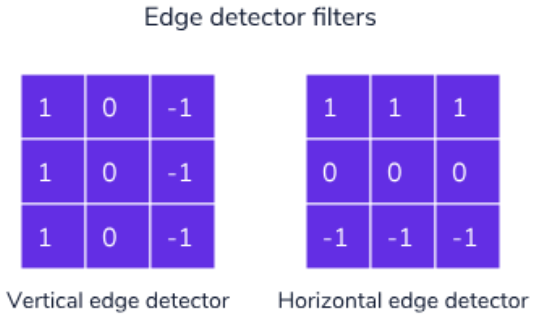
**Why do convolution-based approaches work well for image data?**

Convolution can reduce the size of an input image using only a few parameters.

Filters compute new features by only combining features that are near each other in the image. This operation encourages the model to look for local patterns (e.g., edges and objects).

Convolutional layers will produce similar outputs even when the objects in an image are translated (For example, if there were a giraffe in the bottom or top of the frame). This is because the same filters are applied across the entire image.

Before deep nets, researchers in computer vision would hand design these filters to capture specific information. For example, a 3x3 filter could be hard-coded to activate when convolved over pixels along a vertical or horizontal edge:




However, with deep nets, we can learn each weight in our filter (along with a bias term)! As in our feed-forward layers, these weights are learnable parameters. Typically, we randomly initialize our filters and use gradient descent to learn a better set of weights. By randomly initializing our filters, we ensure that different filters learn different types of information, like vertical versus horizontal edge detectors.

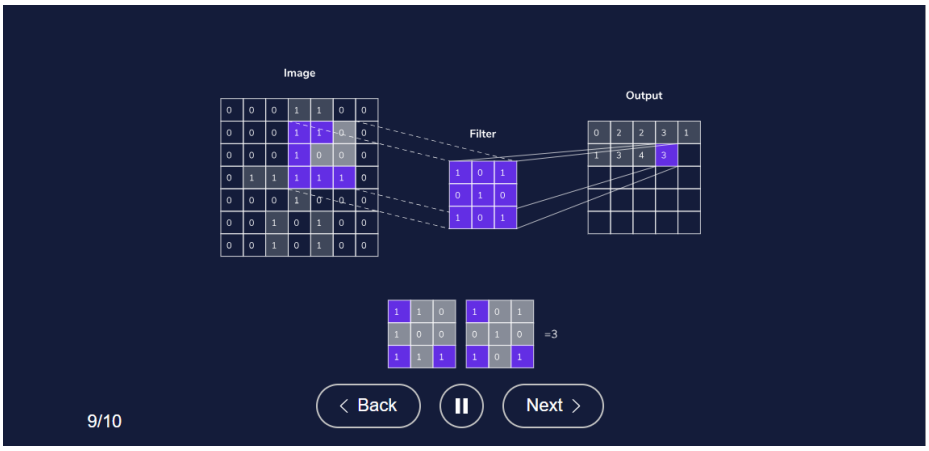



Image Classification

# Configuring a Convolutional Layer - Filters

In Keras, we can define a `Conv2D` layer to handle the forward and backward passes of convolution.

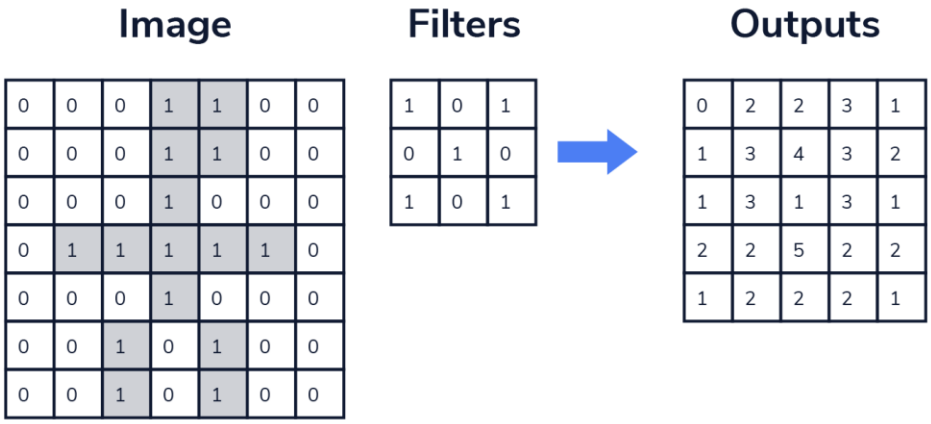
```
#Defines a convolutional layer with 4 filters, each of size 5 by 5:

tf.keras.layers.Conv2D(4, 5, activation="relu"))
```

Copy to Clipboard

When defining a convolutional layer, we can specify the number and size of the filters that we convolve across each image.

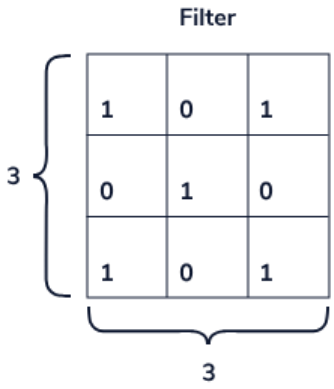
**Number of Filters**

When using convolutional layers, we don't just convolve one filter. Instead, we define some number of filters. We convolve each of these in turn to produce a new set of features. Then we stack these outputs (one for each filter) together in a new "image."



Our output tensor is then `(batch_size, new height, new width, number of filters)`. We call this last dimension number of *channels* ( or *feature maps* ). These are the result of applying a single filter across the entire image.

**Filter Size**




Beyond configuring the number of filters, we can also configure their size. Each filter has three dimensions: `[Height, Width, Input Channels]`

Height: the height of our filter (in pixels)

Width: the width of our filter (also in pixels)

Input Channels: The number of input channels. In a black and white image, there is 1 input channel (grayscale). However, in an RGB image, there are three input channels. Note that we don't have control over this dimension (it depends on the input), and Keras takes care of this last dimension for us.

Increasing height or width increases the number of pixels that a filter can pay attention to at each step in the convolution. However, doing so also increases the number of learnable parameters. People commonly use filters of size 5x5 and 3x3.

In total, the number of parameters in a convolution layer is:

$$\text{Number of filters} \times (\text{Input Channels} \times \text{Height} \times \text{Width} + 1)$$

$$\textit{Number of filters} \times (\textit{Input Channels} \times \textit{Height} \times \textit{Width} + 1)$$

Every filter has height, width, and thickness (The number of input channels), along with a bias term.

**Instructions**

Checkpoint 1 Passed

**1.**

Let's see how changing the number of filters changes the output of the layer.

Copy the following code into your workspace. The code creates a model with only one convolutional layer. This convolutional layer has 8 filters, and each is 3x3.

**Observe the dimensionality of the output, and the number of parameters.**

```
print("\n\nModel with 8 filters:")

model = tf.keras.Sequential()
model.add(tf.keras.Input(shape=(256, 256, 1)))

#Adds a Conv2D layer with 8 filters, each size 3x3:
model.add(tf.keras.layers.Conv2D(8, 3,activation="relu"))
model.summary()
```

Copy to Clipboard

Checkpoint 2 Passed

**2.**

Now modify change the number of filters to `16`.

How do the number of parameters and the output shape change?

Because we are using twice as many filters, this will double the number of parameters. Similarly, our output shape will now have 16 channels. Each channel results from convolving each filter over our image.

Checkpoint 3 Passed

**3.**

Now, modify the `Conv2D` layer to have filters of size 7x7. How does this change the number of parameters? How does this change the height and width of the image?

Before, each of our 16 filters was 3x3. Now, they are 7x7. This means that for every larger filter we have 7x7-3x3 = 49-9 = `40` more parameters. So there should be 40x16 = `640` more parameters in this layer. Previously, our model had `160` parameters, and now it has `800`.

However, when we convolve a 7x7 image, it's impossible to apply the kernel to any pixel within 6 pixels of our image edge. This is because a 7x7 filter will "hang" off of the image. As a result, the new height and width of output will now be 250x250.

```
import tensorflow as tf

print("\n\nModel with 8 filters:")

model = tf.keras.Sequential()
model.add(tf.keras.Input(shape=(256, 256, 1)))

#Adds a Conv2D layer with 8 filters, each size 3x3:
model.add(tf.keras.layers.Conv2D(16, 7,activation="relu"))
model.summary()
```

Image Classification

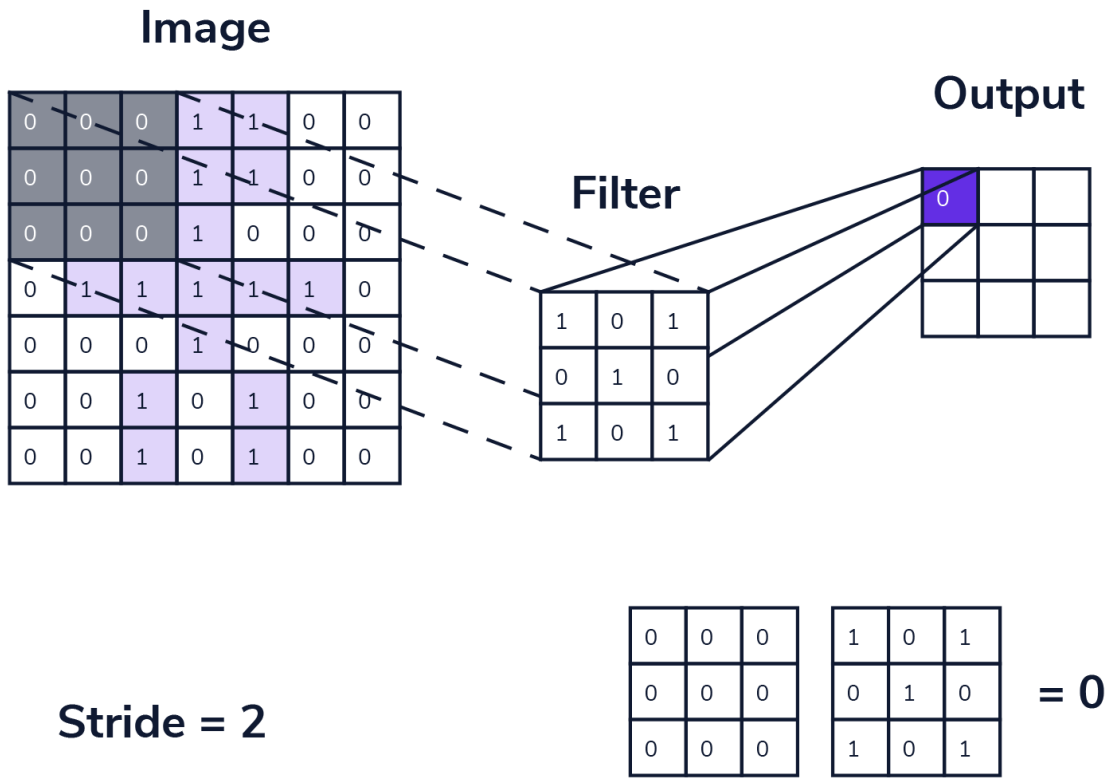
# Configuring a Convolutional Layer - Stride and Padding

Two other hyperparameters in a convolutional layer are *Stride* and *Padding*.

**Stride**

The stride hyperparameter is how much we move the filter each time we apply it. The default stride is 1, meaning that we move the filter across the image 1-pixel at a time. When we reach the end of a row in the image, we then go to the next one.

If we use a stride greater than 1, we do not apply our filter centered on every pixel. Instead, we move the filter multiple pixels at a time.




For example, if `strides = 2`, we move the filter two columns over at a time, and then skip every other row.
We can set the stride to any integer. For example, we can define a `Conv2D` layer with a stride of `3`:

```
#Adds a Conv2D layer with 8 filters, each size 5x5, and uses a stride of 3:
model.add(tf.keras.layers.Conv2D(8, 5,
strides=3,
activation="relu"))
```

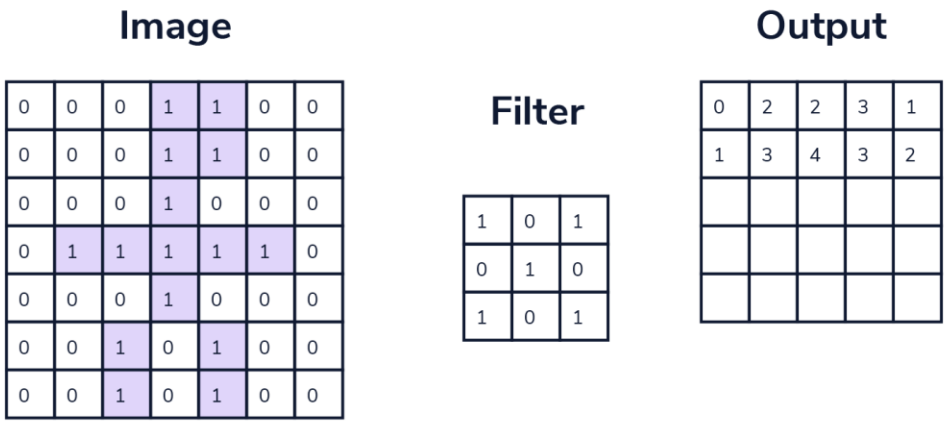
Copy to Clipboard

Larger strides allow us to decrease the size of our output. In the case where our `stride=2`, we apply our filter to every other pixel. As a result, we will halve the height and width of our output.

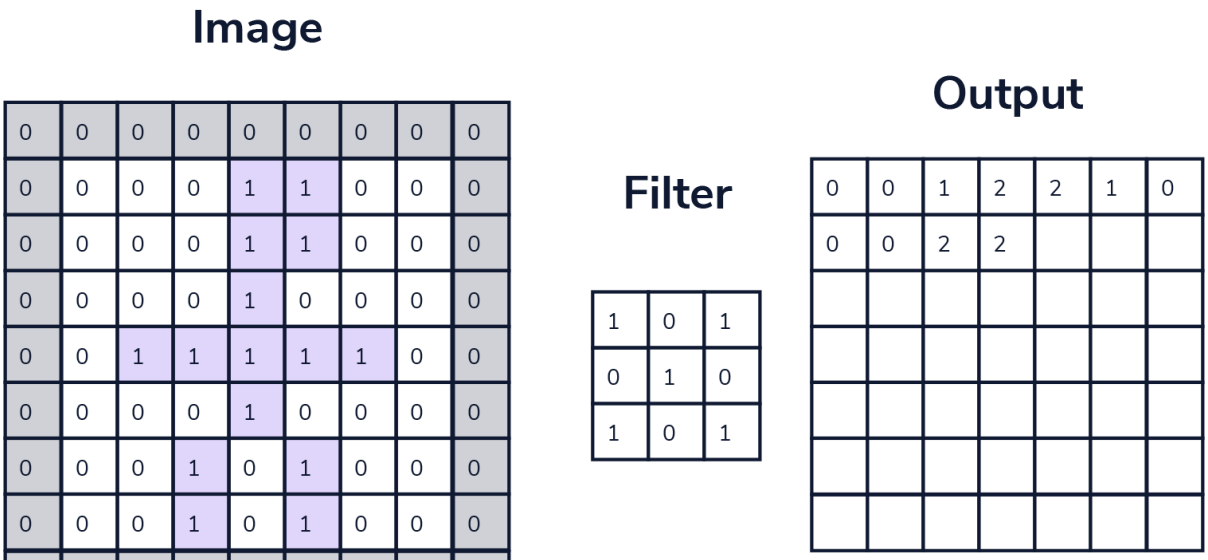
**Padding**

The padding hyperparameter defines what we do once our filter gets to the end of a row/column. In other words: "what happens when we run out of image?" There are two main methods for what to do here:

We just stop (*valid padding*): The default option is to just stop when our kernel moves off the image. Let's say we are convolving a `3x3` filter across a `7x7` image with `stride=1`. Our output will then be a `5x5` image, because we can't process the 6th pixel without our filter hanging off the image.




We keep going (*same padding*): Another option is to pad our input by surrounding our input with zeros. In this case, if we add zeros around our `7x7` image, then we can apply the `3x3` filter to every single pixel. This approach is called “same” padding, because if `stride=1`, the output of the layer will be the same height and width as the input.




We can use “same” padding by setting the `padding` parameter:

```
#Adds a Conv2D layer with 8 filters, each size 5x5, and uses a stride of 3:
model.add(tf.keras.layers.Conv2D(8, 5,
strides=3,
padding='same',
activation="relu"))
```

Copy to Clipboard

**Instructions**

Checkpoint 1 Passed

**1.**

Change the number of strides to `2`. How does this change the dimensions of the output?

Doubling the stride halves the height and width of the output. It goes from 250x250 to 125x125. If we used “same” instead of “valid” padding, the resulting output would have a height and width of 128x128.

Checkpoint 2 Passed

**2.**

Change the padding type to be `"same"`, and set `strides` equal to `1`.

What is the shape the resulting output?

The shape of the output will be 256x256, the same shape as the input image!

Specifying “same” padding told this layer to add 3 zeros of padding around the image. As a result, we can apply the 7x7 filter to every pixel, even those on the edge of the image.

```
import tensorflow as tf

print("Model with 16 filters:")

model = tf.keras.Sequential()
model.add(tf.keras.Input(shape=(256, 256, 1)))
```

```
#Adds a Conv2D layer with 16 filters, each size 7x7, and uses a stride of 1 with
valid padding:
model.add(tf.keras.layers.Conv2D(16, 7,
strides=1,
padding="same",
activation="relu"))
model.summary()
```

```
Model with 16 filters:
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 250, 250, 16)      800
=================================================================
Total params: 800
Trainable params: 800
Non-trainable params: 0
_________________________________________________________________
```

Image Classification

# Adding Convolutional Layers to Your Model

**Adding One Convolutional Layer**

Now, we can modify our feed-forward image classification code to use a convolutional layer:

- First, we are going to replace the first two `Dense` layers with a `Conv2D` layer.
- Then, we want to move the `Flatten` layer between the convolutional and last dense layer. Because dense layers apply their matrix to the dimension, we will always need to flatten the output of convolutional layers before passing them into a dense layer.

**Stacking Convolutional Layers**

However, we won't stop there! The beauty of neural networks is that we can stack many layers to learn richer combinations of features. We can stack convolutional layers the same way we stacked dense layers.

For example, we can stack three convolutional layers with distinct filter shapes and strides:

```
# 8 5x5 filters, with strides of 3
model.add(tf.keras.layers.Conv2D(8, 5, strides=3, activation="relu"))

# 4 3x3 filters, with strides of 3
model.add(tf.keras.layers.Conv2D(4, 3, strides=3, activation="relu"))

# 2 2x2 filters, with strides of 2
model.add(tf.keras.layers.Conv2D(2, 2, strides=2, activation="relu"))
```

Copy to Clipboard

Like with dense layers, the output of one convolutional layer can be passed as input to another. You can think of the output as a new input "image," with a height, width, and number of channels. The number of filters used in the previous layer becomes the number of channels that we input into the next!

As with dense layers, we should use non-linear activation functions between these convolutional layers.

**Instructions**

Checkpoint 1 Passed

**1.**

First, add a `Conv2D` layer after the `Input` layer.

Use `2` filters of size `5`
A stride of `3`
`"valid"` padding
A `relu` activation

Then, delete the first two `Dense`layers.

To add a convolutional layer with the necessary parameters, use the following line of code:

```
model.add(tf.keras.layers.Conv2D(2, 5, strides=3, activation="relu"))
```

Copy to Clipboard

Checkpoint 2 Passed

**2.**

Now, let's try stacking another `Conv2D` layer.

After you first `Conv2D` layer, add another with:

4 filters of size 3
A stride of `1`
`"valid"` padding
A `relu` activation

Similarly, to add a second convolutional layer with the necessary parameters, use could the following line of code:

```
model.add(tf.keras.layers.Conv2D(4, 3, strides=1, activation="relu"))
```

Copy to Clipboard

Checkpoint 3 Passed

**3.**

Click run and take a look at the shape of the model. What are the dimensions of each layer?

The output of `model.summary()` should look like:

```
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 84, 84, 2)         52
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 82, 82, 4)         76
_________________________________________________________________
flatten (Flatten)            (None, 26896)             0
_________________________________________________________________
dense (Dense)                (None, 2)                 53794
=================================================================
Total params: 53,922
Trainable params: 53,922
Non-trainable params: 0
```

```
import tensorflow as tf

model = tf.keras.Sequential()


model.add(tf.keras.Input(shape=(256,256,1)))

#Add a Conv2D layer
# - with 2 filters of size 5x5
# - strides of 3
# - valid padding

model.add(tf.keras.layers.Conv2D(2, 5, strides=3, activation="relu"))

model.add(tf.keras.layers.Conv2D(4, 3, strides=1, activation="relu"))
```

```
model.add(tf.keras.layers.Flatten())

#Remove these two dense layers:
# model.add(tf.keras.layers.Dense(100,activation="relu"))
# model.add(tf.keras.layers.Dense(50,activation="relu"))

model.add(tf.keras.layers.Dense(2,activation="softmax"))

#Print model information:
model.summary()
```

Run
Loading Complete
View solution

## Output-only Terminal

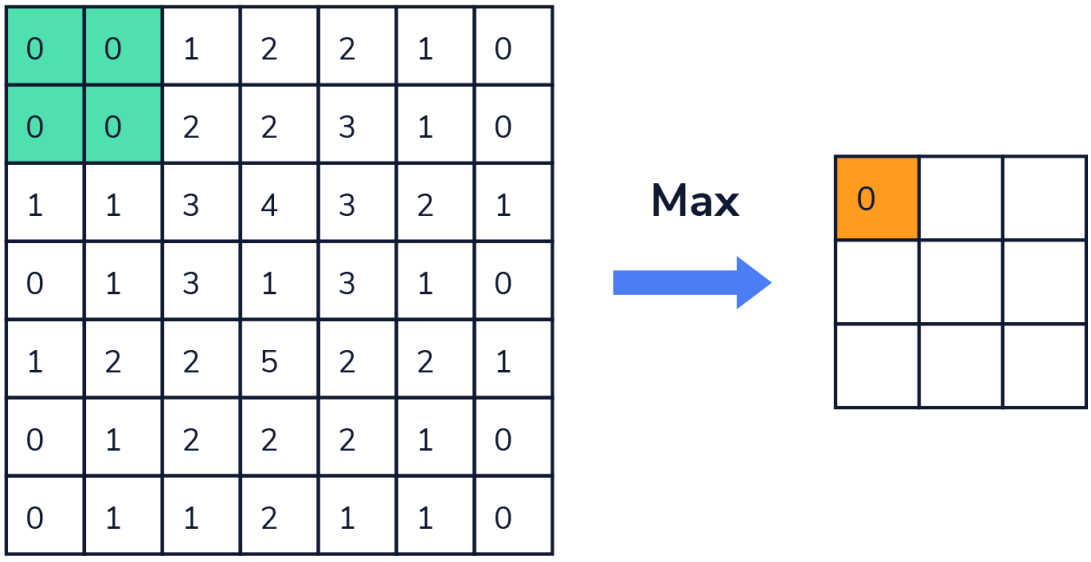
```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
flatten (Flatten)            (None, 65536)             0
_________________________________________________________________
dense (Dense)                (None, 100)               6553700
_________________________________________________________________
dense_1 (Dense)              (None, 50)                5050
_________________________________________________________________
dense_2 (Dense)              (None, 2)                 102
=================================================================
Total params: 6,558,852
Trainable params: 6,558,852
Non-trainable params: 0
_________________________________________________________________
```

### Image Classification

# Pooling

Another part of Convolutional Networks is *Pooling Layers*: layers that pool local information to reduce the dimensionality of intermediate convolutional outputs.
There are many different types of pooling layer, but the most common is called *Max pooling*:

- Like in convolution, we move windows of specified size across our input. We can specify the stride and padding in a max pooling layer.
- However, instead of multiplying each image patch by a filter, we replace the patch with its maximum value.




For example, we can define a max pooling layer that will move a `3x3` window across the input, with a stride of `3` and `valid` padding:

```
max_pool_2d = tf.keras.layers.MaxPooling2D(pool_size=(3, 3),    strides=(3, 3), padding='valid')
```

Copy to Clipboard

Beyond helping reduce the size of hidden layers (and reducing overfitting), max pooling layers have another useful property: they provide some amount of *translational invariance*. In other words, even if we move around objects in the input image, the output will be the same. This is very useful for classification. For example, we usually want to classify an image of a cat as a cat, regardless of how the cat is oriented in the image.

**Instructions**

Checkpoint 1 Passed

**1.**

Let's add max pooling layers and train the model. Try adding two max pooling layers.

First, define a `MaxPooling2D` layer with:

`pool_size` equal to `(5,5)`

`strides` equal to `(5,5)`

Place this layer after the first convolutional layer.

Second, define a `MaxPooling2D` layer with:

`pool_size` equal to `(2,2)`

`strides` equal to `(2,2)`

Place this layer immediately after the second convolutional layer.

Checkpoint 2 Passed

**2.**

Look at the output of `model.summary()`. How does the number of parameters in the model change? What is the output shape of the max pooling layers?

The number of parameters should drop to `522`! We can see that the first max-pooling layer shrinks height and width of its input shape by a factor of `4`. The second max-pooling layer shrinks its input shape by a factor of `2`.

```
import tensorflow as tf

model = tf.keras.Sequential()
```

```
model.add(tf.keras.Input(shape=(256,256,1)))

model.add(tf.keras.layers.Conv2D(2,5,strides=3,padding="valid",activation="relu"))

#Add first max pooling layer here.

#model.add(...)
model.add(tf.keras.layers.MaxPooling2D(pool_size=(5, 5), strides=(5,5)))

model.add(tf.keras.layers.Conv2D(4,3,strides=1,padding="valid",activation="relu"))

#Add the second max pooling layer here.

#model.add(...)
model.add(tf.keras.layers.MaxPooling2D( pool_size=(2,2), strides=(2,2)))

model.add(tf.keras.layers.Flatten())

model.add(tf.keras.layers.Dense(2,activation="softmax"))

#Print model information:
model.summary()
```

**Output-only Terminal**

```
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 84, 84, 2)         52
_________________________________________________________________
conv2d_1 (Conv2D)            (None, 82, 82, 4)         76
_________________________________________________________________
flatten (Flatten)            (None, 26896)             0
_________________________________________________________________
dense (Dense)                (None, 2)                 53794
=================================================================
Total params: 53,922
Trainable params: 53,922
Non-trainable params: 0
_________________________________________________________________
```

Image Classification

# Training the Model

Now, we are going to put everything together and train our model!
We have to do three additional things:

- Define another `ImageDataGenerator` and use it to load our validation data.
- Compile our model with an optimizer, metric, and a loss function.
- Train our model using `model.fit()`.

**Validation Data Generator**

We have already defined an `ImageDataGenerator` called `training_data_generator`. Like in the second exercise, we use `training_data_generator.flow_from_directory()` to preprocess and augment our training data.
Now, we need another `ImageDataGenerator` to load our validation data, which consists of 100 Normal X-rays, and 100 with Pneumonia. Like with our training data, we are going to need to normalize our pixels. However, unlike for our training data, we will not augment the validation data with random shifts.

**Loss, Optimizer, and Metrics**

Because our labels are onehot (`[1,0]` and `[0,1]`), we will use `keras.losses.CategoricalCrossentropy`. We will optimize this loss using the `Adam` optimizer.
Because our dateset is balanced, accuracy is a meaningful metric. We will also include *AUC* (area under the ROC curve). An ROC curve gives us the relationship between our *true positive* rate and our *false positive* rate. A true positive would be correctly identifying a patient with Pneumonia, while a false positive would be incorrectly identifying a healthy person as having pneumonia. Like with accuracy, we want our AUC to be as close to `1.0` as possible.

**Training the Model**

To train our model, we have to call `model.fit()` on our training data `DirectoryIterator` and validation data `DirectoryIterator`.
To reap the benefits of data augmentation, we will iterate over our training data five times (five epochs).

**Instructions**

Checkpoint 1 Passed

**1.**

First, define a new `ImageDataGenerator` called `validation_data_generator`.
Use its `.flow_from_directory()` method to load the validation grayscale images from the **'data/test'** directory.
**NOTE**: In this exercise, we train and test the model using hundreds of image files. As a result, the execution time will be longer than what is typical — please allow up to 5 minutes for the program to run.

We can create the `validation_data_generator` by instructing an `ImageDataGenerator`:

```
validation_data_generator = ImageDataGenerator()
```

Copy to Clipboard

Additionally, we can pass `rescale = 1.0/255` into the constructor to automatically normalize pixels to values between `0` and `1`.
Next, we can define our validation `DirectoryIterator` using the `flow_from_directory()` method:

```
validation_iterator =
validation_data_generator.flow_from_directory('insert_test_directory_here',class_mode='categorical',color_mode='grayscale',batch_size=BATCH_SIZE)
```

Copy to Clipboard

Checkpoint 2 Passed

**2.**

Now, let's compile our model with an optimizer and loss function.

Set the optimizer to:

```
tf.keras.optimizers.Adam(learning_rate=0.005)
```

Copy to Clipboard

Because our labels are onehot categorical labels, we should use:

```
tf.keras.losses.CategoricalCrossentropy()
```

Copy to Clipboard

as our loss function

Uncomment lines 52-65. Then, set:

`optimizer` to be `tf.keras.optimizers.Adam(learning_rate=0.005)`
`loss` to be `tf.keras.losses.CategoricalCrossentropy()`

Checkpoint 3 Passed

**3.**

We can train the model on our training data by passing in `training_iterator`. We can also evaluate our model by passing in our `validation_iterator` as validation data.
For both training and validation, we need to define the number of steps: the number of images in the dataset, divided by the batch size.
We can get the number of images in the training set using `training_iterator.samples`. Set `steps_per_epoch` to this number divided by `BATCH_SIZE`.

```
steps_per_epoch=training_iterator.samples/BATCH_SIZE
```

Copy to Clipboard

Similarly, we can compute `validation_steps` using `validation_iterator.samples`, and dividing by our validation batch size.
Lastly, set `epochs` equal to `5` to train on the training data for five epochs. Note that training may take up to a minute!

We pass the following into `model.fit()`:

`training_iterator`
`steps_per_epoch` equal to `training_iterator.samples/BATCH_SIZE`
`epochs` equal to `5`
`validation_data` equal to `validation_iterator`
`validation_steps` equal to `validation_iterator.samples/BATCH_SIZE`

Now, we can run the code to train our model.
After five epochs, our model should be able to achieve an accuracy > 0.80, and an AUC > 0.85! Congrats on training a convolutional neural network!
Once we've trained the model, feel free to try out different hyperparameters. Are their architectures that work even better?

```
from tensorflow.keras.preprocessing.image import ImageDataGenerator
import tensorflow as tf

BATCH_SIZE = 16

print("\nLoading training data...")

training_data_generator = ImageDataGenerator(
        rescale=1./255,
        zoom_range=0.2,
        rotation_range=15,
        width_shift_range=0.05,
        height_shift_range=0.05)

training_iterator =
training_data_generator.flow_from_directory('data/train',class_mode='categorical',
color_mode='grayscale',batch_size=BATCH_SIZE)


print("\nLoading validation data...")

#1) Create validation_data_generator, an ImageDataGenerator that just performs
pixel normalization:

validation_data_generator = ImageDataGenerator(rescale=1.0/255)

#2) Use validation_data_generator.flow_from_directory(...) to load the validation
data from the 'data/test' folder:

validation_iterator = validation_data_generator.flow_from_directory('data/test',
class_mode='categorical', color_mode='grayscale', batch_size=BATCH_SIZE)
```

```
print("\nBuilding model...")

#Rebuilds our model from the previous exercise, with convolutional and max pooling
layers:

model = tf.keras.Sequential()
model.add(tf.keras.Input(shape=(256, 256, 1)))
model.add(tf.keras.layers.Conv2D(2, 5, strides=3, activation="relu"))
model.add(tf.keras.layers.MaxPooling2D(
    pool_size=(5, 5), strides=(5,5)))
model.add(tf.keras.layers.Conv2D(4, 3, strides=1, activation="relu"))
model.add(tf.keras.layers.MaxPooling2D(
    pool_size=(2,2), strides=(2,2)))
model.add(tf.keras.layers.Flatten())

model.add(tf.keras.layers.Dense(2,activation="softmax"))

model.summary()


print("\nCompiling model...")

#3) Compile the model with an Adam optimizer, Categorical Cross Entropy Loss, and
Accuracy and AUC metrics:

model.compile(
   optimizer=tf.keras.optimizers.Adam(learning_rate=0.005),
   loss=tf.keras.losses.CategoricalCrossentropy(),
   metrics=[tf.keras.metrics.CategoricalAccuracy(),tf.keras.metrics.AUC()]
)

print("\nTraining model...")

#4) Use model.fit(...) to train and validate our model for 5 epochs:

model.fit(
       training_iterator,
       steps_per_epoch=training_iterator.samples/BATCH_SIZE,
       epochs=5,
       validation_data=validation_iterator,
       validation_steps=validation_iterator.samples/BATCH_SIZE
)
```

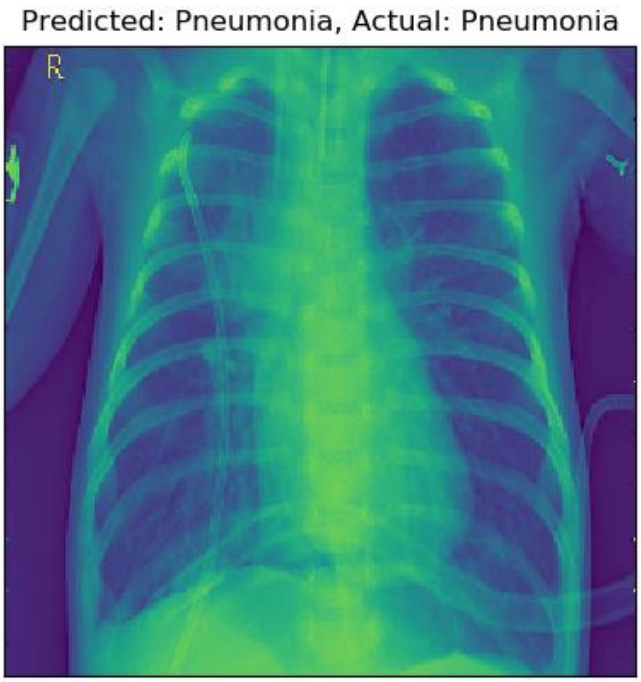
```
Loading training data...
Found 1000 images belonging to 2 classes.

Loading validation data...

Building model...
Model: "sequential"
_________________________________________________________________
Layer (type)                 Output Shape              Param #
=================================================================
conv2d (Conv2D)              (None, 84, 84, 2)         52
_________________________________________________________________
max_pooling2d (MaxPooling2D) (None, 16, 16, 2)         0
_________________________________________________________________
```

```
conv2d_1 (Conv2D)            (None, 14, 14, 4)         76
_________________________________________________________________
max_pooling2d_1 (MaxPooling2 (None, 7, 7, 4)           0
_________________________________________________________________
flatten (Flatten)            (None, 196)               0
_________________________________________________________________
dense (Dense)                (None, 2)                 394
=================================================================
Total params: 522
Trainable params: 522
Non-trainable params: 0
_________________________________________________________________

Compiling model...

Training model…
```

Image Classification

# What Do Filters Learn?

We just trained our neural network to classify chest X-rays. Now, let's peek at what our different filters actually learn. There are a few ways to visualize the internal workings of our model. When working with convolutional networks, one of the most common approaches is to generate *feature maps*: the result of convolving a single filter across our input. Feature maps allow us to see how our network responds to a particular image in ways that are not always apparent when we only examine the raw filter weights.
For example, consider this x-ray, which our model correctly classifies as Pneumonia:

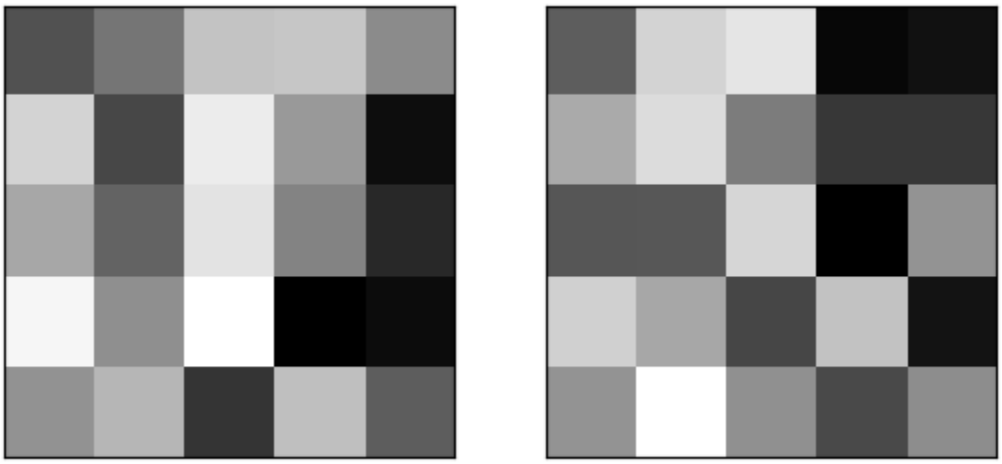



Our first layer generated the two feature maps (one for each filter) on the right.
**Instructions**

1. **Patterns**
   What patterns characterize each feature map? What aspects of the X-ray are visible in the first feature map? What about the second?

**Click to find out!**
The first filter appears to capture the texture of the lungs. In contrast, the second filter primarily responds to the vertical edges of the ribcage.

**Learned Filters**
And here are the filters that produced each feature map. Darker squares correspond to more negative weights, while whiter squares are more positive:

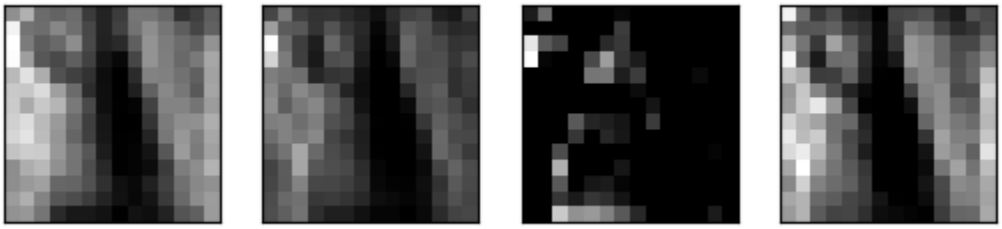


Is it possible to connect these raw weights to their resulting feature maps?
**Click to find out!**
It is exceedingly hard to connect the raw filter weights to the resulting feature maps. However, there is at least one notable characteristic.

Do you see the darkened right side on both filters (especially the second)? This contrast accounts for why the filters produce large values along the vertical edges of the ribcage.

As each filter moves off the edge of the ribcage, it encounters the dark background. When the dark side of the filter is over this dark background, its negative weights are canceled out, resulting in a brighter activation.

**First vs. Second Layer Features**
Now, compare the feature maps generated by our first layer to the feature maps in our second:

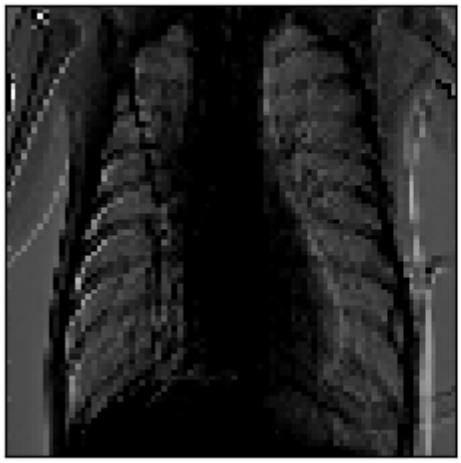


Why are there more feature maps?
What seems to change as we go deeper into the network?

**Click to find out!**
Our second layer has four feature maps, because we used four filters.
In the first layers, our filters tend to pick up on lower-level features like vertical and horizontal edges. As we move deeper into the network, we generally see the image representation becoming more blurry, abstract, and less interpretable (for us *humans*).

**Community Support**
Still have questions? Get help from the **Codecademy community**.

## Image



Back
Next
Image Classification: Review

Image Classification

# Review

**That's a wrap!**
In this lesson, we have covered:

- How to use Keras to preprocess and load image data.
- How we can use Convolutional Neural Networks to classify images, and why these models outperform feed-forward models based on linear layers.
- How we can adjust the filter dimensions, stride, and padding of a convolutional layer, and how these hyperparameters affect output size.
- How we can use pooling methods to further reduce the size of our hidden layers and also gain some spatial invariance properties.
- Examples of the patterns learned by different convolutional filters.

# Review: Classification

**You've developed deep learning skills for classification.**

# Classification

## Goals of this Unit

Congratulations! The goal of this unit was to apply the deep learning concepts you have learned to perform classification models. You have worked with a wider range of types of data, including image data, and you now have more deep learning concepts at your disposal. With these in hand, you are prepared to dive into more projects on your own to show off your arsenal of accomplishments.

Having completed this unit, you are now able to:

- Use your deep learning knowledge to perform classification models
- Work with different types of data, including images
- Increase your deep learning skillset to tackle a wider array of projects

Happy coding!

# Introduction: Deep Learning in the Real World

**See what's covered in Deep Learning in the Real World**

## Deep Learning in the Real World

### Goals of this Unit

The goal of this unit is to take a deep dive into real-world applications of deep learning. You will see new types of deep learning models and learn how to choose the best type of neural network for a given problem. With knowledge of these new models added to your deep learning tool belt, you will walk away with the ability to analyze any problem involving lots of data and brainstorm different solutions you could apply.

After this unit, you will be able to:

- Add an arsenal of neural network types to your deep learning tool belt
- Understand various ways deep learning models are applied in the real-world
- Develop the beginning-to-end problem-solving skills needed to apply a neural network to a given dataset

Happy coding!

# Common Applications of Deep Learning

**This article reviews some of deep learning's common applications.**

## Introduction

So far, we have gone from single-layer neural networks to multi-layer models with many hidden layers. We have also reviewed how these neural networks can serve as powerful tools for both classification and regression tasks. However, at this point, we still have only just scratched the surface; pushing beyond image classification, or simple regression use cases, deep learning approaches are now ascendant in fields ranging from cybersecurity and transportation to games and robotics.

Critically, deep learning hinges on one fundamental idea: given a dataset and a loss function, descend the gradient. As a result, we can generalize neural networks to solve a variety of different problems. However, specific domains present both unique challenges **and** useful assumptions.

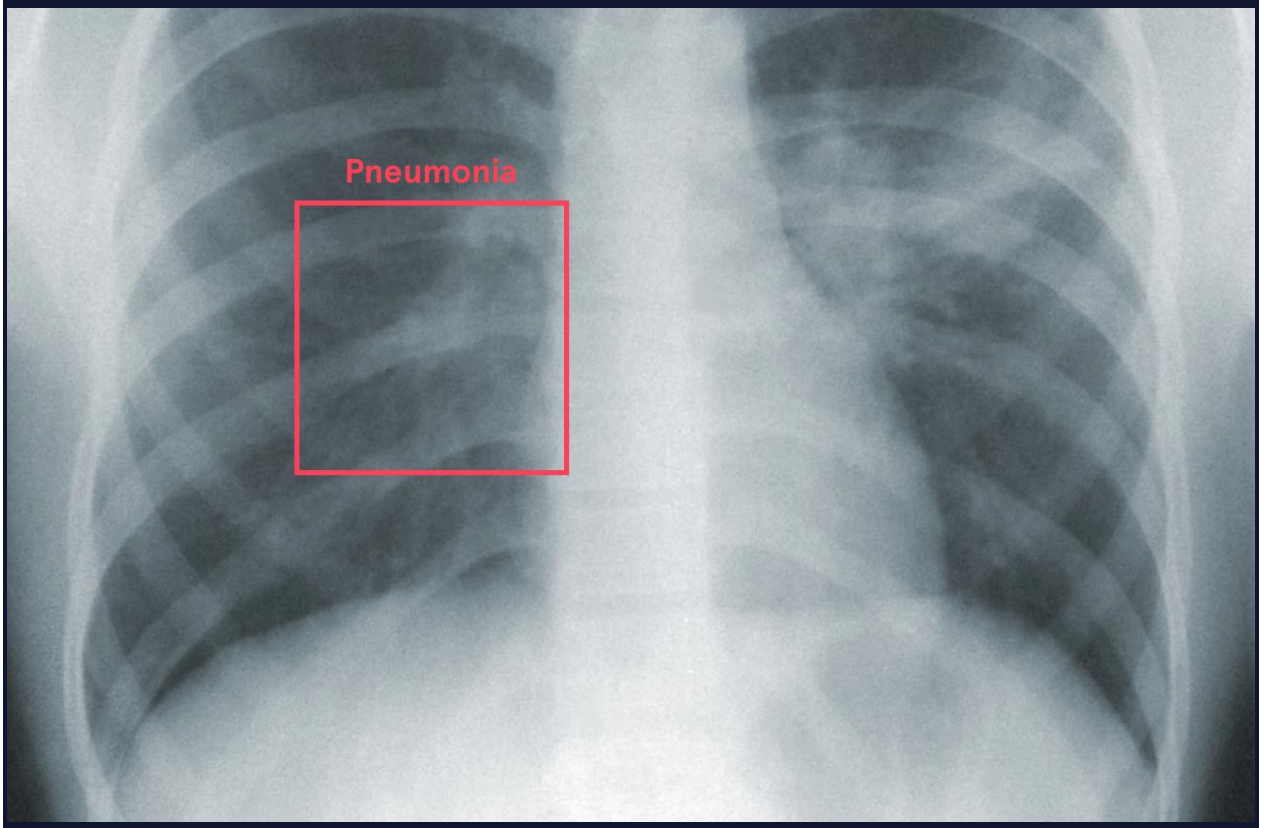
In this article, we will discuss many common applications for deep learning, and highlight how neural networks have been adapted to these respective tasks.

## Classification and Prediction in Challenging Domains

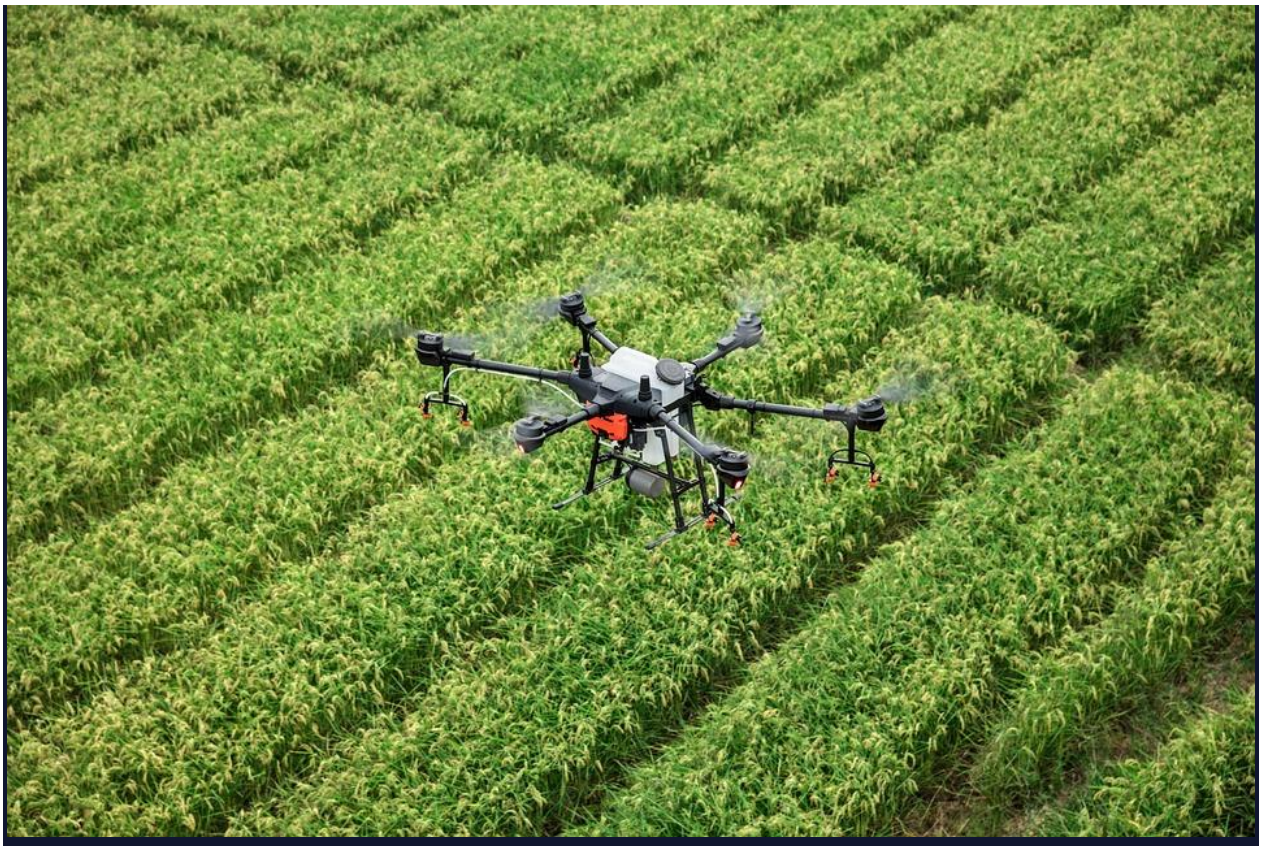
Neural networks excel at recognizing complex patterns in data, especially when that data is plentiful. It follows that deep learning is most commonly applied to datasets with many input features or where those features interact in complicated ways. As a result, neural networks have been wildly successful at tackling complex prediction and classification problems in domains including medicine and agriculture.

Consider the task of diagnosing a patient. While doctors may have an entire dossier of patient health records, only a select few of these data will be useful for making a correct prediction. Similarly, meaningful features can be extracted from combinations of raw data. For example, old age, a smoking habit, and a persistent cough together are much more predictive of lung cancer than those three features in isolation.

Because neural networks fantastically perform feature selection and extraction, they have successfully tackled many problems in medical subdomains, and show promise in others. Practitioners use deep learning to detect abnormalities in medical scans, predict health outcomes, or even combine clinical notes and medical codes to make a diagnosis.




Agronomics, the science of agriculture, is another field where deep nets are blossoming. Agriculture represents a precarious environment: harvests can be derailed by disease, drought, or plant infestations, which emerge from the complex interactions between crops, soil, and climate. In this setting, neural networks can provide powerful forecasting and classification tools to monitor and identify these threats. Additionally, neural networks can provide tools to assist farmers with scaling-up production.



Neural networks can ingest the rich data coming from sources like moisture sensors, drone footage, satellite imagery, and climate data, and extract relevant features for soil management, crop yield prediction, classification of plant species, livestock monitoring, and drought forecasting.

## Sequential Data

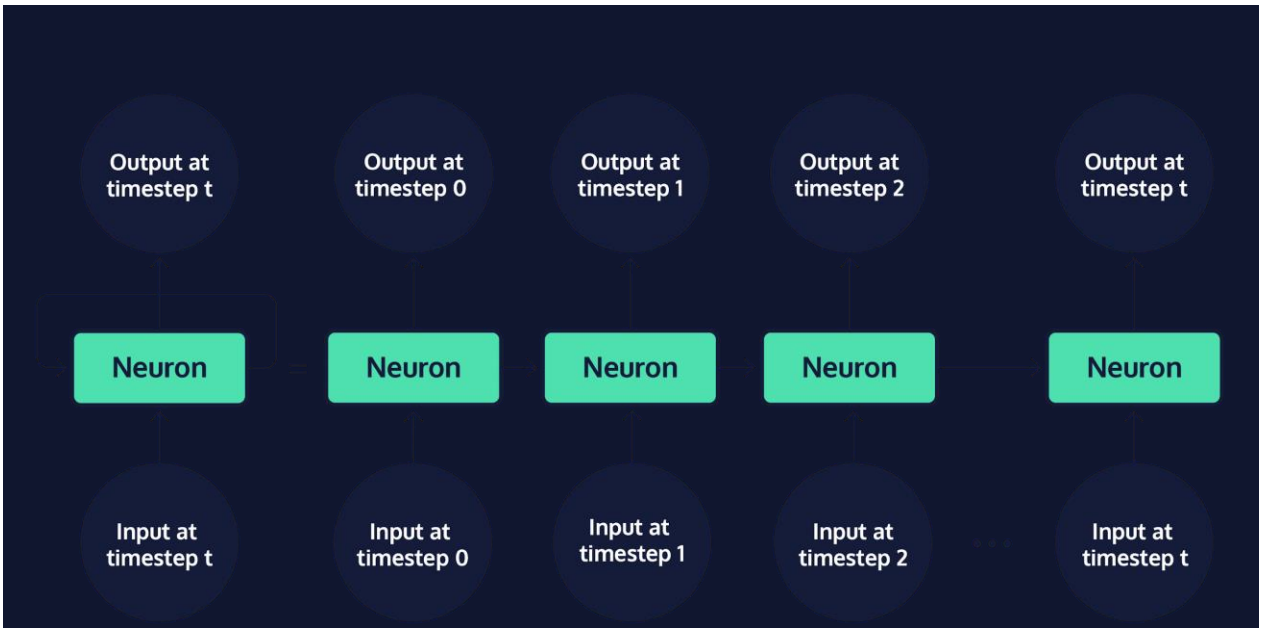
Many different tasks can be described as: “Given a sequence of data, how can we predict its next item(s)?” For example, consider the task of predicting the next word in a sentence. In computer science, this specific task is referred to as *language modeling*.

Language generation has undergone exceptional progress since the inception of deep learning. This has largely been because there is **a lot** of language data available (from the complete works of Shakespeare to our text messages) and because the rules of language are very complex (so alternative approaches to deep learning are hard to come by).

| | |
|---|---|
| SYSTEM PROMPT (HUMAN-WRITTEN) | *In a shocking finding, scientist discovered a herd of unicorns living in a remote, previously unexplored valley, in the Andes Mountains. Even more surprising to the researchers was the fact that the unicorns spoke perfect English.* |
| MODEL COMPLETION (MACHINE-WRITTEN, 10 TRIES) | The scientist named the population, after their distinctive horn, Ovid's Unicorn. These four-horned, silver-white unicorns were previously unknown to science.<br><br>Now, after almost two centuries, the mystery of what sparked this odd phenomenon is finally solved.<br><br>Dr. Jorge Pérez, an evolutionary biologist from the University of La Paz, and several companions, were exploring the Andes Mountains when they found a small valley, with no other animals or humans. Pérez noticed that the valley had what appeared to be a natural fountain, surrounded by two peaks of rock and silver snow.<br><br>Pérez and the others then ventured further into the valley. "By the time we reached the top of one peak, the water looked blue, with some crystals on top," said Pérez.<br><br>Pérez and his friends were astonished to see the unicorn herd. These creatures could be seen from the air without having to move too much to see them – they were so close they could touch their horns.<br><br>While examining these bizarre creatures the scientists discovered that the creatures also spoke some fairly regular English. Pérez stated, "We can see, for example, that they have a common 'language,' something like a dialect or dialectic." |

(Source: OpenAI: Better Language Models and their Implications)

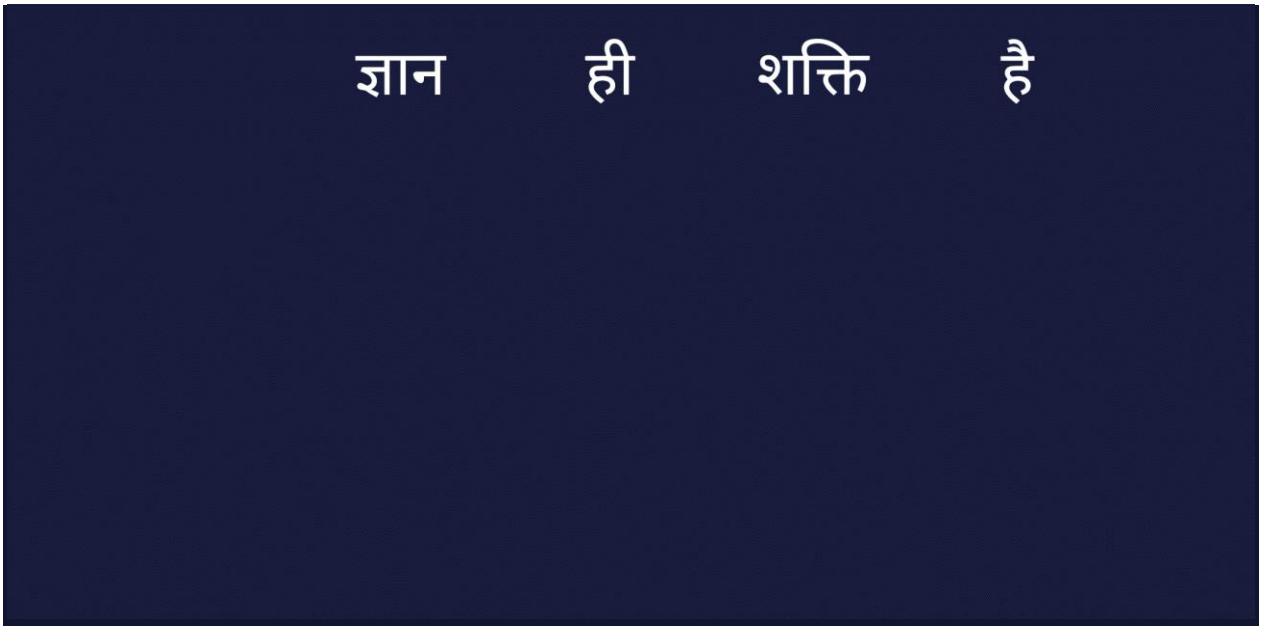
Some of the neural network architectures that perform the best on language modeling tasks make use of language's sequential nature. One of the most widely applied models for sequential data is called a ***Recurrent Neural Network (RNN)***.



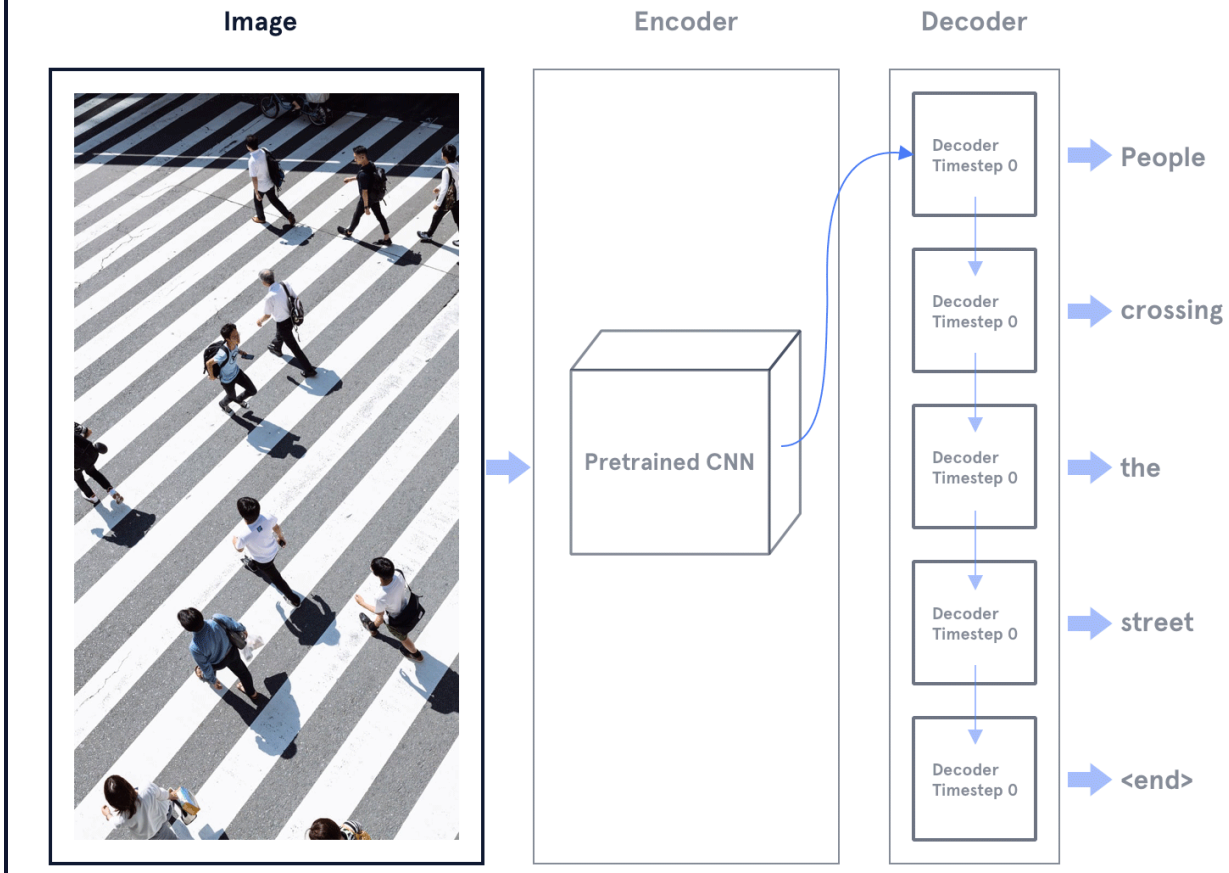

Rather than just concatenating our input words together, RNNs process them sequentially. For every input word in order, we pass that word to our model. At each timestep, the input is then used to update the model's hidden state. If we just want to predict the last word, we simply feed all but the last word into the model, then use the final hidden state to predict the next word. Alternatively, if we want to generate the entire sentence, we can feed a starting word in, update the hidden state, generate a new word, then pass that back into the new model, and so on.

Of course, this approach isn't just useful for text generation. RNNs can be applied to sequential problems ranging from time series forecasting (like predicting stock prices), to even music generation.

Another popular use of neural networks is for translation between sequences. For example, to translate from Hindi to English, we can use one RNN to encode the Hindi sentence. Then can pass these encoded representations to another RNN, which decodes out the corresponding French sentence. This is called an *Encoder-Decoder* network.




Encoder-Decoder architectures are used for a variety of tasks, including language translation and summarization. We can even replace the RNN Encoder with a convolutional neural network, and utilize the resulting model for image captioning!

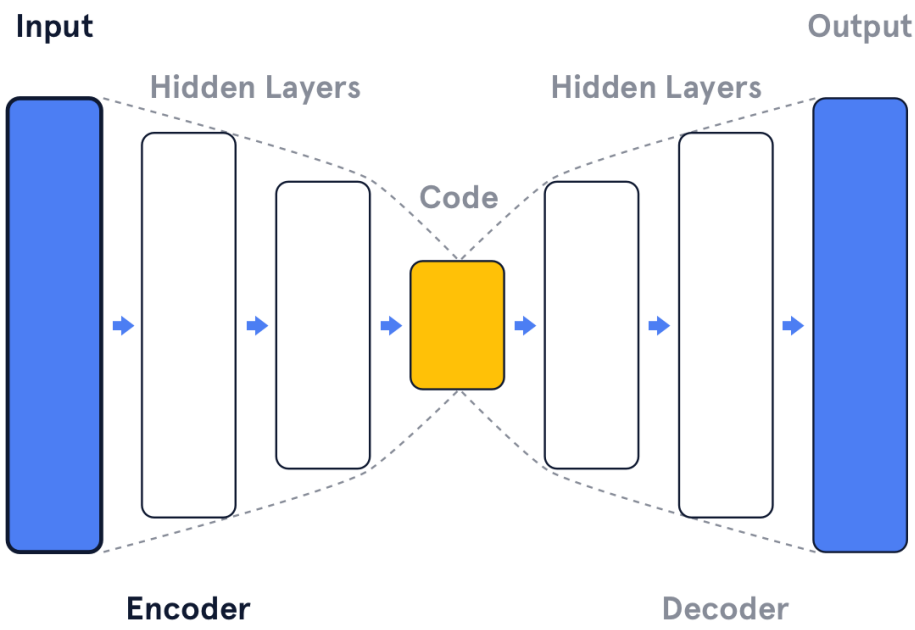



Any time we apply neural networks, we are sacrificing interpretability for effectiveness. This may be most true when working with sequential data, where entire sequences of information are combined into single vectors. This lack of interpretability raises a few problems: these models can secretly pick up on spurious correlations (false patterns that don't capture the true meaning of language), or even worse, secretly encode bias from the text datasets used to train these models.

## Autoencoders and Anomaly Detection

Let's imagine we train an Encoder-Decoder architecture to encode an image into a hidden state, then decode out that very same input. If we do this, that hidden, intermediate vector of information will learn to encode the information from that input necessary for re-generating that same picture. In other words, that single intermediate vector will store the "meaning" of the input data.

Now, what happens if we make the intermediate hidden state smaller? In this case, we must compress the information in our input further, while still trying to preserve the features that matter. In order to do this, the encoder must also throw away features that don't matter.

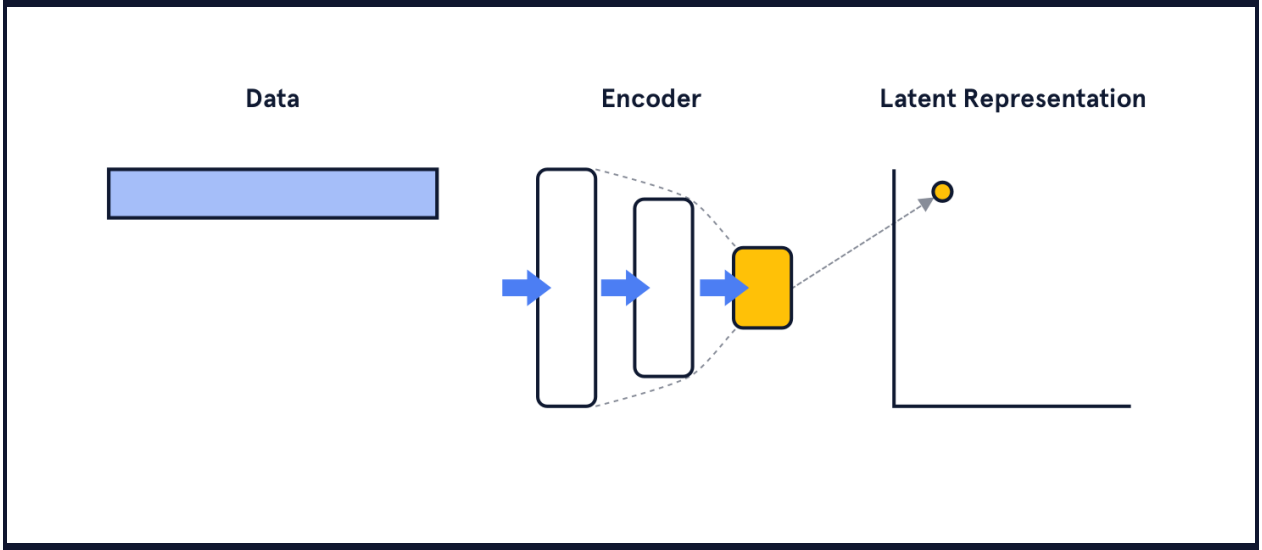



This is the big idea of *autoencoders*:

So far in this course, we have focused on *supervised learning* approaches, where we train our model to map an input to a label. Autoencoders are our first example of *unsupervised learning*: approaches where we learn the patterns and structure of our data without labels.

Autoencoders have many uses. For one, they can be used as a preprocessing step, to compress documents and images. These smaller vectors can also be used in downstream tasks like classification, clustering, or information retrieval.

Without any additional labeled data, autoencoders can also be used for *anomaly detection*: the identification of rare, or suspicious data points (e.g. fake documents or credit card fraud).




As we noted earlier, autoencoders throw away information not needed to reconstruct regular training data. As a result, if we give the autoencoder an anomalous data point as input, it will be very different from the average training data, and will be harder to reconstruct. This means that your model will have a higher reconstruction error! This approach is used to accurately detect anomalies in datasets ranging from accounting data to brain scans.

## Reinforcement Learning

For many games, the best player in the world isn't human. DeepMind's AlphaZero dominates the best human players in both Chess and Go. In Atari, *Deep Q Learning* has produced agents that are as good — or better — than any human gamer.

In the reinforcement learning framework, an agent takes actions in an environment and receives rewards. These are positive when the agent does good things (e.g. scores a point) and negative when the agent does bad things (e.g. loses health or dies). Using neural networks, combined with reinforcement learning loss functions, we can teach agents complex behaviors from scratch.

https://youtu.be/Lu56xVlZ40M

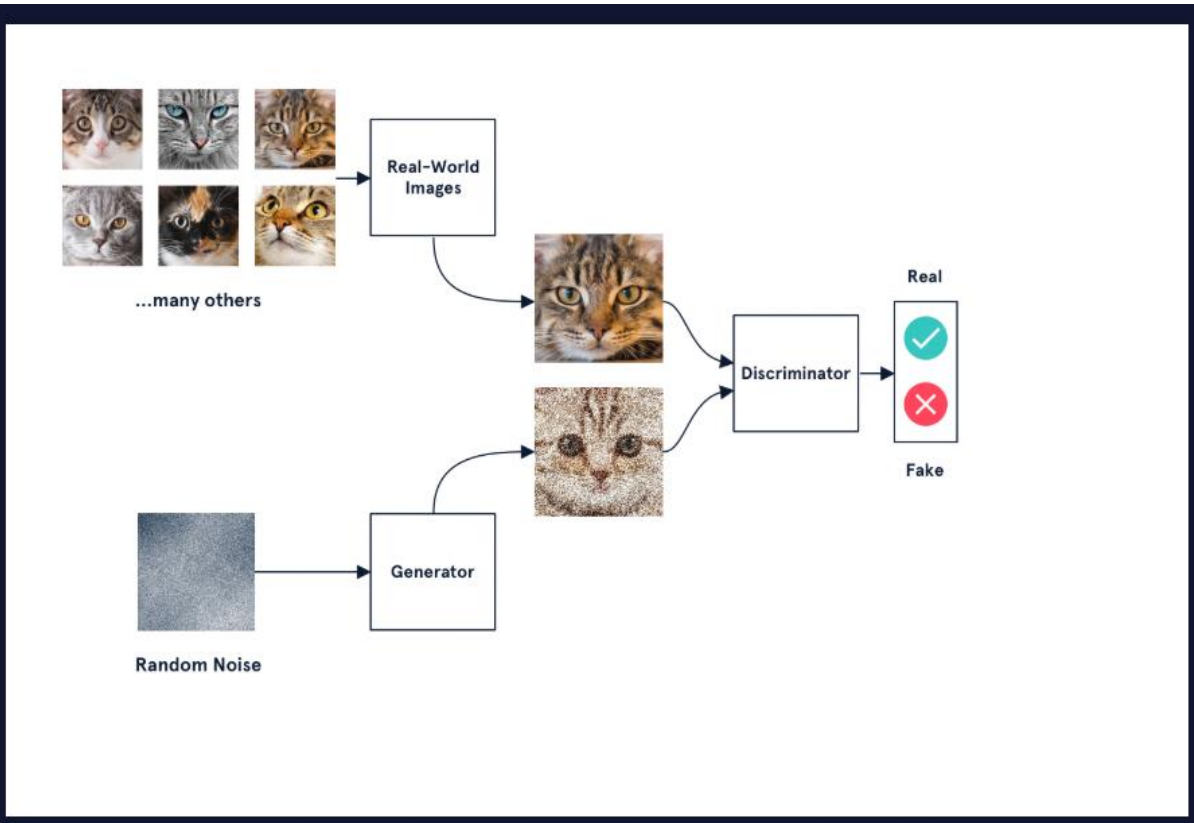
Simulation environments are bridging the divide between these games and real-world applications. For example, deep learning models are being trained via reinforcement learning to drive cars in a virtual setting. These models can then be fine-tuned in the real world!

## GANs

Let's say we want a model that generates pictures of cats. One possibility would be to use reinforcement learning. For example, we could have a human give our model a positive reward or negative reward: positive for good cat generations, negative for bad cat generations. However, getting those human labels would reduce training to a snail's pace.

Alternatively, what if we train another model to determine whether our network is generating cat-like photos? Now, rather than just a cat-generator network, we introduce a real cat/generated cat classifier, called a *discriminator network*, and task this model with sorting out generated cats from real ones. This other model can then provide the training signal for the generator.

That's the big idea behind *Generative Adversarial Networks (GANs)*.




More formally, we train a *generator network* to generate images (by constantly fine-tuning its parameters) that will be classified as 'real' by a discriminator network. At the same time, we train our discriminator to take the latest generations from our generator, along with real images, and to classify them as real or fake.

Here's how it all fits together, in the case of "cat generation":

- The generator network takes in a random noise vector, and transforms it into a candidate cat image.
- The discriminator network is fed both generated images and real images.
- The discriminator is trained to differentiate generated cats from real cats.
- The generator tries to maximize how much it fools the discriminator.

And voila! GANs.

Excitingly, GANs can work very well, with no hand-engineered features. However, it's important to note that training can be finicky, and selecting the right hyper-parameters is difficult. Sometimes your

discriminator will get too good (and the generator won't learn), and sometimes your generator will learn to only generate a single cat (mode collapse).

However, GAN-approaches have proved wildly successful, from image editing to face generation and style transfer. GANs have also been used to generate additional samples to augment existing datasets, including in medical domains.

https://youtu.be/AmUC4m6w1wo

Yet, GANs also introduce several ethical challenges. First, they have more sinister applications. Researchers have used GANs to generate seemingly genuine footage and audio of politicians, stoking fears that GANs offer dangerous tools for production of fake news content: so called Deep Fakes.

There are other, less explicit dangers to GANs. Because generators are training to replicate existing data, they can reproduce, or exacerbate biases in those data sets. For example, researchers found that Snapchat's GAN-based filters, trained on imbalanced data, regularly whiten the skin of users.

In sum, GANs are representative of both the power and perils of neural networks. As deep learning practitioners, we should not only appreciate the utility of our models, but also be wary of the implications of our work.

## Conclusion

In this article, we have discussed how neural networks are applied to hard classification and prediction problems, tasks involving sequential data, unsupervised anomaly detection, reinforcement learning, and GANs.

These applications represent just a sampling of the ever-developing, vibrant field of deep learning.

# Selecting the Best Neural Network

https://youtu.be/m_tRayMFhRE

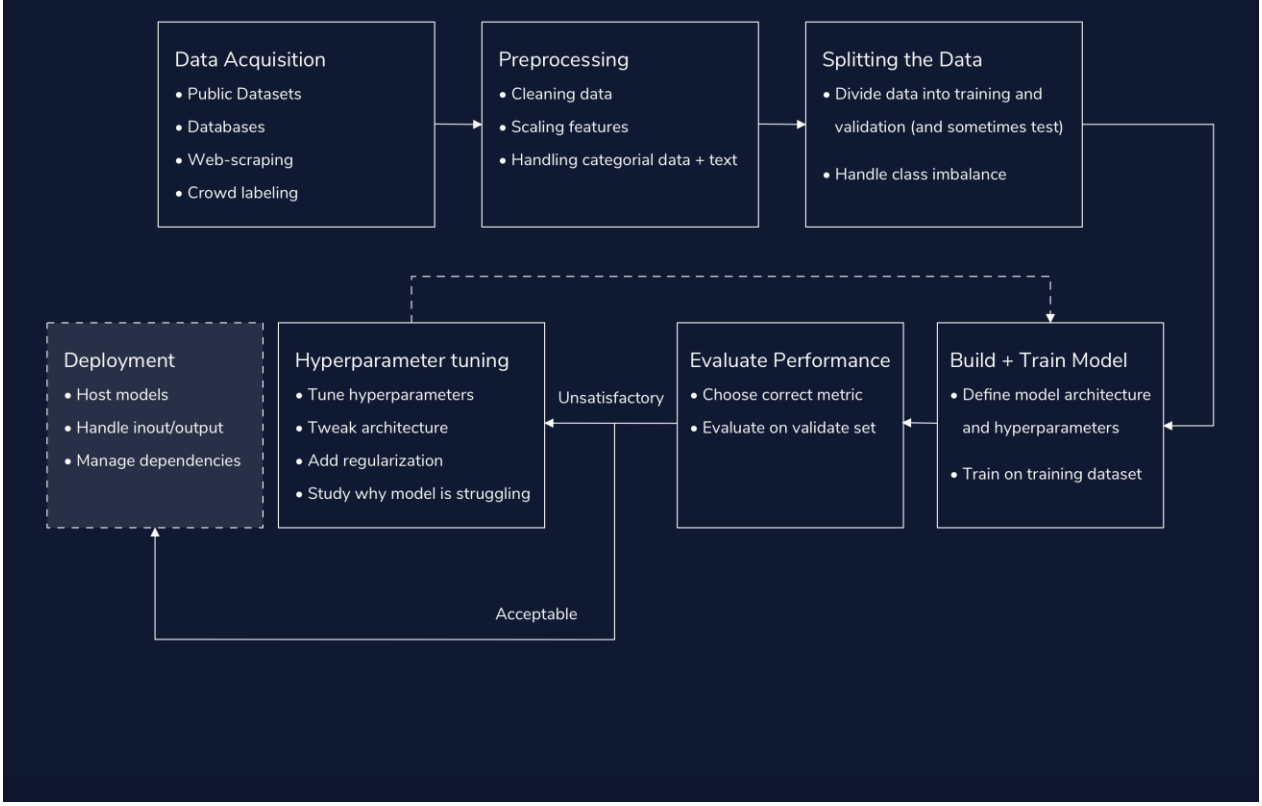
Now that you've learned about a few different deep learning architectures, find out how to select the right one for your goal.

# Deep Learning Workflow

**In this article, we cover the workflow for a deep learning project.**

## Introduction

Successfully using deep learning requires more than just knowing how to build neural networks; we also need to know the steps required to apply them in real-world settings effectively.



In this article, we cover the workflow for a deep learning project: how we build out deep learning solutions to tackle real-world tasks. The deep learning workflow has seven primary components:

# Part 1: Acquiring Data

In a deep learning project, the most pressing concern is almost always: “Can we get enough labeled data?” The more labeled data we have, the better our model can be. Our ability to acquire data can make or break our solution. Not only is getting data usually the most important part of a deep learning project, but it’s also often the hardest.

Luckily, there are many potential data sources:

## Publicly Available Datasets

The best source of data is often publicly available datasets. Sites like Kaggle host thousands of large, labeled data sources. Working with these curated datasets helps reduce the overhead of starting a deep learning project.

## Existing Databases

In some cases, our organization may have a large dataset on hand. Often, these datasets are stored in a Relational Database Management System (RDMS). In this case, we can build our specific dataset using SQL queries.

## Web scraping/APIs

Online news, social media posts, and search results represent rich streams of data, which we can leverage for our deep learning projects. We do this via Web scraping: the extraction of data from websites. While scraping and collecting data, we should keep in mind ethical considerations, including privacy and consent issues. There are many tools to web scrape in Python, including BeautifulSoup. Many sites, like Reddit and Twitter, have Python *Application Programming Interfaces* (APIs). We can use APIs to gather data from different applications. While some APIs are free, others are paid services.

Depending on the size of the dataset, we may be able to directly write our scraped data to raw data files (e.g., .txt or .csv). However, for larger datasets, we sometimes need to store the resulting data in our own databases.

### Crowd-sourced Labeling

For many tasks, it's much easier to acquire data than it is to find labeled data. For example, it is much easier to scrape the raw text from an entire Reddit subreddit than to correctly label the contents of each Reddit post. When automated labeling tools aren't available, we require human labels. One possibility would be for us to go through our own data, and annotate each datapoint ourselves. However, sometimes that just isn't feasible, no matter how much coffee we have on hand. An alternative is crowd-sourcing sites like Amazon Mechanical Turk. We can utilize these sites to pay "gig" workers for thousands of human annotations.

## Part 2: Preprocessing

Once we have built our dataset, we need to preprocess it into useful features for our deep learning models. At a high-level, we have three primary goals when preprocessing data for neural networks: we want to 1) clean our data, 2) handle categorical features and text and 3) scale our real-valued features using normalization or standardization techniques. Preprocessing is also a fantastic opportunity to become more familiar with our data.

### Cleaning data

Often, our datasets contain noisy examples, extra features, missing data and outliers. It is good practice to test for and remove outliers, remove unnecessary features, fill-in missing data, and filter out noisy examples.

### Scaling features

Because we initialize neural networks with small weights to stabilize training, our models will struggle when faced with input features that have large values. As a result, we often scale real-valued features in two ways: We can *normalize* features so that they are between 0 and 1, and *standardize* them so they have a mean of zero and a variance of one.

### Handling categorical data and text

Neural networks expect numbers as their inputs. This means we need to convert all categorical data and text to real-valued numbers. - We usually handle categorical variables by assigning each option its own unique integer or converting them to one-hot encodings. - When working with strings of raw text, we need to handle a few extra processing steps before encoding our words as integers. These steps include tokenizing our data (splitting our text into individual words/tokens), and *padding* our data (adding padding tokens to make all of our examples the same length).

## Part 3: Splitting and Balancing the Dataset

Once we have processed our data, it's time to split our dataset. Generally, we split our data into two datasets: training and validation. In certain cases, we also create a third holdout dataset, referred to as the test set. When we don't do this, we often use the terms "validation" and "test" sets interchangeably.

We train our model on the training dataset and we evaluate it on the validation dataset. If we have defined a third holdout test set, we test our model on this dataset after we have finished selecting our model and tuning our hyperparameters. This third step helps us avoid choosing a set of hyperparameters that only happen to work well on the data we chose for our validation set.

When splitting our dataset, there are two major considerations: the size of our splits, and whether we will stratify our data. After we split our data, we need to address imbalances in our training set.

## Splitting Our Data

We usually save 10-30% of our data for validation and testing. When we have a smaller corpus, it is more important to assign a larger proportion of data to the validation set. This helps ensure that our validation dataset better represents the true distribution of our data.

Scikit-learn provides the `train_test_split` function, which splits our data into training and validation datasets and specifies the size of our validation data.

## Stratified Train-Test Splits

We have to be extra careful when splitting a very imbalanced dataset for classification; it's very possible that more instances of our minority classes end up in either the training or the validation set. In the first case, our validation metrics will not accurately capture our model's ability to classify the minority class. In the second case, the model will overestimate the probability of the majority class.

The solution is to use a stratified split: a split that ensures the training and validation sets have the same proportion of examples from each class.

If we set the `train_test_split` function's `stratify` parameter to our array of labels, the function will compute the proportion of each class, and ensure that this ratio is the same in our training and validation data.

## Handling Imbalanced Data

Imbalanced data, where some classes appear much more than others, pose a challenge for deep learning models. If we train neural networks on imbalanced data, our resulting model will be heavily biased towards predicting those majority classes. This is especially problematic

because usually we care much more about identifying instances of the minority classes (like rare cases of disease or credit fraud).

There are two main approaches to dealing with imbalanced training data: undersampling and oversampling. These two approaches should be taken-up with utmost caution, and it's best to have a domain expert on hand to weigh in.

- In undersampling, we balance our data by throwing out examples from our majority class.
- In oversampling, we duplicate instances of our minority class so that they occur more often. A popular alternative to traditional oversampling is called *Synthetic Minority Oversampling TEchnique (SMOTE)*. The SMOTE algorithm creates synthetic examples that are similar to those in our minority class, and adds them to our dataset.

Almost always, we only correct the imbalance in our training data, and leave the validation data as is. In order to only augment our training data, we need to correct for imbalance only after our train-test split.

We never oversample our data before we split it. If we do, copies of our testing data can sneak into our training data. This is called *information leak*.

## Part 4: Building and Training the Model

Once we have split our dataset, it's time to choose our loss function, and our layers.

For each layer, we also need to select a reasonable number of hidden units. There is no absolute science to choosing the right size for each layer, nor the number of layers — it all depends on your specific data and architecture.

It's good practice to start with a few layers (2-6).
Usually, we create each layer with between 32 and 512 hidden units.
We also tend to decrease the size of hidden layers as we move upwards - through the model.

We usually try SGD and Adam optimizers first.

```
0.01
```

## Part 5: Evaluating Performance

Each time we train the model, we evaluate its performance on our validation set. When we provide a validation set at training time, Keras handles this automatically. Our performance on the validation set gives us a sense for how our model will perform on new, unseen data.

When considering performance, it's important to choose the correct metric. If our data set is heavily imbalanced, accuracy (and even AUC) will be less meaningful. In this case, we likely want to consider metrics like precision and recall. F1-score is another useful metric that combines both precision and recall. A confusion matrix can help visualize what data-points are misclassified and what aren't.

## Part 6: Tuning Hyperparameters

We will almost always need to iterate upon our initial hyperparameters. When training and evaluating our model, we explore different learning rates, batch sizes, architectures, and regularization techniques.

As we tune our parameters, we should watch our loss and metrics, and be on the lookout for clues as to why our model is struggling.:

- Unstable learning means that we likely need to reduce our learning rate and or/increase our batch size.
- A disparity between performance on the training and evaluation sets means we are overfitting, and should reduce the size of our model, or add regularization (like dropout).
- Poor performance on both the training and the test set means that we are underfitting, and may need a larger model or a different learning rate.

A common practice is to start with a smaller model and scale up our hyperparameters until we do see training and validation performance diverge, which means we have overfit to our data.

Critically, because neural network weights are randomly initialized, your scores will fluctuate, regardless of hyperparameters. One way to make accurate judgments is to run the same hyperparameter configuration multiple times, with different random seeds.

Once our results are satisfactory, we are ready to use our model!

If we made a holdout test set, separate from our validation data, now is when we use it to test out our model. The holdout test set provides a final guarantee of our model's performance on unseen data.

## Part 7: Deployment (For Industry)

Once we have trained a model, we may want to deploy it into the real world. This is especially true in industry settings, when our networks will be used by our coworkers and customers, or working behind the scenes in our products and internal tools.

When deploying a neural network there are three big considerations…

### How will we handle the compute requirements for running our models?

It takes a significant amount of computation to evaluate a single input using a neural network, let alone manage traffic from many different users. As a result, when deploying a neural network model in a Docker container, it's important to host the container where it can access powerful computing resources. Cloud platforms like AWS, GCP and Azure are great places to start. These platforms provide flexible hosting services for applications that can scale up to meet changing demand.

### How will we pass inputs into the model?

A common approach for interfacing with our model over the web is Flask, a Python-based web framework. Flask can handle requests and pass inputs to our model.

### How will we run the code and manage dependencies, wherever we host our application? (Optional)

This can depend on where we host our model. However, a popular general-purpose solution to this last question is Docker Containers. Docker containers are a way to package up our code and its dependencies (e.g. the correct version of TensorFlow), in such a way that our application can run quickly in any computing environment.

## Conclusion

In this article, we covered the general workflow for a deep learning project. We covered a lot of material, so don't sweat every detail. Our goal is to provide a sense for the overarching flow of a deep learning project, from data acquisition and preprocessing to evaluation and hyperparameter tuning.

These guidelines are not completely ironclad rules. Rather, in a successful deep learning project, we often pivot back and forth between different steps, continuously tweaking and debugging our scripts, data, and architectures on our quest for the best performing model.

# Review: Deep Learning in the Real World

**Review what's covered in Deep Learning in the Real World.**

## Deep Learning in the Real World

### Goals of this Unit

Congratulations! The goal of this unit was to take a deep dive into real-world applications of deep learning. You have seen new types of deep learning models and learned how to choose the best type of neural network for a given problem. With knowledge of these new models added to your deep learning toolbelt, you now have the ability to analyze any problem involving massive datasets and brainstorm different solutions you could apply.

Having completed this unit, you are now able to:

- Add an arsenal of neural network types to your deep learning toolbelt

- Understand various ways deep learning models are applied in the real-world
- Develop the beginning-to-end problem-solving skills needed to apply a neural network to a given dataset

Happy coding!

In this project, you will use deep learning to predict forest cover type (the most common kind of tree cover) based only on cartographic variables. The actual forest cover type for a given 30 x 30 meter cell was determined from US Forest Service (USFS) Region 2 Resource Information System data. The covertypes are the following:

Spruce/Fir
Lodgepole Pine
Ponderosa Pine
Cottonwood/Willow
Aspen
Douglas-fir
Krummholz

Independent variables were then derived from data obtained from the US Geological Survey and USFS. The data is raw and has not been scaled or preprocessed for you. It contains binary columns of data for qualitative independent variables such as wilderness areas and soil type.

This study area includes four wilderness areas located in the Roosevelt National Forest of northern Colorado. These areas represent forests with minimal human-caused disturbances, so existing forest cover types are mainly a result of ecological processes rather than forest management practices.

**Project Objectives:**

- Develop one or more classifiers for this multi-class classification problem.
- Use TensorFlow with Keras to build your classifier(s).
- Use your knowledge of hyperparameter tuning to improve the performance of your model(s).
- Test and analyze performance.
- Create clean and modular code.

**Prerequisites:**

All lessons, articles, and previous projects in Build Deep Learning Models with TensorFlow

**Preprocess and explore the dataset**
Download the dataset, preprocess it, and conduct some exploratory data analysis to understand the data. See if any data can be removed or if you need to balance your data at all.

**Build and train your model**
Split your dataset for training, validation, and testing. Then create and train your model using TensorFlow with Keras.

**Adjust your hyperparameters**
Use your validation data to evaluate your model's performance. Most likely, your model isn't performing very well yet. That's okay!

Try adjusting various hyperparameters to see what combination(s) improves your model's performance:

- batch size
- optimizer type
- learning rate
- hidden units
- number of layers
- dropout parameters
- early stopping
- activation functions

Feel free to build, train, and tweak hyperparameters for a few different models that you can compare.

**Put your model to the test**
When you're satisfied with the performance(s) you're seeing, save and load your model(s). It's time to put your model(s) to the test with the test data you set aside earlier.

We recommend plotting performance to better understand whether the predictions are accurate or not. You might also want to create a classification report.

**Build a report**
Nice work! Now, to really build this into a portfolio piece, create a report to explain your findings.

If you compared multiple models, discuss why certain models may have performed better than others. Provide reasons as to why each of your models classifies certain cover-types well and others not so well.

- Provide some performance metrics.
- Also, include suggestions on how those mis-classifications could be improved.

# Final Wrap-up

**What will you do next?**

Congratulations on completing the Deep Learning Skill Path! Now that you have finished your learning journey, you may be asking, “What’s next?”. Here are our recommendations for next steps:

## Machine Learning

Deep learning is just one part of the machine learning universe. If you don’t already know the field well, you may want to explore more machine learning algorithms you can use to test against your deep learning models. We would recommend the Build a Machine Learning Model Skill Path to get started.

## Big Data

In this skill path, we played around with large datasets a lot. If this is some that interests you, you can look into big data. Big data is all about processing giant quantities of data. If this interests you, you may want to check out some common tools for this field, including Hadoop, Spark, and H2O.

## Natural Language Processing

Deep learning and NLP are interrelated, and you have touched upon it slightly in this skill path. There is still plenty to learn about it, and we have two options that may interest you. Check out Codecademy’s Natural Language Processing course and Build Chatbots with Python Skill Path if you would like to dive further into NLP.

## Data Science

Data science is a broad field of study that includes deep learning. If you are interested in pursuing more skills that work around refining and analyzing data, you should check out our Data Scientist Career Path.

Once again, congratulations on finishing your Career Path! We’re excited to see what you accomplish next.